

算法竞赛知识库

ICPC/CCPC 高频算法与数据结构体系化整理

算法学习笔记

2026 年 5 月 9 日

摘要

本文档系统整理了 ICPC（国际大学生程序设计竞赛）中高频出现的算法与数据结构，涵盖从基础到进阶的完整知识体系。根据对历年赛题的统计分析，约 80% 的题目都集中在核心算法与数据结构上。本文档将知识点划分为三个优先级梯队，并为每个算法提供详细的代码实现和应用说明。

目录

第一部分 ICPC 竞赛考点大纲	8
1 第一梯队：绝对核心，必须精通	8
2 第二梯队：高分必备，拉分关键	8
3 第三梯队：冲刺大神，决胜巅峰	8
4 学习路径建议	9
第二部分 基础算法专题	11
5 枚举	11
5.1 子集枚举（二进制枚举）	11
5.2 全排列枚举（DFS 回溯）	12
5.3 组合枚举	13
5.4 四平方和（枚举 + 哈希优化）	13
5.5 百钱百鸡（经典枚举优化）	15
5.6 回文日期	15

6 贪心	17
6.1 区间选点（最少点覆盖所有区间）	17
6.2 最大不相交区间数量	18
6.3 区间覆盖（最小区间数覆盖线段）	19
6.4 合并果子（哈夫曼编码）	20
6.5 排队接水（最小等待时间）	21
6.6 货仓选址（绝对值不等式）	22
7 二分 / 三分	23
7.1 整数二分查找（精确匹配）	23
7.2 二分查找左边界（第一个 $\geq x$ 的位置）	24
7.3 二分查找右边界（最后一个 $\leq x$ 的位置）	24
7.4 浮点数二分（求平方根/立方根）	25
7.5 二分答案——最小化最大值	26
7.6 三分查找（单峰函数极值）	27
7.7 二分查找旋转数组的最小值	28
8 前缀和与差分	30
8.1 一维前缀和——区间和查询	30
8.2 二维前缀和——子矩阵和查询	31
8.3 一维差分——区间增减操作	32
8.4 二维差分——子矩阵增减操作	32
8.5 前缀和——区间内特定值出现次数	33
9 排序	35
9.1 快速排序	35
9.2 归并排序	36
9.3 归并排序求逆序对数量	37
9.4 快速选择（求第 k 小的数）	38
9.5 计数排序	39
9.6 基数排序	40
第三部分 动态规划专题	42
10 线性 DP	42
10.1 最长上升子序列（LIS）	42
10.2 最长公共子序列（LCS）	43
10.3 最大子数组和（Kadane 算法）	45

10.4 最长回文子串	46
11 背包 DP	48
11.1 0-1 背包	48
11.2 完全背包	49
11.3 多重背包（二进制优化）	50
11.4 分组背包	51
12 区间 DP	53
12.1 石子合并	53
12.2 括号匹配	54
13 树形 DP	56
13.1 树的最大独立集（没有上司的舞会）	56
13.2 树的直径	57
13.3 树形背包	58
14 数位 DP	60
14.1 统计 1 的个数	60
14.2 不含连续 1 的数字	61
15 状态压缩 DP	63
15.1 TSP 问题（旅行商问题）	63
15.2 棋盘覆盖（铺砖问题）	64
16 概率 DP/期望 DP	66
16.1 掷骰子游戏	66
16.2 地牢游戏	67
17 DP 优化技巧	69
17.1 单调队列优化	69
17.2 斜率优化	70
18 DP 算法总结	72
 第四部分 数据结构专题	 73
19 初级数据结构	73
19.1 栈——括号匹配	73
19.2 栈——单调栈求下一个更大元素	74

19.3 队列——滑动窗口最大值	74
20 高级数据结构	75
20.1 树状数组——单点修改区间求和	75
20.2 线段树——区间最值与单点修改	76
20.3 平衡树 (Treap) ——排名查询与第 k 大	77
21 算法总结表	79
 第五部分 并查集算法专题	 80
22 并查集基础	80
22.1 标准并查集 (路径压缩 + 按秩合并)	80
22.2 带权并查集 (维护到根节点的距离)	82
23 并查集扩展应用	85
23.1 连通块统计	85
23.2 并查集判环 (无向图)	87
23.3 并查集求最小生成树 (Kruskal 算法)	88
24 并查集算法复杂度总结	91
 第六部分 数学与思维题算法专题	 92
25 基础数学思维	92
25.1 试除法判断质数	92
25.2 埃拉托色尼筛法	93
25.3 线性筛 (欧拉筛)	94
25.4 最大公约数 (辗转相除法)	94
25.5 最小公倍数	95
26 高级数学算法	96
26.1 快速幂	96
26.2 扩展欧几里得算法	97
26.3 乘法逆元 (费马小定理)	98
26.4 组合数 (阶乘逆元法)	98
26.5 博弈论——Nim 游戏	99
26.6 博弈论——SG 函数	100

27 数学算法总结	101
 第七部分 图论算法专题	 102
28 图的存储与基础遍历	102
28.1 深度优先搜索 (DFS)	102
28.2 广度优先搜索 (BFS)	103
29 最短路算法	105
29.1 Dijkstra 算法 (堆优化)	105
29.2 Floyd-Warshall 算法	106
30 最小生成树	108
30.1 Prim 算法	108
31 拓扑排序	110
32 强连通分量与缩点 (Tarjan)	112
33 最近公共祖先 (LCA 倍增法)	115
34 二分图匹配 (匈牙利算法)	118
35 网络流 (Dinic 最大流算法)	120
36 图论算法总结	123
 第八部分 字符串算法专题	 124
37 字符串基础	124
37.1 字符串匹配 (暴力枚举)	124
37.2 字符串反转	125
37.3 字符串分割	126
38 字符串哈希	128
38.1 单哈希	128
38.2 双哈希	129
39 KMP 算法	131
39.1 字符串匹配	131
39.2 最小循环节	132

39.3 统计前缀出现次数	133
40 Trie 树（字典树）	135
40.1 字符串插入与查询	135
40.2 最大异或和（Trie 求异或极值）	137
41 Manacher 算法	139
41.1 最长回文子串	139
41.2 统计回文子串个数	140
42 扩展 KMP（Z 算法）	142
42.1 计算 Z 数组	142
42.2 字符串匹配（Z 算法）	143
43 AC 自动机	145
43.1 多模式串匹配	145
43.2 拓扑优化统计（避免重复跳 fail）	147
44 后缀自动机（SAM）	150
44.1 构建后缀自动机	150
44.2 最长公共子串（多字符串，SAM）	151
45 最小表示法	155
45.1 循环同构最小表示	155
46 字符串算法总结	157
 第九部分 计算几何算法专题	 158
47 基础运算	158
47.1 点积与叉积	158
47.2 点到直线距离	159
47.3 点到线段距离	160
48 线段相交	162
48.1 判断线段相交（规范相交）	162
48.2 判断线段相交（包括端点）	163
48.3 求线段交点坐标	164

49 凸包	166
49.1 Andrew 算法求凸包	166
49.2 判断点是否在凸多边形内	167
49.3 判断点是否在任意多边形内（射线法）	168
50 多边形面积	171
50.1 计算多边形面积（鞋带公式）	171
51 最近点对	172
51.1 分治法求最近点对距离	172
52 旋转卡壳	174
52.1 凸包直径（最远点对）	174
53 圆的相关计算	176
53.1 两圆交点	176
53.2 圆与直线交点	177
54 最小圆覆盖	179
54.1 随机增量法求最小圆覆盖	179
55 计算几何总结	181

第一部分 ICPC 竞赛考点大纲

1 第一梯队：绝对核心，必须精通

这一部分是竞赛的基石，几乎每场比赛都会出现，必须达到”肌肉记忆”级别的熟练度。

表 1: 第一梯队核心知识点

知识点分类	具体算法与数据结构	核心应用场景与说明
基础算法	枚举、贪心、二分/三分、前缀和与差分、排序	几乎所有题目都涉及的基础思想。用于最优化问题与区间处理。
动态规划 (DP)	线性 DP、背包问题、区间 DP、树形 DP、状态压缩 DP	”算法之王”，用于求解最优子结构问题，如资源分配、路径规划。
数据结构	栈、队列、并查集、哈希表	并查集是处理集合合并与查询的神器，代码短效率高。
图论	DFS/BFS、拓扑排序、最短路 (Dijkstra,Floyd)、最小生成树 (Kruskal)	图论是 ICPC 的”半壁江山”，用于解决网络连通、路径规划等问题。
数学	快速幂、GCD/扩展欧几里得、素数筛法、组合数学	很多问题本质是数学题，扩展欧几里得是解不定方程的必备工具。

2 第二梯队：高分必备，拉分关键

掌握这些能让你的队伍稳稳进入中上游，是解决 3-5 题的关键。

3 第三梯队：冲刺大神，决胜巅峰

这是通往顶尖选手（如 World Finalist）的必经之路，主要用于解决赛题中最难的 1-2 题。

- **DP 优化：**斜率优化、四边形不等式优化、单调队列优化。用于将 $O(N^2)$ 的 DP 降为 $O(N)$ 或 $O(N \log N)$ 。

表 2: 第二梯队进阶知识点

知识点分类	具体算法与数据结构	核心应用场景与说明
高级数据结构	线段树、树状数组、平衡树 (Treap/Splay)	区间操作的利器, 用于处理动态的区间求和、最值、修改问题。
图论进阶	强连通分量 (Tarjan)、LCA(倍增)、二分图匹配、网络流	处理复杂关系问题, 如社交网络圈、树上路径查询、分配问题。
字符串	字符串哈希、KMP、Trie、Manacher、后缀数组	用于文本匹配、模式检索、回文串处理等。
数学进阶	高斯消元、矩阵快速幂、容斥原理、博弈论 (SG 函数)	解决线性方程组、递推优化、计数问题和组合游戏。
计算几何	点积/叉积、判断线段相交、凸包	叉积是计算几何的灵魂, 用于处理几何图形相关问题。

- **高级图论:** 费用流、带花树 (一般图匹配)、Gomory-Hu 树。解决超复杂的网络流或匹配问题。
- **数学终极:** 快速傅里叶变换 (FFT/NTT)、线性基、莫比乌斯反演、概率与期望 DP。
- **复杂算法:** 模拟退火、舞蹈链 (DLX)、启发式搜索。通常用于求解 NP-hard 问题的近似解。

数据洞察: 根据一项针对近十年 ICPC 数据的分析, 一线选手在 Codeforces 上的 Elo 评分, 与其在 ICPC World Finals 上的最终排名具有强相关性 ($\tau = 0.596$)。这表明, 持续参加高强度的线上比赛是掌握这些算法的有效途径。

4 学习路径建议

面对庞大的算法体系, 建议遵循以下通用进阶路径:

1. **入门 (青铜):** 彻底掌握第一梯队。大量刷简单题, 达到看到题目立刻想到用哪种基础算法的程度。
2. **进阶 (白银 黄金):** 主攻第二梯队。针对性地进行专题训练, 例如花两周时间集中攻克线段树的各类题型。
3. **冲刺 (王者):** 拓展第三梯队。重点锻炼思维建模能力, 即如何将实际问题抽象成自己会做的算法模型。

总结：

先稳扎稳打拿下第一梯队，再逐步攻克第二、第三梯队。不必追求所有算法都精通，但要对核心、高频的算法做到心中有数、手到擒来。

第二部分 基础算法专题

5 枚举

枚举（Enumeration）是最基础的算法思想：遍历解空间中的所有候选解，逐一检验是否满足条件。适用于数据规模较小、无法或不需要推导数学公式的情形。核心在于控制枚举范围和剪枝优化，避免不必要的循环。

5.1 子集枚举（二进制枚举）

适用场景：从 n 个元素构成的集合中，枚举所有可能的子集（共 2^n 个），每个元素选或不选，适合 $n \leq 20$ 。

题目：给定一个含有 n 个整数的数组，输出所有子集的元素和。

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      int main() {
6          int n;                // n: 数组元素个数
7          cin >> n;
8          vector<int> a(n);      // a[i]: 第i个元素的值
9          for (int i = 0; i < n; i++) cin >> a[i];
10         // 枚举 [0, 2^n - 1] 之间的每一个二进制掩码
11         int total = 1 << n;    // total = 2^n, 子集总数
12         for (int mask = 0; mask < total; mask++) {
13             int sum = 0;        // sum: 当前子集的元素和
14             // 检查 mask 的每一位，第 j 位为 1 表示选择 a[j]
15             for (int j = 0; j < n; j++) {
16                 if (mask & (1 << j)) { // 判断 mask 的第 j 位是否为 1
17                     sum += a[j];
18                 }
19             }
20             cout << "subset " << mask << " sum = " << sum << endl;
21         }
22         return 0;
23     }
```

Listing 1: 二进制枚举所有子集

5.2 全排列枚举（DFS 回溯）

适用场景：生成 $1 \sim n$ 的所有排列，或对数组元素进行全排列后暴力验证，适合 $n \leq 10$ 。

题目：输出 $1 \sim n$ 的所有全排列。

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      int n;                // n: 排列长度
6      vector<int> path;      // path: 当前排列路径
7      vector<bool> used;     // used[i]: 数字 i 是否已被使用
8
9      // 原理：深度优先搜索，每次从未使用的数字中选一个加入当前路径
10     void dfs() {
11         if ((int)path.size() == n) { // 找到一个完整排列
12             for (int x : path) cout << x << " ";
13             cout << endl;
14             return;
15         }
16         for (int i = 1; i <= n; i++) { // 按字典序尝试每个数字
17             if (used[i]) continue;    // 已使用则跳过（剪枝）
18             used[i] = true;           // 标记使用
19             path.push_back(i);        // 加入路径
20             dfs();                    // 递归搜索下一层
21             path.pop_back();          // 回溯：撤销选择
22             used[i] = false;          // 回溯：恢复未使用状态
23         }
24     }
25
26     int main() {
27         cin >> n;
28         used.assign(n + 1, false);
29         dfs();
30         return 0;
31     }
```

Listing 2: DFS 回溯生成全排列

5.3 组合枚举

适用场景：从 n 个元素中选出恰好 k 个，需要枚举所有组合方案，适合 $n \leq 25$ 。

题目：从 $1 \sim n$ 中选出 k 个数，按字典序输出所有组合。

```

1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      int n, k;                // n: 总数, k: 选取个数
6      vector<int> comb;        // comb: 当前组合
7
8      // 原理: 每次从 start 开始选数, 保证组合内部递增, 避免重复
9      void dfs(int start) {
10         if ((int)comb.size() == k) { // 已选够 k 个, 输出
11             for (int x : comb) cout << x << " ";
12             cout << endl;
13             return;
14         }
15         // 剪枝: 剩余可选数字数量不足时提前返回
16         // 还需要选 k-comb.size() 个, 最多能从 n 选到 start
17         for (int i = start; i <= n - (k - (int)comb.size()) + 1; i++) {
18             comb.push_back(i);
19             dfs(i + 1);           // 下一层从 i+1 开始, 保证递增
20             comb.pop_back();      // 回溯
21         }
22     }
23
24     int main() {
25         cin >> n >> k;
26         dfs(1);
27         return 0;
28     }

```

Listing 3: DFS 枚举组合（字典序）

5.4 四平方和（枚举 + 哈希优化）

适用场景：将枚举从 $O(n^3)$ 或 $O(n^4)$ 优化到 $O(n^2)$ ，通过预处理一部分结果并用哈希表或数组存储，将“枚举前半 + 查找后半”的思想用于降低复杂度。

题目：每个正整数都可以表示为至多 4 个正整数的平方和。给定 $N \leq 5 \times 10^6$ ，求字典序最小的 a, b, c, d 使得 $a^2 + b^2 + c^2 + d^2 = N$ 且 $a \leq b \leq c \leq d$ 。

```

1      #include <iostream>
2      #include <cmath>
3      #include <cstring>
4      using namespace std;
5
6      const int MAXN = 5000010;          // MAXN: 根据 N 上限设定
7      int h[MAXN];                      // h[x]: 当两数平方和为 x 时, 较小的那个数
8
9      int main() {
10         int N;                        // N: 目标正整数
11         cin >> N;
12         memset(h, -1, sizeof(h));    // -1 表示未出现过
13         // 预处理: 枚举后两个数的平方和 c^2 + d^2
14         for (int c = 0; c * c <= N; c++) {
15             for (int d = c; c * c + d * d <= N; d++) { // d 从 c 开始保证 c<=
16                 d
17                 int sum = c * c + d * d;
18                 if (h[sum] == -1) { // 只记录第一次出现 (保证字典序最小)
19                     h[sum] = c;    // 存储较小的数 c, 较大数 d 可由 sum-c^2
20                                     算出
21                 }
22             }
23         }
24         // 枚举前两个数 a^2 + b^2, 查表匹配后两个数
25         for (int a = 0; a * a <= N; a++) {
26             for (int b = a; a * a + b * b <= N; b++) {
27                 int rest = N - a * a - b * b; // rest: 剩余需要凑的值
28                 if (rest < 0) break;
29                 if (h[rest] == -1) continue; // 无法凑出剩余值
30                 int c = h[rest];    // c: 查表得到的第三个数
31                 int d = sqrt(rest - c * c); // d: 由 rest - c^2 开方得到
32                 cout << a << " " << b << " " << c << " " << d << endl;
33                 return 0;          // 字典序最小, 找到即退出
34             }
35         }
36     }

```

Listing 4: 四平方和——枚举前两项 + 哈希查表

5.5 百钱百鸡（经典枚举优化）

适用场景：多元一次不定方程求整数解，通过减少变量个数将多重循环降为单重循环，适用于小数据范围的约束满足问题。

题目：公鸡 5 元一只，母鸡 3 元一只，小鸡 1 元三只。用 100 元买 100 只鸡，求所有购买方案。

```

1      #include <iostream>
2      using namespace std;
3
4      int main() {
5          // 原理：设公鸡 x 只，母鸡 y 只，小鸡 z 只
6          // x + y + z = 100, 5x + 3y + z/3 = 100 且 z 是 3 的倍数
7          // 由两式消去 z，得 7x + 4y = 100，枚举 x 即可确定 y, z
8          for (int x = 0; x <= 20; x++) { // x: 公鸡数, 5x<=100 故 x<=20
9              int y = (100 - 7 * x) / 4; // y: 母鸡数, 由方程解出
10             int z = 100 - x - y;      // z: 小鸡数
11             // 验证方程是否成立且 y,z 为非负整数
12             if (y >= 0 && z >= 0 && z % 3 == 0 && 5*x + 3*y + z/3 == 100) {
13                 cout << "cock=" << x << " hen=" << y << " chick=" << z << endl
14                 ;
15             }
16             return 0;
17     }
```

Listing 5: 百钱百鸡——消元降维枚举

5.6 回文日期

适用场景：枚举日期或回文数，通过生成而非逐一判定的方式缩小枚举范围。适用于日期/数字回文类问题。

题目：给定起始日期和终止日期（8 位数字），求在此范围内的回文日期个数。

```

1      #include <iostream>
2      using namespace std;
3
4      int days[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

```

5      // days[m]: 平年第 m 月的天数
6
7      // 判断是否为闰年
8      bool isLeap(int y) {
9          return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
10     }
11
12     // 判断日期是否合法
13     bool isValid(int y, int m, int d) {
14         if (m < 1 || m > 12) return false; // 月份非法
15         int maxd = days[m];                // maxd: 当月最大天数
16         if (m == 2 && isLeap(y)) maxd = 29; // 闰年二月有 29 天
17         return d >= 1 && d <= maxd;
18     }
19
20     int main() {
21         int start, end;                    // start: 起始日期, end: 终止日期
22         cin >> start >> end;
23         int ans = 0;                       // ans: 回文日期个数
24         // 原理: 枚举年份前四位, 翻转拼接得到完整回文日期, 再检查合法性
25         for (int y = start / 10000; y <= end / 10000; y++) { // y: 年份
26             int rev = 0;                   // rev: y 的翻转数
27             int tmp = y;
28             for (int k = 0; k < 4; k++) {
29                 rev = rev * 10 + tmp % 10;
30                 tmp /= 10;
31             }
32             int m = rev / 100;              // m: 月份 (翻转后的前两位)
33             int d = rev % 100;             // d: 日期 (翻转后的后两位)
34             int date = y * 10000 + rev;    // date: 完整回文日期
35             if (date >= start && date <= end && isValid(y, m, d)) {
36                 ans++;
37             }
38         }
39         cout << ans << endl;
40         return 0;
41     }

```

Listing 6: 回文日期——枚举前四位生成回文

6 贪心

贪心算法（Greedy）在每一步选择中都采取当前状态下最优的选择，期望最终全局最优。使用贪心的前提是问题具有**贪心选择性质**（局部最优能推出全局最优）和**最优子结构**。

6.1 区间选点（最少点覆盖所有区间）

适用场景：给定多个区间 $[l_i, r_i]$ ，在数轴上放置最少的点使得每个区间内至少有一个点。典型应用：雷达覆盖、传感器部署。

题目：给定 N 个闭区间，求最少需要选多少个点使得每个区间内至少包含一个选出的点。

```

1      #include <iostream>
2      #include <algorithm>
3      using namespace std;
4
5      const int MAXN = 100005;          // MAXN: 最大区间数
6      struct Range {
7          int l, r;                      // l: 左端点, r: 右端点
8          // 按右端点升序排序: 右端点越靠左, 留给后面区间的空间越大
9          bool operator<(const Range& other) const {
10             return r < other.r;
11         }
12     } ranges[MAXN];
13
14     int main() {
15         int n;                          // n: 区间个数
16         cin >> n;
17         for (int i = 0; i < n; i++) cin >> ranges[i].l >> ranges[i].r;
18         sort(ranges, ranges + n);      // 按右端点排序
19         int ans = 0;                    // ans: 所选点数
20         int last = -2e9;                 // last: 上一个选点的坐标 (初始为极小值)
21         // 原理: 每次选当前区间右端点, 能尽可能多地覆盖后续区间
22         for (int i = 0; i < n; i++) {
23             if (ranges[i].l > last) { // 当前区间未被上一个点覆盖
24                 ans++;                // 新增一个点
25                 last = ranges[i].r;   // 点选在当前区间右端点
26             }
27     }

```

```

28         cout << ans << endl;
29         return 0;
30     }

```

Listing 7: 贪心——按右端点排序，每次选右端点

6.2 最大不相交区间数量

适用场景：从一组区间中选出尽可能多的互不重叠的区间，如活动安排、课程排课、会议室预定。

题目：给定 N 个区间，选出最多的区间使得它们两两不相交。

```

1     #include <iostream>
2     #include <algorithm>
3     using namespace std;
4
5     const int MAXN = 100005;
6     struct Range {
7         int l, r;                // l: 左端点, r: 右端点
8         bool operator<(const Range& other) const {
9             return r < other.r;  // 按右端点升序
10        }
11    } ranges[MAXN];
12
13    int main() {
14        int n;                    // n: 区间个数
15        cin >> n;
16        for (int i = 0; i < n; i++) cin >> ranges[i].l >> ranges[i].r;
17        sort(ranges, ranges + n);
18        int ans = 0;              // ans: 最大不相交区间数
19        int last = -2e9;          // last: 上一个选中区间的右端点
20        // 原理: 每次选最早结束的区间, 给后续区间留下最大空间
21        for (int i = 0; i < n; i++) {
22            if (ranges[i].l > last) { // 当前区间不与上一个选中区间重叠
23                ans++;
24                last = ranges[i].r;
25            }
26        }
27        cout << ans << endl;
28        return 0;
29    }

```

Listing 8: 贪心——同类按右端点排序，每次选最早结束的

6.3 区间覆盖（最小区间数覆盖线段）

适用场景：用最少的给定区间覆盖一条指定线段，如铺设管道、信号覆盖、灌溉问题。

题目：给定 N 个区间和一个目标线段 $[start, end]$ ，求最少需要多少个区间完全覆盖目标线段。

```
1      #include <iostream>
2      #include <algorithm>
3      using namespace std;
4
5      const int MAXN = 100005;
6      struct Range {
7          int l, r;
8          bool operator<(const Range& other) const {
9              return l < other.l;    // 按左端点升序，方便从左到右扫描
10         }
11     } ranges[MAXN];
12
13     int main() {
14         int st, ed;                // st: 目标线段起点, ed: 目标线段终点
15         cin >> st >> ed;
16         int n;                     // n: 可选区间个数
17         cin >> n;
18         for (int i = 0; i < n; i++) cin >> ranges[i].l >> ranges[i].r;
19         sort(ranges, ranges + n);
20         int ans = 0;                // ans: 最少所需区间数
21         int cur = st;              // cur: 当前已覆盖到的位置（缺口左端）
22         int i = 0;                 // i: 当前遍历到的区间下标
23         // 原理：在左端点 <= cur 的所有区间中，选右端点最大的那个
24         while (cur < ed) {
25             int maxr = -2e9;        // maxr: 能覆盖到的最远右端点
26             while (i < n && ranges[i].l <= cur) {
27                 maxr = max(maxr, ranges[i].r);
28                 i++;
29             }
30             if (maxr <= cur) {       // 无法继续扩展，覆盖失败
```

```

31         cout << -1 << endl;
32         return 0;
33     }
34     ans++;
35     cur = maxr;           // 更新已覆盖位置
36 }
37 cout << ans << endl;
38 return 0;
39 }

```

Listing 9: 贪心——每次选能覆盖当前缺口且右端点最远的区间

6.4 合并果子（哈夫曼编码）

适用场景：将若干堆物品两两合并，每次合并的代价为两堆之和，求最小总代价。适用于数据压缩（哈夫曼树）、最优合并问题。

题目：有 n 堆果子，每次合并任意两堆，代价为两堆重量之和。求将所有堆合并成一堆的最小代价。

```

1     #include <iostream>
2     #include <queue>
3     using namespace std;
4
5     int main() {
6         int n;           // n: 果子堆数
7         cin >> n;
8         priority_queue<int, vector<int>, greater<int>> heap;
9         // heap: 小根堆，每次能  $O(\log n)$  取出最小值
10        for (int i = 0; i < n; i++) {
11            int x;         // x: 第 i 堆的重量
12            cin >> x;
13            heap.push(x);
14        }
15        int ans = 0;       // ans: 最小总代价
16        // 原理：哈夫曼算法——每次合并最小的两堆，合并后放回堆中
17        while (heap.size() > 1) {
18            int a = heap.top(); heap.pop(); // a: 当前最小堆
19            int b = heap.top(); heap.pop(); // b: 当前次小堆
20            int cost = a + b;           // cost: 合并 a 和 b 的代价
21            ans += cost;                // 累加总代价
22            heap.push(cost);            // 新堆放回堆中

```

```

23     }
24     cout << ans << endl;
25     return 0;
26 }

```

Listing 10: 贪心——小根堆每次取最小的两堆合并

6.5 排队接水（最小等待时间）

适用场景：多人排队接受服务，每人服务耗时不同，调整顺序使总等待时间最小。适用于任务调度、客服排队。

题目： n 个人排队接水，第 i 人需要 t_i 时间。求一种排队顺序使所有人的等待时间之和最小。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  const int MAXN = 1005;
6  struct Person {
7      int id;                // id: 人员编号
8      int t;                // t: 接水所需时间
9      bool operator<(const Person& other) const {
10         return t < other.t; // 按时间升序，短作业优先
11     }
12 } p[MAXN];
13
14 int main() {
15     int n;                // n: 人数
16     cin >> n;
17     for (int i = 0; i < n; i++) {
18         p[i].id = i + 1;
19         cin >> p[i].t;
20     }
21     sort(p, p + n);       // 短作业优先排序
22     long long total = 0;   // total: 总等待时间
23     long long cur = 0;     // cur: 当前累计接水时间
24     // 原理: 第 i 个人的等待时间 = 前面所有人的接水时间之和
25     for (int i = 0; i < n; i++) {
26         total += cur;      // cur 就是此人的等待时间
27         cur += p[i].t;     // 加上此人接水时间，传给下一个人

```

```

28     }
29     // 输出顺序
30     for (int i = 0; i < n; i++) cout << p[i].id << " ";
31     cout << endl;
32     printf("%.2f\n", (double)total / n); // 平均等待时间
33     return 0;
34 }

```

Listing 11: 贪心——按服务时间升序排队，短作业优先

6.6 货仓选址（绝对值不等式）

适用场景：在一条直线上选择一点，使得到所有给定点的距离之和最小。适用于物流中心选址、中位数优化问题。

题目：数轴上有 N 个点，选择一个点使得到所有点的距离之和最小，求最小距离和。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4
5  const int MAXN = 100005;
6  int a[MAXN];           // a[i]: 第 i 个点的坐标
7
8  int main() {
9      int n;             // n: 点数
10     cin >> n;
11     for (int i = 0; i < n; i++) cin >> a[i];
12     sort(a, a + n);     // 排序后取中位数
13     int mid = a[n / 2]; // mid: 中位数坐标
14     long long ans = 0;   // ans: 最小距离和
15     // 原理：绝对值函数 |x-a| 的最优解 x 是 a 的中位数
16     for (int i = 0; i < n; i++) {
17         ans += abs(a[i] - mid);
18     }
19     cout << ans << endl;
20     return 0;
21 }

```

Listing 12: 贪心——选中位数，利用绝对值不等式

7 二分 / 三分

二分查找 (Binary Search) 在单调函数或有序序列上每次排除一半的搜索空间, 复杂度 $O(\log n)$ 。三分查找用于单峰/单谷函数求极值。二分答案则将最优化问题转化为判定问题。

7.1 整数二分查找 (精确匹配)

适用场景: 在有序数组中查找目标值是否存在, 返回其下标。

题目: 给定一个升序数组, 判断目标值 x 是否存在。

```
1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 100005;
5      int a[MAXN];           // a[]: 升序数组
6
7      int main() {
8          int n, x;           // n: 数组长度, x: 目标值
9          cin >> n >> x;
10         for (int i = 0; i < n; i++) cin >> a[i];
11         int l = 0, r = n - 1; // l: 左边界, r: 右边界 (闭区间)
12         int ans = -1;         // ans: 目标下标, -1 表示未找到
13         // 原理: 每次比较中间值, 根据大小关系舍弃一半区间
14         while (l <= r) {
15             int mid = l + (r - l) / 2; // mid: 中间下标 (防溢出写法)
16             if (a[mid] == x) {
17                 ans = mid;
18                 break;
19             } else if (a[mid] < x) {
20                 l = mid + 1;           // x 在右半部分
21             } else {
22                 r = mid - 1;           // x 在左半部分
23             }
24         }
25         cout << ans << endl;
26         return 0;
27     }
```

Listing 13: 整数二分——查找目标值

7.2 二分查找左边界（第一个 $\geq x$ 的位置）

适用场景：在有序数组中查找第一个大于等于目标值的位置。典型应用：有序数组插入位置、找第一个满足条件的元素。

题目：给定升序数组，求第一个大于等于 x 的元素下标，不存在则输出 n 。

```
1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 100005;
5      int a[MAXN];
6
7      int main() {
8          int n, x;
9          cin >> n >> x;
10         for (int i = 0; i < n; i++) cin >> a[i];
11         int l = 0, r = n;          // r = n 表示答案可能是 n（不存在的情况）
12         // 原理：维护答案区间 [l, r)，每次将区间二分
13         while (l < r) {
14             int mid = l + (r - l) / 2;
15             if (a[mid] >= x) {
16                 r = mid;           // mid 满足条件，收缩右边界
17             } else {
18                 l = mid + 1;       // mid 不满足，收缩左边界
19             }
20         }
21         cout << l << endl;       // l 即为第一个 >= x 的下标
22         return 0;
23     }
```

Listing 14: 二分查找——第一个大于等于 x 的位置（lower_bound）

7.3 二分查找右边界（最后一个 $\leq x$ 的位置）

适用场景：在有序数组中查找最后一个小于等于目标值的位置。适用于查找某个元素最后一次出现的位置。

```
1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 100005;
5      int a[MAXN];
```



```

6
7     int main() {
8         int n, x;
9         cin >> n >> x;
10        for (int i = 0; i < n; i++) cin >> a[i];
11        int l = 0, r = n - 1;          // 闭区间 [l, r]
12        int ans = -1;                  // ans: 最后一个 <= x 的下标
13        while (l <= r) {
14            int mid = l + (r - l) / 2;
15            if (a[mid] <= x) {
16                ans = mid;              // 记录当前满足条件的下标
17                l = mid + 1;            // 尝试找更靠后的位置
18            } else {
19                r = mid - 1;
20            }
21        }
22        cout << ans << endl;
23        return 0;
24    }

```

Listing 15: 二分查找——最后一个小于等于 x 的位置

7.4 浮点数二分（求平方根/立方根）

适用场景：在实数范围内利用二分逼近解，如求方程根、开方、最大化平均值等。

题目：给定一个正实数 x ，求其平方根（精确到 10^{-6} ）。

```

1     #include <iostream>
2     #include <iomanip>
3     #include <cmath>
4     using namespace std;
5
6     int main() {
7         double x;                      // x: 被开方数
8         cin >> x;
9         double l = 0, r = max(1.0, x); // l: 左边界, r: 右边界 (注意 x<1 时平
10            方根 > x)
11        const double eps = 1e-8;        // eps: 精度要求
12        // 原理: 在实数区间上二分, 直至区间长度小于精度
13        while (r - l > eps) {
14            double mid = (l + r) / 2;    // mid: 区间中点

```

```

14         if (mid * mid >= x) {
15             r = mid;           // mid 的平方太大，平方根在左侧
16         } else {
17             l = mid;           // mid 的平方太小，平方根在右侧
18         }
19     }
20     cout << fixed << setprecision(6) << l << endl;
21     return 0;
22 }

```

Listing 16: 浮点数二分——求平方根

7.5 二分答案——最小化最大值

适用场景：答案具有单调性：如果某个值可行，则更大的值一定可行。将”求最小/最大的最优值”转化为”判断某个值是否可行”的判定问题。

题目：将 n 个正整数分成连续的 m 段，使得各段和的最大值最小，求这个最小值。

```

1     #include <iostream>
2     using namespace std;
3
4     const int MAXN = 100005;
5     int a[MAXN];           // a[i]: 第 i 个数
6     int n, m;              // n: 数字个数, m: 要分成的段数
7
8     // 判断在每段和不超过 limit 的情况下能否分成 <= m 段
9     bool check(long long limit) {
10         int cnt = 1;       // cnt: 段数 (至少为 1)
11         long long sum = 0;  // sum: 当前段的累加和
12         for (int i = 0; i < n; i++) {
13             if (a[i] > limit) return false; // 单个元素大于限制, 不可能
14             if (sum + a[i] > limit) { // 当前段已满
15                 cnt++;               // 新开一段
16                 sum = a[i];          // 新段的第一个元素
17             } else {
18                 sum += a[i];
19             }
20         }
21         return cnt <= m;          // 段数是否在允许范围内
22     }
23

```

```

24     int main() {
25         cin >> n >> m;
26         long long l = 0, r = 0;    // l: 二分左界（最大单元素），r: 二分右界
                                   // （总和）
27         for (int i = 0; i < n; i++) {
28             cin >> a[i];
29             l = max(l, (long long)a[i]);
30             r += a[i];
31         }
32         // 原理: limit 越大越容易满足，答案单调，二分查找最小的可行 limit
33         while (l < r) {
34             long long mid = l + (r - l) / 2;
35             if (check(mid)) {
36                 r = mid;           // mid 可行，尝试更小的值
37             } else {
38                 l = mid + 1;       // mid 不可行，需要更大的值
39             }
40         }
41         cout << l << endl;
42         return 0;
43     }

```

Listing 17: 二分答案——最小化各段和的最大值

7.6 三分查找（单峰函数极值）

适用场景：求单峰函数（先增后减）的极大值或单谷函数（先减后增）的极小值，适用于凸优化问题。

题目：给定一个先增后减的数组，求其最大值。

```

1     #include <iostream>
2     using namespace std;
3
4     const int MAXN = 100005;
5     int a[MAXN];           // a[]: 单峰数组（先严格递增再严格递减）
6
7     int main() {
8         int n;             // n: 数组长度
9         cin >> n;
10        for (int i = 0; i < n; i++) cin >> a[i];
11        int l = 0, r = n - 1; // l: 左边界, r: 右边界

```

```

12 // 原理：取两个三分点 m1, m2, 比较大小, 每次舍弃单调性不符的那 1/3 区间
13 while (r - l > 2) { // 区间足够大时才三分
14     int m1 = l + (r - l) / 3; // m1: 左三分点
15     int m2 = r - (r - l) / 3; // m2: 右三分点
16     if (a[m1] < a[m2]) {
17         l = m1; // 最大值在 m1 右侧
18     } else {
19         r = m2; // 最大值在 m2 左侧
20     }
21 }
22 // 小区间内直接扫描
23 int ans = a[l];
24 for (int i = l + 1; i <= r; i++) ans = max(ans, a[i]);
25 cout << ans << endl;
26 return 0;
27 }

```

Listing 18: 三分查找——求单峰数组最大值

7.7 二分查找旋转数组的最小值

适用场景：有序数组经过旋转（前面若干元素移到末尾）后查找最小值或目标值。利用部分有序的特性仍可二分。

题目：给定一个升序数组经过一次旋转后的数组（元素互不相同），求其最小值。

```

1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN]; // a[]: 旋转后的数组（原数组升序无重复）
6
7  int main() {
8      int n; // n: 数组长度
9      cin >> n;
10     for (int i = 0; i < n; i++) cin >> a[i];
11     int l = 0, r = n - 1;
12     // 原理：将 a[mid] 与 a[r] 比较, 判断最小值在左半还是右半
13     while (l < r) {
14         int mid = l + (r - l) / 2;
15         if (a[mid] < a[r]) {
16             r = mid; // 右半有序, 最小值在左半（含 mid）

```

```
17         } else {
18             l = mid + 1;           // 左半有序，最小值在右半
19         }
20     }
21     cout << a[l] << endl;        // l 即为最小值下标
22     return 0;
23 }
```

Listing 19: 二分查找——旋转有序数组中的最小值

8 前缀和与差分

前缀和（Prefix Sum）将区间求和从 $O(n)$ 降至 $O(1)$ ；差分（Difference）将区间增减从 $O(n)$ 降至 $O(1)$ 。两者互为逆运算，是处理批量区间操作的核心工具。

8.1 一维前缀和——区间和查询

适用场景：静态数组上频繁查询任意区间 $[L, R]$ 的元素和，预处理 $O(n)$ ，单次查询 $O(1)$ 。

题目：给定 n 个整数和 q 个查询，每次查询区间 $[L, R]$ 的元素和。

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAXN = 100005;
5  int a[MAXN];           // a[i]: 原数组第 i 个元素 (1-indexed)
6  long long pre[MAXN];   // pre[i]: 前缀和, pre[i] = a[1] + a[2] + ... + a[i]
7
8  int main() {
9      int n, q;           // n: 数组长度, q: 查询次数
10     cin >> n >> q;
11     for (int i = 1; i <= n; i++) cin >> a[i];
12     // 预处理前缀和: pre[i] = pre[i-1] + a[i]
13     for (int i = 1; i <= n; i++) {
14         pre[i] = pre[i-1] + a[i];
15     }
16     while (q--) {
17         int L, R;        // L: 查询左端点, R: 查询右端点
18         cin >> L >> R;
19         // 原理: 区间和 = 前缀和[R] - 前缀和[L-1]
20         cout << pre[R] - pre[L-1] << endl;
21     }
22     return 0;
23 }
```

Listing 20: 一维前缀和—— $O(1)$ 区间和查询

8.2 二维前缀和——子矩阵和查询

适用场景：静态矩阵上频繁查询子矩阵的元素和，如激光炸弹、图像积分图、子矩阵求和。预处理 $O(nm)$ ，查询 $O(1)$ 。

题目：给定 $n \times m$ 矩阵和 q 个查询，每次求 (x_1, y_1) 到 (x_2, y_2) 子矩阵的元素和。

```

1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 1005;
5      int a[MAXN][MAXN];           // a[i][j]: 原矩阵第 i 行第 j 列元素
6      long long pre[MAXN][MAXN];   // pre[i][j]: (1,1) 到 (i,j) 的矩阵和
7
8      int main() {
9          int n, m, q;              // n: 行数, m: 列数, q: 查询次数
10         cin >> n >> m >> q;
11         for (int i = 1; i <= n; i++)
12             for (int j = 1; j <= m; j++)
13                 cin >> a[i][j];
14         // 预处理二维前缀和
15         // 原理: pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j]
16         for (int i = 1; i <= n; i++) {
17             for (int j = 1; j <= m; j++) {
18                 pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j];
19             }
20         }
21         while (q--) {
22             int x1, y1, x2, y2;    // (x1,y1): 左上角, (x2,y2): 右下角
23             cin >> x1 >> y1 >> x2 >> y2;
24             // 原理: 子矩阵和 = 右下角前缀和 - 左 - 上 + 左上 (容斥原理)
25             long long ans = pre[x2][y2] - pre[x1-1][y2] - pre[x2][y1-1] + pre[x1-1][y1-1];
26             cout << ans << endl;
27         }
28         return 0;
29     }

```

Listing 21: 二维前缀和—— $O(1)$ 子矩阵和查询

8.3 一维差分——区间增减操作

适用场景：对一个数组进行多次区间加减操作，最后查询最终数组。差分将每次区间修改从 $O(n)$ 降到 $O(1)$ 。

题目：初始全零数组，进行 m 次操作：对 $[L, R]$ 区间每个元素加 c 。输出最终数组。

```

1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 100005;
5      int diff[MAXN];           // diff[i]: 差分数组, diff[i] = a[i] - a[
6                                 i-1]
7
8      int main() {
9          int n, m;             // n: 数组长度, m: 操作次数
10         cin >> n >> m;
11         // diff 初始全 0 (对应原数组全 0)
12         // 原理: 对 [L,R] 加 c, 等价于 diff[L] += c, diff[R+1] -= c
13         while (m--) {
14             int L, R, c;       // L,R: 操作区间, c: 增加值
15             cin >> L >> R >> c;
16             diff[L] += c;      // 从 L 开始影响生效
17             diff[R + 1] -= c;  // 从 R+1 开始影响消除
18         }
19         // 对差分数组做前缀和还原原数组
20         int cur = 0;           // cur: 当前位置的前缀和即 a[i]
21         for (int i = 1; i <= n; i++) {
22             cur += diff[i];     // cur = a[i]
23             cout << cur << " ";
24         }
25         cout << endl;
26         return 0;
27     }

```

Listing 22: 一维差分—— $O(1)$ 区间增减

8.4 二维差分——子矩阵增减操作

适用场景：对矩阵进行多次子矩阵批量加减，最后输出最终矩阵。典型应用：二维区间修改、棋盘覆盖。

题目：初始全零 $n \times m$ 矩阵， q 次操作：对 (x_1, y_1) 到 (x_2, y_2) 子矩阵每个元素加 c 。输出最终矩阵。

```

1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 1005;
5      int diff[MAXN][MAXN];          // diff[i][j]: 二维差分数组
6
7      int main() {
8          int n, m, q;                // n: 行数, m: 列数, q: 操作次数
9          cin >> n >> m >> q;
10         // 原理: 子矩阵加 c 转化为四个角上的差分操作 (二维容斥)
11         // diff[x1][y1] += c, diff[x2+1][y1] -= c,
12         // diff[x1][y2+1] -= c, diff[x2+1][y2+1] += c
13         while (q--) {
14             int x1, y1, x2, y2, c;  // (x1,y1): 左上角, (x2,y2): 右下角, c: 增
                                     // 加值
15             cin >> x1 >> y1 >> x2 >> y2 >> c;
16             diff[x1][y1] += c;
17             diff[x2 + 1][y1] -= c;
18             diff[x1][y2 + 1] -= c;
19             diff[x2 + 1][y2 + 1] += c;
20         }
21         // 对差分数组做二维前缀和还原
22         for (int i = 1; i <= n; i++) {
23             for (int j = 1; j <= m; j++) {
24                 diff[i][j] += diff[i-1][j] + diff[i][j-1] - diff[i-1][j-1];
25                 cout << diff[i][j] << " ";
26             }
27             cout << endl;
28         }
29         return 0;
30     }

```

Listing 23: 二维差分—— $O(1)$ 子矩阵增减

8.5 前缀和——区间内特定值出现次数

适用场景：多次查询某个区间内某个值出现了多少次（或区间内满足某条件的元素个数）。对每种值分别建前缀和。

题目：给定一个只包含小写字母的字符串， q 次查询某区间内某字母的出现次数。

```
1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 100005;
5      int pre[MAXN][26];          // pre[i][c]: 前 i 个字符中字母 c 的出现次
                                   数
6
7      int main() {
8          string s;                // s: 输入字符串
9          cin >> s;
10         int n = (int)s.size();    // n: 字符串长度
11         // 预处理：统计每种字母的前缀出现次数
12         for (int i = 1; i <= n; i++) {
13             for (int c = 0; c < 26; c++) {
14                 pre[i][c] = pre[i-1][c]; // 继承之前的前缀
15             }
16             pre[i][s[i-1] - 'a']++; // 当前字符累计加一
17         }
18         int q;                    // q: 查询次数
19         cin >> q;
20         while (q--) {
21             int L, R;              // L: 左边界, R: 右边界
22             char ch;               // ch: 查询的字母
23             cin >> L >> R >> ch;
24             int c = ch - 'a';      // c: 字母对应编号
25             // 原理：区间内出现次数 = 前缀[R] - 前缀[L-1]
26             cout << pre[R][c] - pre[L-1][c] << endl;
27         }
28         return 0;
29     }
```

Listing 24: 前缀和——区间内字母出现次数

9 排序

排序是将无序数据按特定顺序排列的过程，是许多算法的基础预处理步骤。

9.1 快速排序

适用场景：通用排序场景，平均 $O(n \log n)$ ，是实际应用中最高效的比较排序算法之一。不稳定排序。

题目：给定 n 个整数，将其从小到大排序。

```
1      #include <iostream>
2      using namespace std;
3
4      const int MAXN = 100005;
5      int a[MAXN];           // a[]: 待排序数组
6
7      // 原理：选取基准 pivot，将数组分为"小于 pivot"和"大于 pivot"两部分，递归
      // 排序
8      // 分区函数：返回基准元素的最终位置
9      int partition(int l, int r) {
10         int pivot = a[l + (r - l) / 2]; // pivot: 选取中间位置元素作为基准
11         int i = l - 1, j = r + 1;      // i: 左指针, j: 右指针
12         while (true) {
13             do { i++; } while (a[i] < pivot); // 从左找第一个 >= pivot 的元素
14             do { j--; } while (a[j] > pivot); // 从右找第一个 <= pivot 的元素
15             if (i >= j) return j;          // 指针相遇，分区完成
16             swap(a[i], a[j]);             // 交换逆序对
17         }
18     }
19
20     void quickSort(int l, int r) {
21         if (l >= r) return;              // 区间长度 <= 1, 已有序
22         int p = partition(l, r);         // p: 分区点
23         quickSort(l, p);                 // 递归排序左半部分
24         quickSort(p + 1, r);             // 递归排序右半部分
25     }
26
27     int main() {
28         int n;                          // n: 数组长度
29         cin >> n;
30         for (int i = 0; i < n; i++) cin >> a[i];
```

```

31     quickSort(0, n - 1);
32     for (int i = 0; i < n; i++) cout << a[i] << " ";
33     cout << endl;
34     return 0;
35 }

```

Listing 25: 快速排序——分治 + 分区

9.2 归并排序

适用场景：通用排序，稳定排序，同时可顺便统计逆序对数量。适合需要稳定性的场景。

题目：给定 n 个整数，使用归并排序将其从小到大排序。

```

1     #include <iostream>
2     using namespace std;
3
4     const int MAXN = 100005;
5     int a[MAXN];           // a[]: 待排序数组
6     int tmp[MAXN];         // tmp[]: 归并时的临时数组
7
8     // 原理: 将两个有序子数组合并为一个有序数组
9     void merge(int l, int mid, int r) {
10         int i = l, j = mid + 1, k = 1; // i: 左子数组指针, j: 右子数组指针, k:
            // 临时数组指针
11         while (i <= mid && j <= r) {
12             if (a[i] <= a[j]) tmp[k++] = a[i++];
13             else tmp[k++] = a[j++];
14         }
15         while (i <= mid) tmp[k++] = a[i++]; // 左半有剩余
16         while (j <= r) tmp[k++] = a[j++]; // 右半有剩余
17         for (int p = l; p <= r; p++) a[p] = tmp[p]; // 写回原数组
18     }
19
20     void mergeSort(int l, int r) {
21         if (l >= r) return;
22         int mid = l + (r - l) / 2; // mid: 中点
23         mergeSort(l, mid);         // 递归排序左半
24         mergeSort(mid + 1, r);     // 递归排序右半
25         merge(l, mid, r);          // 合并两个有序部分
26     }

```

```

27
28     int main() {
29         int n;
30         cin >> n;
31         for (int i = 0; i < n; i++) cin >> a[i];
32         mergeSort(0, n - 1);
33         for (int i = 0; i < n; i++) cout << a[i] << " ";
34         cout << endl;
35         return 0;
36     }

```

Listing 26: 归并排序——分治 + 合并有序数组

9.3 归并排序求逆序对数量

适用场景：统计数组中 $i < j$ 且 $a_i > a_j$ 的数对个数。逆序对数量反映数组的有序程度。

题目：给定一个数组，求其中逆序对的总数。

```

1     #include <iostream>
2     using namespace std;
3
4     const int MAXN = 100005;
5     int a[MAXN];
6     int tmp[MAXN];
7     long long ans = 0;           // ans: 逆序对总数（可能超过 int 范围）
8
9     // 原理：在合并两个有序子数组时，每当从右子数组取元素，
10    // 说明左子数组中剩余的所有元素都大于当前元素，均构成逆序对
11    void merge(int l, int mid, int r) {
12        int i = l, j = mid + 1, k = l;
13        while (i <= mid && j <= r) {
14            if (a[i] <= a[j]) {
15                tmp[k++] = a[i++];
16            } else {
17                tmp[k++] = a[j++];
18                ans += mid - i + 1; // 左半剩余元素个数即为新增逆序对数
19            }
20        }
21        while (i <= mid) tmp[k++] = a[i++];
22        while (j <= r) tmp[k++] = a[j++];

```

```
23     for (int p = l; p <= r; p++) a[p] = tmp[p];
24 }
25
26 void mergeSort(int l, int r) {
27     if (l >= r) return;
28     int mid = l + (r - l) / 2;
29     mergeSort(l, mid);
30     mergeSort(mid + 1, r);
31     merge(l, mid, r);
32 }
33
34 int main() {
35     int n;
36     cin >> n;
37     for (int i = 0; i < n; i++) cin >> a[i];
38     mergeSort(0, n - 1);
39     cout << ans << endl;
40     return 0;
41 }
```

Listing 27: 归并排序——计数逆序对

9.4 快速选择（求第 k 小的数）

适用场景：在无序数组中查找第 k 小（或第 k 大）的元素，平均时间复杂度 $O(n)$ ，无需完全排序。

题目：给定 n 个整数，求其中第 k 小的数。

```
1     #include <iostream>
2     using namespace std;
3
4     const int MAXN = 100005;
5     int a[MAXN];
6
7     // 分区函数（同快速排序）
8     int partition(int l, int r) {
9         int pivot = a[l + (r - l) / 2];
10        int i = l - 1, j = r + 1;
11        while (true) {
12            do { i++; } while (a[i] < pivot);
13            do { j--; } while (a[j] > pivot);
```

```

14         if (i >= j) return j;
15         swap(a[i], a[j]);
16     }
17 }
18
19 // 原理：分区后只递归包含第 k 小的那一半，另一半舍弃
20 int quickSelect(int l, int r, int k) { // k: 查找第 k 小的数 (k 从 1 开
    始)
21     if (l == r) return a[l]; // 区间只剩一个数
22     int p = partition(l, r); // p: 分区点
23     int leftLen = p - l + 1; // leftLen: 左半部分的元素个数
24     if (k <= leftLen) {
25         return quickSelect(l, p, k); // 第 k 小在左半部分
26     } else {
27         return quickSelect(p + 1, r, k - leftLen); // 在右半部分找第 k-
            leftLen 小
28     }
29 }
30
31 int main() {
32     int n, k;
33     cin >> n >> k;
34     for (int i = 0; i < n; i++) cin >> a[i];
35     cout << quickSelect(0, n - 1, k) << endl;
36     return 0;
37 }

```

Listing 28: 快速选择——基于快排分区，只递归一边

9.5 计数排序

适用场景：数据范围较小（值域有限）的整数排序，时间复杂度 $O(n+m)$ ，其中 m 为值域大小。稳定排序。

题目：给定 n 个范围在 $[0, 10^5]$ 内的整数，将其从小到大排序。

```

1     #include <iostream>
2     using namespace std;
3
4     const int MAXV = 100005; // MAXV: 值域上限
5     int cnt[MAXV];           // cnt[x]: 数值 x 出现的次数
6

```

```

7   int main() {
8       int n;                // n: 元素个数
9       cin >> n;
10      int maxv = 0;          // maxv: 实际数据中的最大值
11      // 原理: 统计每个数值出现的次数, 再按数值顺序输出
12      for (int i = 0; i < n; i++) {
13          int x;              // x: 当前读入的数值
14          cin >> x;
15          cnt[x]++;           // 计数
16          maxv = max(maxv, x);
17      }
18      for (int v = 0; v <= maxv; v++) {
19          while (cnt[v]--) {   // 输出 cnt[v] 次数值 v
20              cout << v << " ";
21          }
22      }
23      cout << endl;
24      return 0;
25  }

```

Listing 29: 计数排序——统计各值出现次数

9.6 基数排序

适用场景：对整数或定长字符串排序，通过从低位到高位逐位进行稳定排序（通常搭配计数排序），时间复杂度 $O(d(n+k))$ ， d 为位数。

题目：给定 n 个非负整数，使用基数排序将其从小到大排序。

```

1   #include <iostream>
2   #include <cstring>
3   using namespace std;
4
5   const int MAXN = 100005;
6   int a[MAXN];           // a[]: 待排序数组
7   int b[MAXN];           // b[]: 辅助数组
8   int cnt[10];           // cnt[d]: 当前位数字 d 的出现次数 (基数为
9                           10)
10
11  int main() {
12      int n;               // n: 元素个数
13      cin >> n;

```



```
13     int maxv = 0;                // maxv: 最大值, 用于确定位数
14     for (int i = 0; i < n; i++) {
15         cin >> a[i];
16         maxv = max(maxv, a[i]);
17     }
18     // 原理: 从低位到高位, 每次按当前位做一次稳定排序 (计数排序)
19     // exp 表示当前处理的位权 (1=个位, 10=十位, 100=百位...)
20     for (int exp = 1; maxv / exp > 0; exp *= 10) {
21         memset(cnt, 0, sizeof(cnt));
22         // 统计当前位各数字出现次数
23         for (int i = 0; i < n; i++) {
24             int digit = (a[i] / exp) % 10; // digit: a[i] 在当前位的数字
25             cnt[digit]++;
26         }
27         // 前缀和转换为位置索引 (保证稳定性)
28         for (int d = 1; d < 10; d++) {
29             cnt[d] += cnt[d - 1];
30         }
31         // 从后往前遍历, 确保稳定排序
32         for (int i = n - 1; i >= 0; i--) {
33             int digit = (a[i] / exp) % 10;
34             b[--cnt[digit]] = a[i]; // 放入对应位置
35         }
36         // 写回原数组
37         for (int i = 0; i < n; i++) a[i] = b[i];
38     }
39     for (int i = 0; i < n; i++) cout << a[i] << " ";
40     cout << endl;
41     return 0;
42 }
```

Listing 30: 基数排序——按位 LSD 排序 (低位优先)

基础算法对比总结

表 3: 各算法核心思想与适用条件对比

算法类别	核心思想	适用条件
枚举	遍历解空间中所有候选解	数据规模小 ($n \leq 20 \sim 30$), 解空间有限
贪心	每步选局部最优, 期望全局最优	具有贪心选择性质和最优子结构
二分查找	在单调区间上每次排除一半	数据有序或函数单调
二分答案	将最优化转为判定, 利用答案单调性	答案存在上下界且满足单调性
三分查找	在单峰/单谷函数上缩小区间	函数严格单峰或单谷
前缀和	预处理累加和, $O(1)$ 区间查询	静态数组, 需要频繁区间求和
差分	记录变化量, $O(1)$ 区间修改	批量区间修改后统一查询结果
快速排序	分治 + 分区, 平均 $O(n \log n)$	通用比较排序, 不稳定
归并排序	分治 + 合并, 稳定 $O(n \log n)$	需要稳定排序或统计逆序对
计数排序	统计出现次数, $O(n+m)$	值域小的整数排序
基数排序	按位稳定排序, $O(d(n+k))$	整数/定长串, 位数少

第三部分 动态规划专题

10 线性 DP

10.1 最长上升子序列 (LIS)

解决问题: 给定一个序列, 求最长的严格递增子序列的长度。适用于股票走势分析、基因序列比对、导弹拦截系统等场景。

题目描述

给定一个长度为 n 的数组 a , 找出其中最长的严格递增子序列的长度。子序列不要求连续, 但顺序必须与原数组一致。

输入示例

8
3 1 2 6 4 5 10 7

输出示例

5

输出解释

最长上升子序列为：1, 2, 4, 5, 10（长度为 5）

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 数组长度
5          cin >> n;
6          vector<int> a(n); // a: 存储原始数组的向量
7          for (int i = 0; i < n; i++) {
8              cin >> a[i]; // 循环读入n个整数到数组a中
9          }
10         // dp[i]: 以a[i]结尾的最长上升子序列长度，初始化为1（子序列只包含自身）
11         vector<int> dp(n, 1);
12         int ans = 0; // ans: 全局最长上升子序列的长度
13         // 核心原理：对于每个位置i，遍历它前面所有位置j，如果a[j] < a[i]，
14         // 则a[i]可以接在以a[j]结尾的序列后面，形成更长的上升子序列
15         for (int i = 0; i < n; i++) { // i: 当前考虑的子序列结尾元素下标
16             for (int j = 0; j < i; j++) { // j: 遍历i之前的所有元素下标
17                 if (a[j] < a[i]) { // 判断a[j]是否小于a[i]，保证严格递
18                                     增
19                     dp[i] = max(dp[i], dp[j] + 1); // 状态转移：取最大值，接在j
20                                                         后长度+1
21             }
22         }
23         ans = max(ans, dp[i]); // 更新全局最长长度
24     }
25     cout << ans << "\n"; // 输出最终结果
26     return 0;
27 }
```

10.2 最长公共子序列（LCS）

解决问题：给定两个字符串/序列，求它们最长的公共子序列的长度。适用于版本对比、基因序列相似度分析、文件差异比较等场景。

题目描述

给定两个字符串 s 和 t ，找出它们的最长公共子序列的长度。公共子序列是指在两个字符串中都按顺序出现（不一定连续）的字符序列。

输入示例

```
abcde
ace
```

输出示例

3

输出解释

最长公共子序列为"ace"，长度为 3

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, t; // s, t: 两个输入的字符串
5          cin >> s >> t;
6          int n = s.size(), m = t.size(); // n: 字符串s的长度, m: 字符串t的长度
7          // dp[i][j]: s的前i个字符和t的前j个字符的最长公共子序列长度
8          // i和j的取值范围都是从0到n/m, 0表示空串
9          vector<vector<int>>> dp(n + 1, vector<int>(m + 1, 0));
10         // 核心原理: 遍历两个字符串的所有前缀组合
11         // 若当前字符相等, 则LCS长度在左上角基础上+1
12         // 若不相等, 则取左边或上边的最大值
13         for (int i = 1; i <= n; i++) { // i: s字符串的当前处理前缀长度
14             for (int j = 1; j <= m; j++) { // j: t字符串的当前处理前缀长度
15                 if (s[i - 1] == t[j - 1]) { // 当前两个字符相等 (注意字符串索引
16                     // 从0开始)
17                     // 相等时, LCS长度 = 去掉这两个字符后的LCS长度 + 1
18                     dp[i][j] = dp[i - 1][j - 1] + 1;
19                 } else {
20                     // 不相等时, LCS长度 = max(去掉s当前字符, 去掉t当前字符)
21                     dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
22                 }
23             }
24         }
25     }
```

```
23     }
24     cout << dp[n][m] << "\n"; // dp[n][m] 即为两个完整字符串的LCS长度
25     return 0;
26 }
```

10.3 最大子数组和（Kadane 算法）

解决问题：给定一个整数数组（可能包含负数），求连续子数组的最大和。适用于股票最大收益、最优区间选择、信号处理等场景。

题目描述

给定一个整数数组 *nums*，请找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

输入示例

-2 1 -3 4 -1 2 1 -5 4

输出示例

6

输出解释

连续子数组 [4, -1, 2, 1] 的和最大，为 6

```
1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n; // n: 数组长度
5         cin >> n;
6         vector<int> a(n); // a: 存储原始整数的向量
7         for (int i = 0; i < n; i++) {
8             cin >> a[i];
9         }
10        int cur = a[0]; // cur: 以当前位置结尾的最大子数组和（当前最优）
11        int ans = a[0]; // ans: 全局最大子数组和
12        // 核心原理：Kadane算法，遍历数组时维护两个值
13        // 对于每个元素，要么将其加入之前的子数组，要么以它作为新子数组的起点
14        // cur = max(当前元素, 当前元素 + 之前的cur)
```

```

15     for (int i = 1; i < n; i++) { // i: 从第二个元素开始遍历
16         // 状态转移: 决定是开启新子数组(a[i]), 还是延续旧子数组(cur + a[i])
17         cur = max(a[i], cur + a[i]);
18         ans = max(ans, cur); // 用当前子数组和更新全局最大值
19     }
20     cout << ans << "\n";
21     return 0;
22 }

```

10.4 最长回文子串

解决问题: 给定一个字符串, 找出其中最长的回文子串。回文是指正着读和反着读都相同的字符串。适用于文本处理、DNA 序列分析等场景。

题目描述

给定一个字符串 s , 找出其中最长的回文子串。如果有多个, 返回任意一个。

输入示例

babad

输出示例

aba 或 bab

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         string s; // s: 输入的原始字符串
5         cin >> s;
6         int n = s.size(); // n: 字符串的长度
7         // dp[i][j]: 子串s[i..j]是否为回文串, true表示是回文
8         vector<vector<bool>> dp(n, vector<bool>(n, false));
9         int start = 0, maxlen = 1; // start: 最长回文子串的起始索引, maxlen:
10        // 最长回文子串的长度
11        // 核心原理: 回文串去掉两端字符后仍是回文串
12        // 先初始化所有长度为1和2的子串, 再逐步扩展长度
13        for (int i = 0; i < n; i++) {
14            dp[i][i] = true; // 所有单个字符都是回文串

```

```
15     for (int i = 0; i < n - 1; i++) {
16         if (s[i] == s[i + 1]) { // 相邻两个字符相等
17             dp[i][i + 1] = true; // 长度为2的子串是回文
18             start = i;         // 更新最长回文起始位置
19             maxlen = 2;        // 更新最长回文长度为2
20         }
21     }
22     for (int len = 3; len <= n; len++) { // len: 当前检查的子串长度, 从
23         3开始
24         for (int i = 0; i + len - 1 < n; i++) { // i: 子串的左边界索引
25             int j = i + len - 1;                // j: 子串的右边界索引
26             // 当两端字符相等, 且去掉两端后的内部子串是回文时, 当前子串为回
27             文
28             if (s[i] == s[j] && dp[i + 1][j - 1]) {
29                 dp[i][j] = true; // 标记[i, j]区间为回文串
30                 start = i;        // 更新最长回文串的起始位置
31                 maxlen = len;     // 更新最长回文串的长度
32             }
33         }
34     }
35     cout << s.substr(start, maxlen) << "\n"; // 截取并输出最长回文子串
36     return 0;
37 }
```

11 背包 DP

11.1 0-1 背包

解决问题：有 n 个物品，每个物品只能选一次，在容量为 W 的背包中装入价值最大的物品组合。适用于资源分配、投资组合、货物装载等场景。

题目描述

有 n 件物品和一个容量为 W 的背包。第 i 件物品的重量为 w_i ，价值为 v_i 。每件物品只能选择一次，求在不超过背包容量的前提下，能装入的最大总价值。

输入示例

```
4 5
1 2
2 4
3 4
4 5
```

输出示例

```
8
```

输出解释

选择物品 1、2、3（重量 $1+2+3=6$ 超重），最优是选择物品 2 和 3（重量 $2+3=5$ ，价值 $4+4=8$ ）

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, W; // n: 物品的总数量, W: 背包的总容量
5          cin >> n >> W;
6          vector<int> w(n), v(n); // w[i]: 第i个物品的重量, v[i]: 第i个物品的价
              值
7          for (int i = 0; i < n; i++) {
8              cin >> w[i] >> v[i];
9          }
10         // dp[j]: 背包容量为j时能装入的最大总价值, j的范围是0到W
11         vector<int> dp(W + 1, 0);
12         // 核心原理: 对于每个物品, 逆序遍历容量, 确保每个物品只被选一次
```



```

13 // 状态转移方程: dp[j] = max(不选当前物品, 选当前物品)
14 for (int i = 0; i < n; i++) { // i: 当前正在处理的物品的索引
15     // 逆序遍历容量, 防止同一个物品被重复计算 (01背包特性)
16     for (int j = W; j >= w[i]; j--) { // j: 当前考虑的背包容量
17         // dp[j]: 不选第i个物品; dp[j-w[i]]+v[i]: 选第i个物品
18         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
19     }
20 }
21 cout << dp[W] << "\n"; // 容量为W时的最大价值即为答案
22 return 0;
23 }

```

11.2 完全背包

解决问题: 有 n 种物品, 每种物品无限数量, 在容量为 W 的背包中装入价值最大的物品组合。适用于零钱兑换、原材料无限供应等场景。

题目描述

有 n 种物品和一个容量为 W 的背包。第 i 种物品的重量为 w_i , 价值为 v_i , 每种物品的数量是无限的。求在不超过背包容量的前提下, 能装入的最大总价值。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n, W; // n: 物品种类的数量, W: 背包的总容量
5      cin >> n >> W;
6      vector<int> w(n), v(n); // w[i]: 第i种物品的重量, v[i]: 第i种物品的价
7                               值
8      for (int i = 0; i < n; i++) {
9          cin >> w[i] >> v[i];
10     }
11     // dp[j]: 背包容量为j时能装入的最大总价值
12     vector<int> dp(W + 1, 0);
13     // 核心原理: 对于每种物品, 正序遍历容量, 实现无限次选取
14     // 因为正序遍历时, dp[j-w[i]]可能已经包含了当前物品, 所以可以无限取
15     for (int i = 0; i < n; i++) { // i: 当前正在处理的物品种类的索引
16         // 正序遍历容量, 允许同一物品被多次选取
17         for (int j = w[i]; j <= W; j++) { // j: 当前考虑的背包容量
18             // 与01背包的唯一区别就是内层循环是正序
19             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
20         }
21     }
22 }

```

```

19     }
20 }
21 cout << dp[W] << "\n";
22 return 0;
23 }
```

11.3 多重背包（二进制优化）

解决问题：有 n 种物品，每种物品有有限数量，在容量为 W 的背包中装入价值最大的物品组合。适用于库存有限、配额分配等场景。

题目描述

有 n 种物品和一个容量为 W 的背包。第 i 种物品的重量为 w_i ，价值为 v_i ，数量为 c_i 个。求在不超过背包容量的前提下，能装入的最大总价值。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n, W; // n: 物品种类的数量, W: 背包的总容量
5      cin >> n >> W;
6      vector<int> w(n), v(n), c(n); // w[i]: 第i种物品的重量, v[i]: 价值, c
7                                     [i]: 该物品的最大数量
8      for (int i = 0; i < n; i++) {
9          cin >> w[i] >> v[i] >> c[i];
10     }
11     vector<int> dp(W + 1, 0); // dp[j]: 容量为j的背包能装的最大价值
12     // 核心原理: 二进制优化, 将每种物品按2的幂次分解成多个新的物品组
13     // 这样将多重背包问题转化为01背包问题, 时间复杂度降至O(W * Σlog(c[i]))
14     for (int i = 0; i < n; i++) { // i: 当前处理的物品种类索引
15         int k = 1; // k: 当前拆分的数量, 从2^0=1开始
16         int remain = c[i]; // remain: 当前剩余未拆分的物品数量
17         while (remain > 0) {
18             int take = min(k, remain); // take: 当前组要取多少个物品
19             int weight = w[i] * take; // weight: 当前组的重量总和
20             int value = v[i] * take; // value: 当前组的价值总和
21             // 将拆分出的这一组视为一个新的物品, 用01背包的方式处理
22             for (int j = W; j >= weight; j--) {
23                 dp[j] = max(dp[j], dp[j - weight] + value);
24             }
25             remain -= take; // 从剩余数量中减去已取走的
26         }
27     }
```

```

25         k <= 1;           // k = k * 2, 继续下一个2的幂次
26     }
27 }
28 cout << dp[W] << "\n";
29 return 0;
30 }

```

11.4 分组背包

解决问题：物品分成若干组，每组中最多只能选一个物品，在容量为 W 的背包中装入价值最大的物品组合。适用于班级选课、产品线选择等场景。

题目描述

有 n 组物品和一个容量为 W 的背包。每组中有若干物品，每组最多只能选一个物品。求能装入的最大总价值。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n, W; // n: 物品组数, W: 背包总容量
5         cin >> n >> W;
6         vector<vector<int>> group_w, group_v; // group_w[i]: 第i组所有物品的
7                                               // 重量, group_v[i]: 第i组所有物品的价值
8         for (int i = 0; i < n; i++) { // i: 当前正在读取的组索引
9             int cnt; // cnt: 当前组内的物品数量
10            cin >> cnt;
11            vector<int> w(cnt), v(cnt); // 临时存储当前组的重量和价值
12            for (int j = 0; j < cnt; j++) {
13                cin >> w[j] >> v[j];
14            }
15            group_w.push_back(w);
16            group_v.push_back(v);
17        }
18        // dp[j]: 容量为j的背包能装的最大价值
19        vector<int> dp(W + 1, 0);
20        // 核心原理: 先遍历组, 再逆序容量, 最后遍历组内物品
21        // 逆序容量确保每组最多只选一个物品
22        for (int i = 0; i < n; i++) { // i: 当前正在处理的组的索引
23            for (int j = W; j >= 0; j--) { // j: 当前考虑的背包容量

```

```
23         for (int k = 0; k < group_w[i].size(); k++) { // k: 组内物品的
24             索引
25             if (j >= group_w[i][k]) {
26                 dp[j] = max(dp[j], dp[j - group_w[i][k]] + group_v[i][k
27                 ]);
28             }
29         }
30     cout << dp[W] << "\n";
31     return 0;
32 }
```

12 区间 DP

12.1 石子合并

解决问题：给定一排石子，每次只能合并相邻两堆，花费为两堆石子数之和，求合并成一堆的最小总花费。适用于最优括号匹配、矩阵链乘等场景。

题目描述

有 n 堆石子排成一排，每堆石子有数量 a_i 。每次可以合并相邻的两堆，花费为两堆石子数量之和。经过 $n - 1$ 次合并后成为一堆，求最小总花费。

```

1      #include <bits/stdc++.h>
2
3      using namespace std;
4
5      int main() {
6          int n; // n: 石子堆的数量
7          cin >> n;
8          vector<int> a(n + 1); // a[i]: 第i堆石子的数量, 索引从1开始
9          vector<int> prefix(n + 1, 0); // prefix[i]: 前i堆石子的总重量 (前缀和)
10
11         for (int i = 1; i <= n; i++) {
12             cin >> a[i];
13             prefix[i] = prefix[i - 1] + a[i]; // 计算前缀和
14         }
15
16         // dp[i][j]: 将区间[i, j]内的石子合并成一堆的最小花费
17         vector<vector<int>> dp(n + 2, vector<int>(n + 2, 0));
18         // 核心原理: 枚举区间长度, 再枚举左端点, 最后枚举区间内的分割点
19         // 状态转移: dp[i][j] = min(dp[i][k] + dp[k+1][j] + sum(i,j))
20         for (int len = 2; len <= n; len++) { // len: 当前枚举的区间长度
21             for (int i = 1; i + len - 1 <= n; i++) { // i: 区间左端点
22                 int j = i + len - 1; // j: 区间右端点
23                 dp[i][j] = 1e9; // 初始化为一个很大的值, 用于求最小值
24                 for (int k = i; k < j; k++) { // k: 分割点, 将区间分成[i,k]和[k+1,j]
25                     // 合并总花费 = 左区间最小花费 + 右区间最小花费 + 当前合并花费(区间和)
26                     int cost = dp[i][k] + dp[k + 1][j] + (prefix[j] - prefix[i] - 1);
27                     dp[i][j] = min(dp[i][j], cost);
28                 }
29             }
30         }
31     }

```

```

27         cout << dp[1][n] << "\n"; // 输出合并整个区间[1,n]的最小花费
28         return 0;
29     }

```

12.2 括号匹配

解决问题：给定一个括号序列，求最长有效括号子串的长度或最少添加几个括号使其合法。适用于代码语法检查、表达式求值等场景。

题目描述

给定一个只包含 '(' 和 ')' 的字符串，找出最长的有效（格式正确且连续）括号子串的长度。

输入示例

((()())

输出示例

6

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         string s; // s: 输入的括号字符串
5         cin >> s;
6         int n = s.size(); // n: 字符串的长度
7         // dp[i]: 以第i个字符结尾的最长有效括号子串的长度
8         vector<int> dp(n, 0);
9         int ans = 0; // ans: 全局最长有效括号长度
10        // 核心原理：只处理')'，因为'('无法形成有效括号结尾
11        // 分两种情况：'...( )' 或 '...))... '，利用dp数组向前跳跃匹配
12        for (int i = 1; i < n; i++) { // i: 当前处理的字符索引，从1开始（0位置
            // 无法形成有效括号）
13            if (s[i] == ')') { // 只有右括号才可能形成有效括号结尾
14                if (s[i - 1] == '(') {
15                    // 情况1: 形成 "...()" 模式
16                    // 有效长度 = 当前匹配的2 + i-2位置的有效长度
17                    dp[i] = (i >= 2 ? dp[i - 2] : 0) + 2;

```

```
18         } else if (i - dp[i - 1] - 1 >= 0 && s[i - dp[i - 1] - 1] == '  
19             (') {  
20                 // 情况2: 形成 "...))..." 模式, 需要找到匹配的左括号  
21                 // i - dp[i-1] - 1 是可能匹配的左括号位置  
22                 // 有效长度 = 内部有效长度 + 2 + 左括号前的有效长度  
23                 dp[i] = dp[i - 1] + 2;  
24                 if (i - dp[i - 1] - 2 >= 0) {  
25                     dp[i] += dp[i - dp[i - 1] - 2];  
26                 }  
27                 ans = max(ans, dp[i]); // 更新全局最长长度  
28             }  
29         }  
30         cout << ans << "\n";  
31         return 0;  
32     }
```

13 树形 DP

13.1 树的最大独立集（没有上司的舞会）

解决问题：在树形结构中，选择不相邻的节点使得权值和最大。适用于公司年会邀请、选课系统、家庭聚会等场景。

题目描述

某公司有 n 个员工，每个员工有一个快乐值。员工之间的上下级关系构成一棵树。如果邀请了某个员工，就不能邀请他的直接上级。求能获得的最大快乐值总和。

```

1      #include <bits/stdc++.h>
2
3      using namespace std;
4
5      int main() {
6          int n; // n: 员工的总人数
7          cin >> n;
8          vector<int> happy(n + 1); // happy[i]: 第i个员工的快乐值
9          for (int i = 1; i <= n; i++) {
10             cin >> happy[i];
11         }
12         vector<vector<int>> g(n + 1); // g[u]: 存储以u为父节点的所有子节点列表
13         vector<int> indeg(n + 1, 0); // indeg[i]: 节点i的入度，用于寻找根节点
14         for (int i = 0; i < n - 1; i++) {
15             int u, v; // u: 上级（父节点），v: 下级（子节点）
16             cin >> u >> v;
17             g[u].push_back(v);
18             indeg[v]++; // 子节点的入度加1
19         }
20         int root = 1; // 寻找根节点（没有父节点的节点，即入度为0的节点）
21         for (int i = 1; i <= n; i++) {
22             if (indeg[i] == 0) {
23                 root = i;
24                 break;
25             }
26         }
27         // dp[u][0]: 不邀请员工u时，以u为根的子树能获得的最大快乐值
28         // dp[u][1]: 邀请员工u时，以u为根的子树能获得的最大快乐值
29         vector<vector<int>> dp(n + 1, vector<int>(2, 0));
30         // 核心原理：后序遍历DFS，利用树结构进行DP
31         // 若选当前节点，则子节点都不能选；若不选当前节点，子节点可选可不选
32         function<void(int)> dfs = [&](int u) {

```



```

31         dp[u][1] = happy[u]; // 邀请当前节点，先加上自己的快乐值
32         for (int v : g[u]) { // v: 当前节点u的所有子节点
33             dfs(v); // 递归处理子节点
34             dp[u][0] += max(dp[v][0], dp[v][1]); // u不选时，子节点取最优
35             dp[u][1] += dp[v][0]; // u选时，子节点都不能选
36         }
37     };
38     dfs(root); // 从根节点开始深度优先搜索
39     cout << max(dp[root][0], dp[root][1]) << "\n"; // 输出根节点选或不选
        的最大值
40     return 0;
41 }

```

13.2 树的直径

解决问题：在树中找出一条最长的路径（任意两点间的最远距离）。适用于网络布线、地图上最远两个城市等场景。

题目描述

给定一棵 n 个节点的树，每条边有长度（默认长度为 1），求树中最长路径的长度（即树的直径）。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n; // n: 树中节点的数量
5         cin >> n;
6         vector<vector<int>> g(n + 1); // g[u]: 存储与节点u相邻的所有节点
7         for (int i = 0; i < n - 1; i++) {
8             int u, v; // u, v: 树上的一条无向边
9             cin >> u >> v;
10            g[u].push_back(v);
11            g[v].push_back(u);
12        }
13        int diameter = 0; // diameter: 树的直径（最长路径长度）
14        // 核心原理：DFS返回从当前节点出发能走到的最远距离
15        // 直径 = 经过每个节点的最远距离 + 次远距离 的最大值
16        function<int(int, int)> dfs = [&](int u, int parent) {
17            int max1 = 0, max2 = 0; // max1: 从u出发的最长距离, max2: 从u出发
                的第二长距离

```

```

18         for (int v : g[u]) { // v: 节点u的所有相邻节点
19             if (v == parent) continue; // 跳过父节点, 防止向上回走
20             int dist = dfs(v, u) + 1; // 从子节点v返回的最远距离加上当前边
21             if (dist > max1) { // 如果当前距离比最大值还大, 更新最大值和次
                大值
22                 max2 = max1;
23                 max1 = dist;
24             } else if (dist > max2) { // 如果只比次大值大, 更新次大值
25                 max2 = dist;
26             }
27         }
28         diameter = max(diameter, max1 + max2); // 经过节点u的最长路径 =
                max1 + max2
29         return max1; // 返回从u出发的最长距离
30     };
31     dfs(1, -1); // 从任意节点开始DFS, 通常选1, -1表示无父节点
32     cout << diameter << "\n";
33     return 0;
34 }

```

13.3 树形背包

解决问题: 在树上做背包选择, 每个节点选或不选会影响子节点, 同时有容量限制。适用于选课系统 (先修课)、依赖安装等场景。

题目描述

有 n 门课程, 课程之间有依赖关系 (形成一棵树)。每门课程有学分, 要选修一门课程必须先修其父课程。总共最多选 m 门课, 求能获得的最大总学分。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n, m; // n: 课程数量, m: 最多能选的课程数量
5         cin >> n >> m;
6         vector<vector<int>> g(n + 1); // g[pre]: 以pre为前置课程的课程列表
7         vector<int> score(n + 1, 0); // score[i]: 第i门课程的学分
8         for (int i = 1; i <= n; i++) {
9             int pre; // pre: 课程i的前置课程编号
10            cin >> pre >> score[i];
11            g[pre].push_back(i); // 将课程i添加到前置课程pre的子课程列表中

```

```
12     }
13     // dp[u][j]: 在以u为根的子树中, 恰好选择j门课程 (且必须选u) 的最大总学
        分
14     vector<vector<int>> dp(n + 1, vector<int>(m + 2, 0));
15     // 核心原理: 依赖关系形成树, 选子节点必须先选父节点
16     // 使用分组背包的思想, 将每个子节点视为一组物品, 合并到父节点上
17     function<void(int)> dfs = [&](int u) {
18         for (int i = m; i >= 1; i--) { // 初始化: 强制选择当前节点u
19             dp[u][i] = score[u];    // 选了u, 获得其学分, 占用1个名额
20         }
21         dp[u][0] = 0; // 不选u时, 学分是0
22         for (int v : g[u]) { // v: 当前节点u的所有子节点
23             dfs(v); // 递归处理子节点v
24             // 合并子节点v的DP结果到当前节点u (分组背包, 逆序防止重复)
25             for (int i = m; i >= 1; i--) { // i: 当前已选的总课程数
26                 for (int k = 1; k <= i; k++) { // k: 从子节点v的子树中选的
                    课程数
27                     dp[u][i] = max(dp[u][i], dp[u][i - k] + dp[v][k]);
28                 }
29             }
30         }
31     };
32     dfs(0); // 从虚拟根节点0开始深度优先搜索
33     cout << dp[0][m] << "\n"; // 输出选m门课的最大总学分
34     return 0;
35 }
```

14 数位 DP

14.1 统计 1 的个数

解决问题：统计区间 $[L, R]$ 内满足某种数字条件的数的个数。适用于数字统计、密码学、电话号码分析等场景。

题目描述

给定两个整数 L 和 R ，统计区间 $[L, R]$ 内所有数字中，数字 1 出现的总次数。

输入示例

1 13

输出示例

6

输出解释

1,2,3,4,5,6,7,8,9,10,11,12,13 中 1 出现了 6 次（1,10,11 中的两个 1,12,13）

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      // 核心原理：数位DP，按位处理数字，利用记忆化搜索统计
4      // 计算1~x中数字1出现的总次数
5      int countDigitOne(int x) {
6          if (x <= 0) return 0;
7          string s = to_string(x); // s: 将数字x转换为十进制字符串表示
8          int n = s.size();        // n: 数字x的位数
9          // memo[pos][cnt][isLimit][isNum]: 记忆化数组
10         // pos: 当前处理到的位数 (0~n-1)
11         // cnt: 已经出现的数字1的个数
12         // isLimit: 当前位是否受上界约束 (0/1)
13         // isNum: 当前是否已经开始填写有效数字 (用于处理前导零)
14         vector<vector<vector<vector<int>>>> memo(
15             n, vector<vector<vector<int>>>>(
16                 n + 1, vector<vector<int>>>(2, vector<int>(2, -1)))));
17         function<int(int, int, bool, bool)> dfs = [&](int pos, int cnt, bool
18             isLimit, bool isNum) {
19             if (pos == n) return cnt; // 所有位都处理完毕，返回当前的1的个数

```

```

19         if (memo[pos][cnt][isLimit][isNum] != -1)
20             return memo[pos][cnt][isLimit][isNum];
21         int res = 0;
22         int limit = isLimit ? s[pos] - '0' : 9; // limit: 当前位能填的最大
           数字
23         for (int d = 0; d <= limit; d++) { // d: 当前位尝试填的数字
24             bool nextLimit = isLimit && (d == limit); // 下一位是否受限制
25             bool nextNum = isNum || (d != 0); // 下一位是否开始有非零数
           字
26             int nextCnt = cnt;
27             if (nextNum && d == 1) nextCnt++; // 如果当前位是1且是有效数
           字, 计数加1
28             res += dfs(pos + 1, nextCnt, nextLimit, nextNum);
29         }
30         return memo[pos][cnt][isLimit][isNum] = res;
31     };
32     return dfs(0, 0, true, false);
33 }
34 int main() {
35     int L, R; // L: 区间左边界, R: 区间右边界
36     cin >> L >> R;
37     // 利用前缀和思想: [L, R]的答案 = [1, R]的答案 - [1, L-1]的答案
38     int ans = countDigitOne(R) - countDigitOne(L - 1);
39     cout << ans << "\n";
40     return 0;
41 }

```

14.2 不含连续 1 的数字

解决问题：统计区间内二进制表示中不含连续 1 的数的个数。适用于数字编码、信号处理等场景。

题目描述

给定一个正整数 n ，统计 $[0, n]$ 范围内有多少个数的二进制表示中不包含连续的 1。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int findIntegers(int n) {
4         string s; // s: 存储n的二进制字符串表示
5         int tmp = n;

```

```

6      while (tmp) { // 将n转换为二进制字符串（从低位到高位）
7          s += to_string(tmp & 1); // 取最低位
8          tmp >>= 1; // 右移一位
9      }
10     reverse(s.begin(), s.end()); // 反转得到从高位到低位的正确顺序
11     int len = s.size(); // len: 二进制表示的位数
12     // memo[pos][prev][isLimit]: 记忆化数组
13     vector<vector<vector<int>>> memo(len, vector<vector<int>>(2, vector<
14         int>(2, -1)));
15     // 核心原理：按位处理二进制表示，确保不出现连续的1
16     function<int(int, int, bool)> dfs = [&](int pos, int prev, bool
17         isLimit) {
18         if (pos == len) return 1; // 成功构造出一个合法数字
19         if (memo[pos][prev][isLimit] != -1)
20             return memo[pos][prev][isLimit];
21         int limit = isLimit ? s[pos] - '0' : 1; // limit: 当前位能填的最大
22             数字（二进制只能0或1）
23         int res = 0;
24         for (int d = 0; d <= limit; d++) { // d: 当前位尝试填的数字（0或1
25             ）
26             if (prev == 1 && d == 1) continue; // 如果上一位是1且当前位也是
27                 1，则出现连续1，跳过
28             bool nextLimit = isLimit && (d == limit);
29             res += dfs(pos + 1, d, nextLimit);
30         }
31         return memo[pos][prev][isLimit] = res;
32     };
33     return dfs(0, 0, true);
34 }
35
36 int main() {
37     int n; // n: 上界数字
38     cin >> n;
39     cout << findIntegers(n) << "\n";
40     return 0;
41 }

```

15 状态压缩 DP

15.1 TSP 问题（旅行商问题）

解决问题：给定 n 个城市和城市间的距离，从某个城市出发，经过每个城市恰好一次后回到起点，求最短路径。适用于物流配送、电路板钻孔、旅游路线规划等场景。

题目描述

有 n 个城市（编号 $0 \sim n-1$ ），给出城市间的距离矩阵 $dist[i][j]$ 。求从城市 0 出发，经过每个城市恰好一次，最后回到城市 0 的最短路径长度。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 城市的数量
5          cin >> n;
6          vector<vector<int>> dist(n, vector<int>(n)); // dist[i][j]: 城市i到城
              市j的距离
7          for (int i = 0; i < n; i++) {
8              for (int j = 0; j < n; j++) {
9                  cin >> dist[i][j];
10             }
11         }
12         // dp[mask][i]: 已经访问过的城市集合为mask, 当前处于城市i时的最小路径花
              费
13         // mask是一个二进制状态, 第j位为1表示城市j已经访问过
14         vector<vector<int>> dp(1 << n, vector<int>(n, 1e9));
15         dp[1][0] = 0; // 初始状态: 只访问了城市0, 当前在城市0, 花费为0
16         // 核心原理: 枚举所有已访问集合mask和当前城市i, 尝试前往下一个未访问城
              市j
17         for (int mask = 1; mask < (1 << n); mask++) { // mask: 已访问城市的二
              进制状态
18             for (int i = 0; i < n; i++) { // i: 当前所在的城市编号
19                 if (!(mask >> i & 1)) continue; // 如果城市i不在已访问集合中,
                    跳过
20                 if (dp[mask][i] >= 1e9) continue; // 如果当前状态不可达到, 跳过
21                 for (int j = 0; j < n; j++) { // j: 下一个要访问的城市编号
22                     if (mask >> j & 1) continue; // 如果城市j已经访问过, 跳过
23                     int nextMask = mask | (1 << j); // 更新集合状态, 加入城市j
24                     // 状态转移: 到达j的花费 = 当前花费 + i到j的距离

```

```

25         dp[nextMask][j] = min(dp[nextMask][j], dp[mask][i] + dist[
26             i][j]);
27     }
28 }
29 int ans = 1e9;
30 int fullMask = (1 << n) - 1; // fullMask: 所有城市都访问过的状态 (全1)
31 for (int i = 0; i < n; i++) { // 枚举最后停留的城市, 加上返回起点的距离
32     ans = min(ans, dp[fullMask][i] + dist[i][0]);
33 }
34 cout << ans << "\n";
35 return 0;
36 }

```

15.2 棋盘覆盖（铺砖问题）

解决问题：给定 $n \times m$ 的棋盘，用 1×2 的骨牌覆盖，求覆盖方案数。适用于地板铺装、电路板设计等场景。

题目描述

用 1×2 的骨牌覆盖一个 $n \times m$ 的棋盘，骨牌可以横着放或竖着放，求完全覆盖棋盘的总方案数。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n, m; // n: 棋盘行数, m: 棋盘列数
5         cin >> n >> m;
6         if (n > m) swap(n, m); // 优化: 让n作为较短的维度, 减少状态数
7         // trans[cur]: 从当前列状态cur可以转移到的下一列状态列表
8         // 状态是一个n位二进制数, 1表示该格已被覆盖
9         vector<vector<int>>> trans(1 << n);
10        // 核心原理: 逐列处理, 枚举当前列的状态, 通过DFS生成合法的下一列状态
11        for (int mask = 0; mask < (1 << n); mask++) { // mask: 当前列的覆盖状态
12            function<void(int, int, int, int)> dfs = [&](int pos, int cur,
13                int nxt, int cnt) {

```



```

14         trans[cur].push_back(nxt);
15         return;
16     }
17     if (cur >> pos & 1) { // 当前行在当前列已被覆盖，直接跳过
18         dfs(pos + 1, cur, nxt, cnt);
19         return;
20     }
21     // 竖着放骨牌：占当前位置和下一列的相同行
22     dfs(pos + 1, cur, nxt | (1 << pos), cnt);
23     // 横着放骨牌：占当前位置和当前列的下一行
24     if (pos + 1 < n && !(cur >> pos & 1) && !(cur >> (pos + 1) &
25         1)) {
26         dfs(pos + 1, cur, nxt, cnt);
27     }
28     };
29     dfs(0, mask, 0, 0);
30 }
31 // dp[col][mask]: 处理完第col列后，当前列覆盖状态为mask的方案数
32 vector<vector<long long>> dp(m + 1, vector<long long>(1 << n, 0));
33 dp[0][(1 << n) - 1] = 1; // 第0列被虚拟为完全覆盖状态
34 for (int col = 0; col < m; col++) { // col: 当前正在处理的列索引
35     for (int mask = 0; mask < (1 << n); mask++) { // mask: 当前列的覆
36         盖状态
37         if (dp[col][mask] == 0) continue;
38         for (int nxt : trans[mask]) { // nxt: 下一列能够达到的状态
39             dp[col + 1][nxt] += dp[col][mask];
40         }
41     }
42 }
43 cout << dp[m][(1 << n) - 1] << "\n"; // 处理完m列，最后一列应为完全覆
44 盖状态
45 return 0;
46 }

```

16 概率 DP/期望 DP

16.1 掷骰子游戏

解决问题：计算达到某个状态所需的期望步数。适用于游戏期望计算、随机过程分析等场景。

题目描述

有一个 n 面的骰子（点数 $1 \sim n$ ），初始分数为 0。每次掷骰子，掷出 k 分就增加 k 分。求分数首次达到或超过 S 时，掷骰子次数的期望值。

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, S; // n: 骰子的面数, S: 目标分数
5          cin >> n >> S;
6          // E[i]: 从当前分数i开始, 达到或超过目标分数S所需的期望掷骰子次数
7          vector<double> E(S + n + 1, 0);
8          // 核心原理: 期望DP通常采用逆推
9          // 对于已经是终点的状态, 期望为0
10         for (int i = S; i <= S + n; i++) {
11             E[i] = 0; // 分数已经达到或超过S, 不需要再掷骰子
12         }
13         for (int i = S - 1; i >= 0; i--) { // i: 当前的分数, 从后向前递推
14             double sum = 0;
15             // 掷一次骰子, 可能掷出1~n点, 等概率1/n
16             for (int k = 1; k <= n; k++) { // k: 掷出的点数
17                 sum += E[i + k]; // 转移到新状态i+k的期望
18             }
19             // 期望公式: E[i] = 1 + (E[i+1] + ... + E[i+n]) / n
20             // 1表示当前这次掷骰子
21             E[i] = 1 + sum / n;
22         }
23         cout << fixed << setprecision(2) << E[0] << "\n";
24         return 0;
25     }
```

16.2 地牢游戏

解决问题：计算从起点到终点的成功概率。适用于游戏关卡通过率计算、风险评估等场景。

题目描述

有一个 $n \times m$ 的网格，从 $(0, 0)$ 出发，要走到 $(n - 1, m - 1)$ 。每次可以向下或向右移动一格。某些格子有陷阱，不能进入。求到达终点的概率（假设在可走的方向中随机选择）。

```

1      #include <bits/stdc++.h>
2
3      using namespace std;
4
5      int main() {
6          int n, m; // n: 网格行数, m: 网格列数
7          cin >> n >> m;
8          vector<vector<int>> grid(n, vector<int>(m)); // grid[i][j]: 0表示可
9              走, 1表示陷阱
10
11         for (int i = 0; i < n; i++) {
12             for (int j = 0; j < m; j++) {
13                 cin >> grid[i][j];
14             }
15         }
16
17         // dp[i][j]: 从格子(i,j)出发到达终点的概率
18         vector<vector<double>> dp(n + 1, vector<double>(m + 1, 0));
19         if (grid[n - 1][m - 1] == 0) {
20             dp[n - 1][m - 1] = 1; // 终点到达自身的概率为1
21         }
22
23         // 核心原理: 从终点向起点逆推, 每个格子的概率是可走方向概率的平均值
24         for (int i = n - 1; i >= 0; i--) { // i: 当前格子的行号, 从下到上
25             for (int j = m - 1; j >= 0; j--) { // j: 当前格子的列号, 从右到左
26                 if (i == n - 1 && j == m - 1) continue; // 终点已处理, 跳过
27                 if (grid[i][j] == 1) {
28                     dp[i][j] = 0; // 陷阱格子, 无法通过
29                     continue;
30                 }
31
32                 int cnt = 0; // cnt: 从当前格子可走的方向数量
33                 double sum = 0; // sum: 可走方向的概率之和
34                 if (i + 1 < n && grid[i + 1][j] == 0) { // 可以向下走
35                     cnt++;
36                     sum += dp[i + 1][j];
37                 }
38             }
39         }

```

```
31         if (j + 1 < m && grid[i][j + 1] == 0) { // 可以向右走
32             cnt++;
33             sum += dp[i][j + 1];
34         }
35         if (cnt > 0) {
36             dp[i][j] = sum / cnt; // 平均概率
37         } else {
38             dp[i][j] = 0; // 无路可走，概率为0
39         }
40     }
41 }
42 cout << fixed << setprecision(2) << dp[0][0] << "\n";
43 return 0;
44 }
```

17 DP 优化技巧

17.1 单调队列优化

解决问题：在 DP 转移中，状态转移要求一个滑动窗口内的最值，可用单调队列将 $O(n^2)$ 优化到 $O(n)$ 。适用于股票交易、连续子段和等问题。

题目描述

给定一个数组，求所有长度为 k 的连续子数组的最大值。

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, k; // n: 数组长度, k: 滑动窗口大小
5          cin >> n >> k;
6          vector<int> a(n); // a: 存储原始数组
7          for (int i = 0; i < n; i++) {
8              cin >> a[i];
9          }
10         deque<int> dq; // dq: 单调队列, 存储元素索引, 队首到队尾对应数组值递减
11         vector<int> ans; // ans: 存储每个滑动窗口的最大值
12         // 核心原理: 维护一个递减队列, 队首始终是当前窗口的最大值
13         for (int i = 0; i < n; i++) { // i: 当前处理元素的索引
14             // 移除队列中已滑出窗口的索引 (索引 <= i-k)
15             while (!dq.empty() && dq.front() <= i - k) {
16                 dq.pop_front();
17             }
18             // 从队尾移除所有小于等于当前元素的值 (维护递减性)
19             while (!dq.empty() && a[dq.back()] <= a[i]) {
20                 dq.pop_back();
21             }
22             dq.push_back(i); // 将当前元素索引入队
23             // 当窗口形成后 (i >= k-1), 记录队首元素的值 (当前窗口最大值)
24             if (i >= k - 1) {
25                 ans.push_back(a[dq.front()]);
26             }
27         }
28         for (int i = 0; i < ans.size(); i++) {
29             cout << ans[i] << " \n"[i == ans.size() - 1];
30         }
31         return 0;
```

32

}

17.2 斜率优化

解决问题：DP 转移方程形如 $dp[i] = \min(dp[j] + a[i] * b[j] + c[j]) + d[i]$ ，可用斜率优化将 $O(n^2)$ 优化到 $O(n)$ 。适用于任务调度、生产线优化等场景。

题目描述

给定 n 个任务，每个任务有耗时 $t[i]$ 和费用 $c[i]$ 。将任务分成若干段，每段的代价为 (该段总时间的平方加上该段费用总和)，求最小总代价。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 任务的数量
5          cin >> n;
6          vector<long long> t(n + 1), c(n + 1); // t[i]: 第i个任务耗时, c[i]:
           第i个任务的费用
7          vector<long long> pt(n + 1, 0), pc(n + 1, 0); // pt: 时间前缀和, pc:
           费用前缀和
8          for (int i = 1; i <= n; i++) {
9              cin >> t[i] >> c[i];
10             pt[i] = pt[i - 1] + t[i]; // 计算时间前缀和
11             pc[i] = pc[i - 1] + c[i]; // 计算费用前缀和
12         }
13         vector<long long> dp(n + 1, 0); // dp[i]: 前i个任务的最小总代价
14         deque<int> dq; // dq: 单调队列, 存储可能的决策点索引
15         dq.push_back(0); // 初始决策点0
16         // 核心原理: 将DP转移方程转化为斜率形式, 维护下凸包
17         // 转移方程:  $dp[i] = \min(dp[j] + (pt[i] - pt[j])^2 + (pc[i] - pc[j]))$ 
18         // 转化为:  $dp[j] + pt[j]^2 - pc[j] = 2 * pt[i] * pt[j] + (dp[i] - pt[i]^2 - pc[i])$ 
19         for (int i = 1; i <= n; i++) { // i: 当前处理的任务位置
20             // 维护队首: 移除不是最优的决策点 (斜率比较)
21             while (dq.size() >= 2) {
22                 int j1 = dq[0], j2 = dq[1];
23                 long long val1 = dp[j1] + pt[j1] * pt[j1] - pc[j1];
24                 long long val2 = dp[j2] + pt[j2] * pt[j2] - pc[j2];
25                 // 如果斜率  $(val2 - val1) / (pt[j2] - pt[j1]) <= 2 * pt[i]$ , 则j1不是最
                   优

```

```

26         if (val2 - val1 <= 2 * pt[i] * (pt[j2] - pt[j1])) {
27             dq.pop_front();
28         } else {
29             break;
30         }
31     }
32     int j = dq.front(); // j: 当前最优决策点
33     // 计算dp[i]的值
34     dp[i] = dp[j] + (pt[i] - pt[j]) * (pt[i] - pt[j]) + (pc[i] - pc[j]
35         ]);
36     // 维护队尾: 确保凸包性质 (新决策点i加入后斜率递增)
37     while (dq.size() >= 2) {
38         int j1 = dq[dq.size() - 2], j2 = dq.back();
39         long long val1 = dp[j1] + pt[j1] * pt[j1] - pc[j1];
40         long long val2 = dp[j2] + pt[j2] * pt[j2] - pc[j2];
41         long long val3 = dp[i] + pt[i] * pt[i] - pc[i];
42         // 检查新点i是否使j2不再是凸包点
43         if ((val2 - val1) * (pt[i] - pt[j2]) >= (val3 - val2) * (pt[j2]
44             ] - pt[j1])) {
45             dq.pop_back();
46         } else {
47             break;
48         }
49     }
50     dq.push_back(i); // 将当前i作为新决策点加入队列
51     cout << dp[n] << "\n";
52     return 0;
53 }

```

18 DP 算法总结

类型	核心思想	时间复杂度	典型例题
线性 DP	按顺序递推	$O(n^2)$ 或 $O(n)$	LIS, LCS, 最大子数组和
背包 DP	容量作为状态	$O(nW)$	0-1 背包, 完全背包, 多重背包
区间 DP	枚举区间长度和分界点	$O(n^3)$	石子合并, 矩阵链乘
树形 DP	DFS 后序遍历	$O(n)$ 或 $O(nm)$	树的最大独立集, 树形背包
数位 DP	按位递推	$O(\text{位数} \times \text{状态})$	数字统计, 不含连续 1
状压 DP	二进制枚举状态	$O(2^n \times n)$	TSP, 棋盘覆盖
概率 DP	期望递推	$O(n)$ 或 $O(n^2)$	掷骰子期望, 迷宫概率

DP 解题步骤

1. 确定状态表示 (dp 数组的含义)
2. 确定状态转移方程 (如何从子问题推导)
3. 确定初始化条件 (边界情况)
4. 确定遍历顺序 (确保子问题已计算)
5. 优化 (空间优化、单调队列、斜率优化等)

常见优化技巧

- 滚动数组: 降低空间复杂度
- 单调队列: 处理滑动窗口最值
- 斜率优化: 处理形如 $dp[i] = \min(dp[j] + a[i] * b[j])$ 的转移
- 四边形不等式优化: 区间 DP 的决策单调性优化
- 矩阵快速幂优化: 线性递推的加速

第四部分 数据结构专题

19 初级数据结构

19.1 栈——括号匹配

解决问题：验证括号序列是否合法，即每个左括号是否都有对应的右括号且类型匹配。适用于编译原理语法检查、XML/HTML 标签匹配等。

题目描述

给定一个仅包含字符 '('、')'、 '['、']'、 '{'、'}' 的字符串 s ，判断括号序列是否合法。合法条件：每个左括号必须由相同类型的右括号闭合，且括号嵌套顺序正确。

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s;          // s: 输入的括号字符串
5          cin >> s;
6          stack<char> st;     // st: 栈，用来存放未匹配的左括号
7          bool ok = true;    // ok: 记录括号序列是否合法
8          for (char c : s) {  // c: 当前字符
9              if (c == '(' || c == '[' || c == '{') {
10                 st.push(c);  // 左括号直接压栈
11             } else {
12                 if (st.empty()) { ok = false; break; } // 栈空，无匹配左括
13                     号
14                 char top = st.top();          // top: 栈顶左括号
15                 if ((c == ')' && top != '(') ||
16                     (c == ']' && top != '[') ||
17                     (c == '}' && top != '{')) {
18                     ok = false; break;        // 类型不匹配，非法
19                 }
20                 st.pop();    // 匹配成功，弹出左括号
21             }
22             if (!st.empty()) ok = false; // 最后栈不空说明有左括号未被匹配
23             cout << (ok ? "YES" : "NO") << "\n";
24             return 0;
25     }
```

19.2 栈——单调栈求下一个更大元素

解决问题：对数组中每个元素，找到其右侧第一个比它大的元素。适用于股票价格跨度、每日温度等待天数等场景。

题目描述

给定长度为 n 的数组 a ，对于每个位置 i ，找出 $a[i]$ 右侧第一个大于 $a[i]$ 的元素；若不存在则输出 -1 。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n;                // n: 数组长度
5          cin >> n;
6          vector<int> a(n);      // a: 输入数组
7          for (int i = 0; i < n; i++) cin >> a[i];
8          vector<int> ans(n, -1); // ans[i]: a[i] 右侧第一个更大的元素，初始
                                // 为-1
9          stack<int> st;         // st: 单调栈，存放下标，栈底到栈顶对应值递
                                // 减
10         for (int i = 0; i < n; i++) { // i: 当前元素下标
11             while (!st.empty() && a[st.top()] < a[i]) {
12                 ans[st.top()] = a[i]; // 当前元素是栈顶元素右侧第一个更大的
13                 st.pop();
14             }
15             st.push(i);
16         }
17         for (int i = 0; i < n; i++) cout << ans[i] << " \n"[i==n-1];
18         return 0;
19     }

```

19.3 队列——滑动窗口最大值

解决问题：在长度为 n 的数组中，求所有长度为 k 的连续子数组的最大值。适用于数据流分析、信号处理等滑动窗口场景。

题目描述

给定数组 a 和窗口大小 k ，输出每个窗口内的最大值。

```

1      #include <bits/stdc++.h>
2      using namespace std;

```

```

3      int main() {
4          int n, k;                // n: 数组长度, k: 窗口大小
5          cin >> n >> k;
6          vector<int> a(n);        // a: 输入数组
7          for (int i = 0; i < n; i++) cin >> a[i];
8          deque<int> dq;           // dq: 双向队列, 存放下标, 队首为当前窗口最
                                   // 大值下标
9          for (int i = 0; i < n; i++) {
10             while (!dq.empty() && dq.front() <= i - k) dq.pop_front(); //
                                   // 移除超出窗口的下标
11             while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); //
                                   // 保持队列递减
12             dq.push_back(i);
13             if (i >= k - 1) cout << a[dq.front()] << " "; // 窗口形成后输
                                   // 出最大值
14         }
15         return 0;
16     }

```

20 高级数据结构

20.1 树状数组——单点修改区间求和

解决问题：对一个数组进行单点更新，并快速查询前缀和与区间和。适用于排行榜分数更新、实时数据统计等场景。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int n;                // n: 数组大小
4      vector<int> bit;       // bit: 树状数组 (1-indexed)
5      void add(int idx, int val) { // 在位置idx增加val
6          while (idx <= n) { bit[idx] += val; idx += idx & -idx; }
7      }
8      int sum(int idx) {      // 查询前缀和[1..idx]
9          int res = 0;
10         while (idx > 0) { res += bit[idx]; idx -= idx & -idx; }
11         return res;
12     }
13     int main() {
14         int m;              // m: 操作次数

```

```

15     cin >> n >> m;
16     bit.assign(n+1, 0);
17     while (m--) {
18         int op;           // op: 操作类型 1-增加 2-查询
19         cin >> op;
20         if (op == 1) {
21             int x, v;      // x: 位置, v: 增量
22             cin >> x >> v;
23             add(x, v);
24         } else {
25             int l, r;      // l,r: 查询区间
26             cin >> l >> r;
27             cout << sum(r) - sum(l-1) << " ";
28         }
29     }
30     return 0;
31 }

```

20.2 线段树——区间最值与单点修改

解决问题：在数组上支持单点修改并快速查询任意区间的最小值（或最大值）。适用于动态区间统计、RMQ 问题等。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     const int INF = 1e9;
4     vector<int> seg;           // seg: 线段树数组, 存储区间最小值
5     int n;                    // n: 原数组大小
6     void build(int idx, int l, int r, vector<int>& a) {
7         if (l == r) { seg[idx] = a[l]; return; }
8         int mid = (l+r)/2;
9         build(idx*2, l, mid, a); build(idx*2+1, mid+1, r, a);
10        seg[idx] = min(seg[idx*2], seg[idx*2+1]);
11    }
12    void update(int idx, int l, int r, int pos, int val) {
13        if (l == r) { seg[idx] = val; return; }
14        int mid = (l+r)/2;
15        if (pos <= mid) update(idx*2, l, mid, pos, val);
16        else update(idx*2+1, mid+1, r, pos, val);
17        seg[idx] = min(seg[idx*2], seg[idx*2+1]);
18    }

```

```

19     int query(int idx, int l, int r, int ql, int qr) {
20         if (ql <= l && r <= qr) return seg[idx];
21         int mid = (l+r)/2, res = INF;
22         if (ql <= mid) res = min(res, query(idx*2, l, mid, ql, qr));
23         if (qr > mid) res = min(res, query(idx*2+1, mid+1, r, ql, qr));
24         return res;
25     }
26     int main() {
27         int m;                // m: 操作数
28         cin >> n >> m;
29         vector<int> a(n+1);    // a: 原始数组 (1-indexed)
30         for (int i = 1; i <= n; i++) cin >> a[i];
31         seg.resize(4*n+5);
32         build(1, 1, n, a);
33         while (m--) {
34             int op;            // op: 操作类型 1-单点修改 2-区间查询
35             cin >> op;
36             if (op == 1) {
37                 int x, v;      // x: 修改位置, v: 新值
38                 cin >> x >> v;
39                 update(1, 1, n, x, v);
40             } else {
41                 int l, r;      // l,r: 查询区间
42                 cin >> l >> r;
43                 cout << query(1, 1, n, l, r) << " ";
44             }
45         }
46         return 0;
47     }

```

20.3 平衡树 (Treap) ——排名查询与第 k 大

解决问题：维护一个有序的可重复集合，支持插入、删除、查询排名、根据排名查数值
 值 求前驱后继 适用于实时排名系统 数据流中位数维护等

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Node {
4         int val, rnd, sz, cnt; // val: 节点值, rnd: 随机优先级, sz: 子树大小, cnt: 重复次数
5         Node *l, *r;

```

```

6         Node(int v) : val(v), rnd(rand()), sz(1), cnt(1), l(nullptr), r(
          nullptr) {}
7     };
8     int getSz(Node* t) { return t ? t->sz : 0; }
9     void upd(Node* t) { if (t) t->sz = t->cnt + getSz(t->l) + getSz(t->r
        ); }
10    void split(Node* t, int key, Node*& a, Node*& b) { // 按值分裂, <=key
        进a, >key进b
11        if (!t) { a = b = nullptr; return; }
12        if (t->val <= key) { a = t; split(t->r, key, a->r, b); }
13        else { b = t; split(t->l, key, a, b->l); }
14        upd(a); upd(b);
15    }
16    Node* merge(Node* a, Node* b) { // 合并两棵树, 要求a中所有值 <= b中所
        有值
17        if (!a || !b) return a ? a : b;
18        if (a->rnd < b->rnd) { a->r = merge(a->r, b); upd(a); return a; }
19        else { b->l = merge(a, b->l); upd(b); return b; }
20    }
21    void insert(Node*& root, int val) {
22        Node *a, *b, *c;
23        split(root, val-1, a, b); split(b, val, b, c);
24        if (b) { b->cnt++; b->sz++; }
25        else b = new Node(val);
26        root = merge(merge(a, b), c);
27    }
28    void erase(Node*& root, int val) {
29        Node *a, *b, *c;
30        split(root, val-1, a, b); split(b, val, b, c);
31        if (b) { b->cnt--; b->sz--; if (b->cnt == 0) b = nullptr; }
32        root = merge(merge(a, b), c);
33    }
34    int kth(Node* t, int k) { // 查询树中第k小的值
35        if (!t) return -1;
36        int ls = getSz(t->l);
37        if (k <= ls) return kth(t->l, k);
38        if (k <= ls + t->cnt) return t->val;
39        return kth(t->r, k - ls - t->cnt);
40    }
41    int smallerCount(Node*& root, int val) { // 小于val的数的个数

```

```

42     Node *a, *b;
43     split(root, val-1, a, b);
44     int res = getSz(a);
45     root = merge(a, b);
46     return res;
47 }
48 int precursor(Node* root, int val) { // 求前驱: 小于val的最大值
49     int k = smallerCount(root, val);
50     if (k == 0) return -1;
51     return kth(root, k);
52 }
53 int successor(Node* root, int val) { // 求后继: 大于val的最小值
54     int k = smallerCount(root, val+1);
55     if (k == getSz(root)) return -1;
56     return kth(root, k+1);
57 }
58 int main() {
59     srand(time(0));
60     Node* root = nullptr; // root: Treap的根节点
61     int m;                // m: 操作数
62     cin >> m;
63     while (m--) {
64         int op, x;        // op: 操作码, x: 操作数
65         cin >> op >> x;
66         if (op == 1) insert(root, x);
67         else if (op == 2) erase(root, x);
68         else if (op == 3) cout << smallerCount(root, x) + 1 << " ";
69         else if (op == 4) cout << kth(root, x) << " ";
70         else if (op == 5) cout << precursor(root, x) << " ";
71         else if (op == 6) cout << successor(root, x) << " ";
72     }
73     return 0;
74 }

```

21 算法总结表

数据结构	核心操作	时间复杂度	典型应用场景
栈	push/pop/top	$O(1)$	括号匹配、表达式求值、单调栈
队列	push/pop/front	$O(1)$	滑动窗口、BFS、任务调度
并查集	find/unite	$O(\alpha(n))$	动态连通性、最小生成树、集合合并
哈希表	insert/find/erase	$O(1)$ 平均	两数之和、去重统计、子串匹配
堆	push/pop/top	$O(\log n)$	动态中位数、Top K、优先队列
树状数组	单点/区间更新与查询	$O(\log n)$	排行榜更新、前缀和、逆序对
线段树	区间更新与查询	$O(\log n)$	区间最值、区间求和、RMQ
Treap	插入/删除/排名/kth	$O(\log n)$ 期望	实时排名、有序集合维护
Splay	伸展/区间翻转	均摊 $O(\log n)$	序列翻转、区间操作、LCT 辅助
分块	区间加/区间和	$O(\sqrt{n})$	替代线段树、简单区间操作

第五部分 并查集算法专题

22 并查集基础

22.1 标准并查集（路径压缩 + 按秩合并）

解决问题：维护一个元素集合，支持两种操作：合并两个元素所在的集合、查询两个元素是否属于同一集合。适用于社交网络好友关系判断、连通分量统计、Kruskal 最小生成树等场景。

题目描述

给定 n 个元素（编号 $1 \sim n$ ），初始每个元素自成一个集合。有 m 次操作，操作类型有两种： $1 \ x \ y$ 表示合并 x 和 y 所在的集合； $2 \ x \ y$ 表示查询 x 和 y 是否在同一个集合中。

输入示例

```
5 6
1 1 2
1 2 3
2 1 3
1 4 5
2 1 4
2 4 5
```


输出示例

Yes No Yes

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct DSU {
4          vector<int> parent; // parent[i]: 节点i的父节点, 根节点的父节点指
           向自己
5          vector<int> rank; // rank[i]: 节点i所在的树的高度 (按秩合并时使用)
6          DSU(int n) { // 构造函数: 初始化并查集
7              parent.resize(n + 1); // 索引从1到n, 所以分配n+1个空间
8              rank.resize(n + 1, 0); // 初始所有节点高度为0
9              for (int i = 1; i <= n; i++) {
10                 parent[i] = i; // 初始时每个节点的父节点都是自己, 即每个元素
           自成一个集合
11             }
12         }
13         // 核心原理: 查找节点x所在集合的根节点 (代表元), 同时进行路径压缩优
           化
14         // 路径压缩的作用: 将查找路径上的所有节点直接指向根节点, 降低后续查
           找的时间复杂度
15         int find(int x) {
16             if (parent[x] != x) { // 如果x不是根节点
17                 parent[x] = find(parent[x]); // 递归查找父节点, 并将结果赋
           值给parent[x] (路径压缩)
18             }
19             return parent[x]; // 返回根节点
20         }
21         // 合并两个节点所在的集合
22         void unite(int x, int y) {
23             int rootX = find(x); // 找到x的根节点
24             int rootY = find(y); // 找到y的根节点
25             if (rootX == rootY) return; // 如果已经在同一集合, 无需合并
26             // 按秩合并: 将高度较小的树合并到高度较大的树, 保持树平衡
27             if (rank[rootX] < rank[rootY]) {
28                 parent[rootX] = rootY; // 将rootX的父节点设为rootY
29             } else if (rank[rootX] > rank[rootY]) {
30                 parent[rootY] = rootX; // 将rootY的父节点设为rootX
31             } else {
32                 parent[rootY] = rootX; // 高度相等时, 任意合并, 并将高度+1

```

```
33         rank[rootX]++; // 新树的高度增加1
34     }
35 }
36 // 查询两个节点是否在同一个集合中
37 bool same(int x, int y) {
38     return find(x) == find(y);
39 }
40 };
41 int main() {
42     int n, m; // n: 元素总数, m: 操作次数
43     cin >> n >> m;
44     DSU dsu(n); // 创建大小为n的并查集
45     for (int i = 0; i < m; i++) {
46         int op, x, y; // op: 操作类型, x, y: 操作的两个元素
47         cin >> op >> x >> y;
48         if (op == 1) {
49             dsu.unite(x, y); // 合并操作
50         } else {
51             if (dsu.same(x, y)) { // 查询操作
52                 cout << "Yes\n";
53             } else {
54                 cout << "No\n";
55             }
56         }
57     }
58     return 0;
59 }
```

22.2 带权并查集（维护到根节点的距离）

解决问题：维护元素之间的相对关系，如食物链中的捕食关系、向量偏移、区间和关系等。适用于解决带关系的集合合并问题，每个元素到根节点的距离有意义。

题目描述

有 n 个动物（编号 $1 \sim n$ ），被分成三类：A 吃 B，B 吃 C，C 吃 A。给出 k 句话，每句话描述 x 和 y 的关系（1 表示 x 和 y 同类，2 表示 x 吃 y ）。判断假话的数量（假话条件：与前面的话冲突，或者编号超出范围）。

输入示例

```

100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5

```

输出示例

```

3

```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct WeightedDSU {
4          vector<int> parent; // parent[i]: 节点i的父节点
5          vector<int> dist; // dist[i]: 节点i到其父节点的关系 (0表示同类, 1表示被父节点吃, 2表示吃父节点)
6          WeightedDSU(int n) { // 构造函数: 初始化带权并查集
7              parent.resize(n + 1);
8              dist.resize(n + 1, 0); // 初始到父节点的距离为0 (自己到自己是同类)
9              for (int i = 1; i <= n; i++) {
10                 parent[i] = i; // 初始每个节点自成一棵树
11             }
12         }
13         // 核心原理: 查找根节点时, 同时更新dist[x]为x到根节点的关系
14         // 关系通过路径上的关系值相加并取模3得到 (因为只有3种关系)
15         int find(int x) {
16             if (parent[x] != x) {
17                 int root = find(parent[x]); // 递归找到根节点
18                 dist[x] = (dist[x] + dist[parent[x]]) % 3; // 更新x到根节点的关系
19                 parent[x] = root; // 路径压缩, 直接指向根节点
20             }
21             return parent[x];
22         }

```

```

23 // 合并操作：根据给定关系type合并x和y
24 // type=0表示x和y同类，type=1表示x吃y
25 bool unite(int x, int y, int type) {
26     int rootX = find(x); // 找到x的根节点
27     int rootY = find(y); // 找到y的根节点
28     if (rootX == rootY) { // 已经同根，检查是否与已有关系矛盾
29         int relation = (dist[x] - dist[y] + 3) % 3; // 计算x到y的关
           系
30         return relation == type; // 如果计算出的关系与给定的type一
           致，则真话；否则假话
31     }
32     // 合并两个树：将rootX合并到rootY，并更新dist[rootX]
33     parent[rootX] = rootY;
34     // 推导dist[rootX]的公式：已知dist[x]到rootX，dist[y]到rootY
35     // 需要满足：x到y的关系 = (dist[x] + dist[rootX] - dist[y]) %
           3 = type
36     // 可得：dist[rootX] = (type + dist[y] - dist[x]) % 3
37     dist[rootX] = (type + dist[y] - dist[x] + 3) % 3;
38     return true; // 合并成功，这句话是真话
39 }
40 };
41 int main() {
42     int n, k; // n: 动物总数，k: 话的句数
43     cin >> n >> k;
44     WeightedDSU dsu(n); // 创建大小为n的带权并查集
45     int falseCount = 0; // falseCount: 假话的数量
46     for (int i = 0; i < k; i++) {
47         int d, x, y; // d: 关系类型（1或2），x, y: 话中涉及的两个动物编
           号
48         cin >> d >> x >> y;
49         if (x > n || y > n) { // 条件2: 编号超出范围，直接判为假话
50             falseCount++;
51             continue;
52         }
53         int type = d - 1; // 将1/2转换为0/1，0表示同类，1表示x吃y
54         if (!dsu.unite(x, y, type)) { // 如果与已有关系矛盾
55             falseCount++;
56         }
57     }
58     cout << falseCount << "\n";

```

```
59     return 0;  
60 }
```

23 并查集扩展应用

23.1 连通块统计

解决问题：维护并查集的同时统计每个集合的大小（元素个数）。适用于动态连通性问题中需要获取集合大小、朋友圈人数统计等场景。

题目描述

有 n 个人，初始相互不认识。每次操作添加一条朋友关系，或者查询某个人所在的朋友圈有多少人，或者查询当前最大的朋友圈有多少人。

```
1  #include <bits/stdc++.h>  
2  using namespace std;  
3  struct DSUWithSize {  
4      vector<int> parent; // parent[i]: 节点i的父节点  
5      vector<int> size; // size[i]: 节点i所在集合的元素个数（仅根节点有效）  
6      vector<int> rank; // rank[i]: 树的高度（用于按秩合并）  
7      int maxSize; // maxSize: 当前最大集合的大小  
8      DSUWithSize(int n) { // 构造函数：初始化  
9          parent.resize(n + 1);  
10         size.resize(n + 1, 1); // 初始每个集合大小为1  
11         rank.resize(n + 1, 0); // 初始高度为0  
12         maxSize = 1; // 初始最大集合大小为1  
13         for (int i = 1; i <= n; i++) {  
14             parent[i] = i; // 初始父节点指向自己  
15         }  
16     }  
17     int find(int x) { // 查找根节点（带路径压缩）  
18         if (parent[x] != x) {  
19             parent[x] = find(parent[x]); // 路径压缩  
20         }  
21         return parent[x];  
22     }  
23     // 合并两个元素所在的集合  
24     void unite(int x, int y) {
```

```
25     int rootX = find(x);
26     int rootY = find(y);
27     if (rootX == rootY) return; // 已经是同一集合
28     // 按秩合并：将高度小的树合并到高度大的树
29     if (rank[rootX] < rank[rootY]) {
30         parent[rootX] = rootY;
31         size[rootY] += size[rootX]; // 集合大小累加
32         maxSize = max(maxSize, size[rootY]); // 更新最大集合大小
33     } else if (rank[rootX] > rank[rootY]) {
34         parent[rootY] = rootX;
35         size[rootX] += size[rootY];
36         maxSize = max(maxSize, size[rootX]);
37     } else {
38         parent[rootY] = rootX;
39         size[rootX] += size[rootY];
40         rank[rootX]++; // 高度相等，新树高度+1
41         maxSize = max(maxSize, size[rootX]);
42     }
43 }
44 // 查询x所在集合的大小
45 int getSize(int x) {
46     return size[find(x)];
47 }
48 // 查询当前最大集合的大小
49 int getMaxSize() {
50     return maxSize;
51 }
52 };
53 int main() {
54     int n, m; // n: 人数, m: 操作次数
55     cin >> n >> m;
56     DSUWithSize dsu(n); // 创建带大小统计的并查集
57     for (int i = 0; i < m; i++) {
58         int op; // op: 操作类型
59         cin >> op;
60         if (op == 1) { // 添加朋友关系（合并）
61             int x, y; // x, y: 成为朋友的两个人编号
62             cin >> x >> y;
63             dsu.unite(x, y);
64         } else if (op == 2) { // 查询某人的朋友圈大小
```

```

65         int x; // x: 要查询的人编号
66         cin >> x;
67         cout << dsu.getSize(x) << "\n";
68     } else if (op == 3) { // 查询当前最大朋友圈大小
69         cout << dsu.getMaxSize() << "\n";
70     }
71 }
72 return 0;
73 }

```

23.2 并查集判环（无向图）

解决问题：在动态加边的过程中判断是否形成环。适用于检测无向图中的环、判断添加边后是否产生回路等场景。

题目描述

有 n 个节点，依次添加 m 条边。每添加一条边，判断这条边是否会形成环（即边的两个端点是否已经连通）。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct DSUCycle {
4         vector<int> parent; // parent[i]: 节点i的父节点
5         vector<int> rank; // rank[i]: 树的高度
6         DSUCycle(int n) { // 构造函数: 初始化n个节点
7             parent.resize(n + 1);
8             rank.resize(n + 1, 0);
9             for (int i = 1; i <= n; i++) {
10                 parent[i] = i; // 初始父节点指向自己
11             }
12         }
13         int find(int x) { // 查找根节点（带路径压缩）
14             if (parent[x] != x) {
15                 parent[x] = find(parent[x]);
16             }
17             return parent[x];
18         }
19         // 尝试合并两个节点，如果已经在同一集合则说明加边后会形成环
20         // 返回值: true表示会形成环, false表示不会形成环
21         bool addEdge(int x, int y) {

```

```
22         int rootX = find(x);
23         int rootY = find(y);
24         if (rootX == rootY) {
25             return true; // 已经连通，添加这条边会产生环
26         }
27         // 按秩合并
28         if (rank[rootX] < rank[rootY]) {
29             parent[rootX] = rootY;
30         } else if (rank[rootX] > rank[rootY]) {
31             parent[rootY] = rootX;
32         } else {
33             parent[rootY] = rootX;
34             rank[rootX]++;
35         }
36         return false; // 成功合并，不会产生环
37     }
38 };
39 int main() {
40     int n, m; // n: 节点总数, m: 边的数量
41     cin >> n >> m;
42     DSUCycle dsu(n); // 创建判环并查集
43     for (int i = 0; i < m; i++) {
44         int u, v; // u, v: 添加的边连接的两个节点
45         cin >> u >> v;
46         if (dsu.addEdge(u, v)) {
47             cout << "Yes\n"; // 会形成环
48         } else {
49             cout << "No\n"; // 不会形成环
50         }
51     }
52     return 0;
53 }
```

23.3 并查集求最小生成树（Kruskal 算法）

解决问题：给定一个带权无向图，求最小生成树的总权值。适用于网络建设、道路规划、电路布线等场景。

题目描述

有 n 个节点和 m 条边，每条边有权值 w 。求最小生成树的总权值（如果图不连通则输出-1）。

输入示例

```
4 5
1 2 1
1 3 2
2 3 1
2 4 3
3 4 2
```

输出示例

```
4
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Edge {
4          int u, v, w; // u: 边的起点, v: 边的终点, w: 边的权值
5          Edge(int u, int v, int w) : u(u), v(v), w(w) {}
6          bool operator<(const Edge& other) const {
7              return w < other.w; // 按权值升序排序
8          }
9      };
10     struct DSU {
11         vector<int> parent; // parent[i]: 节点i的父节点
12         vector<int> rank; // rank[i]: 树的高度
13         DSU(int n) {
14             parent.resize(n + 1);
15             rank.resize(n + 1, 0);
16             for (int i = 1; i <= n; i++) parent[i] = i;
17         }
18         int find(int x) {
19             if (parent[x] != x) parent[x] = find(parent[x]);
20             return parent[x];
21         }
22         bool unite(int x, int y) {
```

```
23         int rootX = find(x), rootY = find(y);
24         if (rootX == rootY) return false;
25         if (rank[rootX] < rank[rootY]) parent[rootX] = rootY;
26         else if (rank[rootX] > rank[rootY]) parent[rootY] = rootX;
27         else { parent[rootY] = rootX; rank[rootX]++; }
28         return true;
29     }
30 };
31 int main() {
32     int n, m; // n: 节点总数, m: 边的总数
33     cin >> n >> m;
34     vector<Edge> edges; // edges: 存储所有边
35     for (int i = 0; i < m; i++) {
36         int u, v, w; // u, v: 边连接的两个节点, w: 边权
37         cin >> u >> v >> w;
38         edges.emplace_back(u, v, w);
39     }
40     sort(edges.begin(), edges.end()); // 将边按权值从小到大排序
41     DSU dsu(n);
42     int totalWeight = 0; // totalWeight: 最小生成树的总权值
43     int edgeCount = 0; // edgeCount: 已加入生成树的边数
44     // 核心原理: Kruskal算法, 贪心地选取权值最小的边, 如果加入后不形成环
45     // 则加入
46     for (const auto& e : edges) {
47         if (dsu.unite(e.u, e.v)) { // 如果两个端点不在同一集合, 加入该
48             // 边
49             totalWeight += e.w;
50             edgeCount++;
51             if (edgeCount == n - 1) break; // 树有n-1条边时完成
52         }
53     }
54     if (edgeCount == n - 1) {
55         cout << totalWeight << "\n";
56     } else {
57         cout << -1 << "\n"; // 图不连通, 无法生成最小生成树
58     }
59     return 0;
60 }
```

24 并查集算法复杂度总结

算法变体	核心思想	时间复杂度	典型应用
标准并查集	路径压缩 + 按秩合并	近似 $O(\alpha(n))$	连通性判断、集合合并
带权并查集	维护节点到根的距离	$O(\alpha(n))$	食物链、向量关系
可持久化并查集	主席树维护版本	$O(\log n)$	版本回滚、历史状态
并查集判环	合并前检查根是否相同	$O(\alpha(n))$	无向图环检测
二分图判定	维护节点颜色关系	$O(\alpha(n))$	动态二分图检测
逆向并查集	离线倒序处理	$O(m \cdot \alpha(n))$	删除节点、删除边
Kruskal 算法	贪心选最小权边	$O(m \log m)$	最小生成树
Tarjan 离线 LCA	DFS+ 并查集	$O(n + q)$	树上最近公共祖先

并查集核心操作

1. 初始化：每个元素的父节点指向自己 ($parent[i] = i$)
2. 查找 (Find)：递归查找根节点，同时进行路径压缩
3. 合并 (Union)：找到两个元素的根节点，按秩合并到同一集合

常见优化技巧

- 路径压缩：查找时将路径上的所有节点直接指向根节点
- 按秩合并：将高度较小的树合并到高度较大的树（或按大小合并）
- 离线处理：将删除操作转换为添加操作进行处理
- 可持久化：主席树维护历史版本的父节点信息

第六部分 数学与思维题算法专题

25 基础数学思维

25.1 试除法判断质数

解决问题：快速判断一个正整数是否为质数。适用于加密算法基础、数学验证、质因子分解的预处理步骤。

题目描述

给定一个正整数 n ，判断它是否为质数。若 n 只能被 1 和自身整除，则为质数。

输入示例

17

输出示例

YES

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 待判断的整数
5          cin >> n;
6          if (n <= 1) { // 1及以下的数不是质数
7              cout << "NO\n";
8              return 0;
9          }
10         bool isPrime = true; // isPrime: 标记是否为质数，初始假设为真
11         // 核心原理：从2遍历到sqrt(n)，若存在整除则说明存在非1和本身的因子
12         for (int i = 2; i * i <= n; i++) { // i: 尝试的因子
13             if (n % i == 0) { // 找到因子，说明不是质数
14                 isPrime = false;
15                 break;
16             }
17         }
18         cout << (isPrime ? "YES" : "NO") << "\n";
19         return 0;
20     }
```

25.2 埃拉托色尼筛法

解决问题：高效求出 1 到 n 范围内的所有质数。时间复杂度 $O(n \log \log n)$ 。适用于数据范围较大的质数表生成、质数相关问题预处理。

题目描述

给定一个整数 n ，输出 1 到 n 之间的所有质数。

输入示例

20

输出示例

2 3 5 7 11 13 17 19

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 筛选范围的上界
5          cin >> n;
6          vector<bool> isPrime(n + 1, true); // isPrime[i]: 标记i是否为质
           数, 初始全为true
7          isPrime[0] = isPrime[1] = false; // 0和1不是质数
8          // 核心原理: 从小到大, 若当前数为质数, 则筛掉其所有倍数
9          for (int i = 2; i * i <= n; i++) { // i: 当前用于筛除倍数的质数
10             if (isPrime[i]) {
11                 for (int j = i * i; j <= n; j += i) { // j: i的倍数, 从i*i
                     开始更高效
12                     isPrime[j] = false; // 标记i的倍数为合数
13                 }
14             }
15         }
16         for (int i = 2; i <= n; i++) {
17             if (isPrime[i]) cout << i << " ";
18         }
19         cout << "\n";
20         return 0;
21     }
```

25.3 线性筛（欧拉筛）

解决问题：在线性时间复杂度 $O(n)$ 内求出 1 到 n 的所有质数，并能计算每个数的最小质因子。适用于需要高效筛法的大数据范围问题。

题目描述

给一个整数 n ，列出不大于 n 的所有质数。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 筛选范围的上界
5          cin >> n;
6          vector<int> primes; // primes: 存储找到的质数
7          vector<bool> isComp(n + 1, false); // isComp[i]: 标记i是否为合数
8          // 核心原理：每个合数只被其最小质因子筛掉一次，保证线性复杂度
9          for (int i = 2; i <= n; i++) { // i: 当前处理的数
10             if (!isComp[i]) primes.push_back(i); // i是质数则加入列表
11             for (int p : primes) { // p: 质数列表中的质数
12                 if (i * p > n) break; // 超过范围则停止
13                 isComp[i * p] = true; // 筛掉合数
14                 if (i % p == 0) break; // p是i的最小质因子，停止避免重复筛
15             }
16         }
17         for (int p : primes) cout << p << " ";
18         cout << "\n";
19         return 0;
20     }
```

25.4 最大公约数（辗转相除法）

解决问题：求两个整数的最大公约数（GCD）。适用于分数约分、最小公倍数计算、数论问题的基础工具。

题目描述

给定两个非负整数 a 和 b ，求它们的最大公约数。

输入示例

48 18

输出示例

6

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      // 递归实现辗转相除法，核心原理：gcd(a, b) = gcd(b, a % b)，直到b为0返回a
4      int gcd(int a, int b) { // a, b: 两个非负整数
5          return b == 0 ? a : gcd(b, a % b); // 递归直到b为0
6      }
7      int main() {
8          int a, b; // a, b: 输入的两个整数
9          cin >> a >> b;
10         cout << gcd(a, b) << "\n";
11         return 0;
12     }

```

25.5 最小公倍数

解决问题：求两个整数的最小公倍数（LCM），通过公式 $lcm(a, b) = a / gcd(a, b) \times b$ 计算。适用于数学计算、周期性事件同步问题。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
4      int lcm(int a, int b) { // a, b: 两个正整数
5          return a / gcd(a, b) * b; // 先除后乘防止溢出
6      }
7      int main() {
8          int a, b; // a, b: 输入的两个正整数
9          cin >> a >> b;
10         cout << lcm(a, b) << "\n";
11         return 0;
12     }

```

26 高级数学算法

26.1 快速幂

解决问题：高效计算 $a^b \bmod m$ 的值，时间复杂度 $O(\log b)$ 。适用于大指数取模运算、RSA 加密解密、组合数取模中求逆元等场景。

题目描述

给定三个整数 a, b, m ，求 $a^b \bmod m$ 。

输入示例

2 10 1000

输出示例

24

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      // 核心原理：将指数b转为二进制，利用  $a^{(2^k)} = (a^{(2^{(k-1)})})^2$  迭代计
      算
5      ll quickPow(ll a, ll b, ll m) { // a: 底数, b: 指数, m: 模数
6          ll res = 1; // res: 累乘结果
7          a %= m; // 先取模防止溢出
8          while (b > 0) {
9              if (b & 1) res = (res * a) % m; // 二进制当前位为1时，乘上对应
              的权重
10             a = (a * a) % m; // 底数平方，对应下一位的权重
11             b >>= 1; // 右移处理下一位
12         }
13         return res;
14     }
15     int main() {
16         ll a, b, m; // a: 底数, b: 指数, m: 模数
17         cin >> a >> b >> m;
18         cout << quickPow(a, b, m) << "\n";
19         return 0;
20     }
```


26.2 扩展欧几里得算法

解决问题：求解方程 $ax + by = \gcd(a, b)$ 的一组整数解，以及求逆元、求解线性同余方程。适用于密码学逆元计算、不定方程通解等场景。

题目描述

给定整数 a, b ，求 x, y 使得 $ax + by = \gcd(a, b)$ 。

输入示例

35 15

输出示例

gcd=5, x=1, y=-2

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      // 核心原理：在辗转相除法回溯时，倒推出x和y的递推关系
4      int exgcd(int a, int b, int& x, int& y) { // a,b: 系数, x,y: 解（引用返回）
5          if (b == 0) { // 递归基：gcd(a,0)=a, 此时ax+0*y=a -> x=1,y=0
6              x = 1; y = 0;
7              return a;
8          }
9          int x1, y1; // x1, y1: 下层递归的解
10         int d = exgcd(b, a % b, x1, y1); // d: 最终的gcd
11         x = y1; // x = 下层y1
12         y = x1 - (a / b) * y1; // y = 下层x1 - (a/b)*下层y1
13         return d;
14     }
15     int main() {
16         int a, b; // a, b: 输入的两个整数
17         cin >> a >> b;
18         int x, y; // x, y: 方程的解
19         int d = exgcd(a, b, x, y);
20         cout << "gcd=" << d << ", x=" << x << ", y=" << y << "\n";
21         return 0;
22     }

```

26.3 乘法逆元（费马小定理）

解决问题：求整数 a 在模 m 下的乘法逆元，即满足 $a \times x \equiv 1 \pmod{m}$ 的 x 。适用于组合数取模、分数取模等场景。

题目描述

给定 a 和质数 p ，求 a 模 p 的乘法逆元。保证 $\gcd(a, p) = 1$ 。

输入示例

3 7

输出示例

5

输出解释

$$3 \times 5 = 15 \equiv 1 \pmod{7}$$

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      ll quickPow(ll a, ll b, ll m) {
5          ll res = 1; a %= m;
6          while (b) { if (b & 1) res = res * a % m; a = a * a % m; b >>= 1;
7              }
8          return res;
9      }
10     int main() {
11         ll a, p; // a: 需求逆元的数, p: 模数 (质数)
12         cin >> a >> p;
13         // 费马小定理:  $a^{p-1} \equiv 1 \pmod{p}$ , 因此逆元为  $a^{p-2} \pmod{p}$ 
14         cout << quickPow(a, p - 2, p) << "\n";
15         return 0;
16     }

```

26.4 组合数（阶乘逆元法）

解决问题：在 $O(n)$ 预处理阶乘和阶乘逆元后， $O(1)$ 查询组合数 $C(n, k) \pmod{p}$ ，适用于 n 较大 ($n \leq 10^6$) 的场景。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      const int MOD = 1e9 + 7;
5      const int MAXN = 1e6 + 5;
6      ll fact[MAXN], invFact[MAXN]; // fact[i]: i的阶乘, invFact[i]: i阶乘
   // 的逆元
7      ll quickPow(ll a, ll b) { ll r = 1; a %= MOD; while(b){ if(b&1) r=r*
   // a%MOD; a=a*a%MOD; b>>=1; } return r; }
8      void precompute() {
9          fact[0] = 1;
10         for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
11         invFact[MAXN - 1] = quickPow(fact[MAXN - 1], MOD - 2);
12         for (int i = MAXN - 2; i >= 0; i--) invFact[i] = invFact[i + 1] *
   // (i + 1) % MOD;
13     }
14     ll C(int n, int k) { // C(n, k) = n! / (k! * (n-k)!)
15         if (k < 0 || k > n) return 0;
16         return fact[n] * invFact[k] % MOD * invFact[n - k] % MOD;
17     }
18     int main() {
19         precompute();
20         int n, k; // n: 总数, k: 选取数
21         cin >> n >> k;
22         cout << C(n, k) << "\n";
23         return 0;
24     }

```

26.5 博弈论——Nim 游戏

解决问题：判断在 Nim 游戏中先手是否必胜。Nim 游戏：有若干堆石子，两人轮流从任意一堆中取任意数量（至少 1 个），无法操作者输。适用于基本的公平组合游戏胜负判断。

题目描述

有 n 堆石子，第 i 堆有 a_i 个石子。两人轮流操作，每次从一堆中取至少一个石子，判先手胜负。

输入示例

```
3
2 3 4
```

输出示例

First

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 堆数
5          cin >> n;
6          int xsum = 0; // xsum: 所有堆石子数的异或和
7          // 核心原理: Nim游戏的Bouton定理: 异或非零则先手必胜
8          for (int i = 0; i < n; i++) {
9              int a; cin >> a; // a: 第i堆的石子数
10             xsum ^= a;
11         }
12         cout << (xsum ? "First" : "Second") << "\n";
13         return 0;
14     }
```

26.6 博弈论——SG 函数

解决问题: 对任意有向图游戏, 计算其 SG 函数值, 从而判断多游戏组合的胜负。适用于各种组合博弈问题 (如取石子变体、棋盘游戏等)。

题目描述

有 n 堆石子, 每堆给定数量 a_i , 每次操作可以选择一堆, 取走 $1 \sim k$ 个石子。判断先手胜负。

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, k; // n: 堆数, k: 每次最多取的石子数
5          cin >> n >> k;
6          // 核心原理: 对于限定取1~k个的巴什博弈, SG(x) = x % (k+1)
7          int xsum = 0; // xsum: 各堆SG值的异或和
```

```

8         for (int i = 0; i < n; i++) {
9             int a; cin >> a; // a: 第i堆的石子数
10            xsum ^= (a % (k + 1)); // SG值 = a % (k+1)
11        }
12        cout << (xsum ? "First" : "Second") << "\n";
13        return 0;
14    }

```

27 数学算法总结

类型	核心思想	时间复杂度	典型例题
基础数论	试除法/筛法	$O(\sqrt{n})$ 或 $O(n \log \log n)$	质数判断、 质因数分解
GCD/扩展欧几里得	辗转相除	$O(\log \min(a, b))$	最大公约数、逆元、 同余方程
快速幂/矩阵快速幂	二进制拆分	$O(\log n)$	幂取模、斐波那契、递推加速
组合数学	阶乘逆元、卢卡斯	预处理 $O(n)$, 查询 $O(1)$	组合数、排列计数
高斯消元	初等行变换	$O(n^3)$	线性方程组、异或方程
容斥原理	子集枚举	$O(2^n)$	整除统计、错排
博弈论 SG	SG 定理、mex	状态依赖	Nim 游戏、组合博弈

第七部分 图论算法专题

28 图的存储与基础遍历

28.1 深度优先搜索 (DFS)

解决问题：用于遍历图的所有顶点，常用来判断图的连通性、寻找路径、拓扑排序、检测环、求连通分量等。

题目描述

给定一个无向连通图，从顶点 1 开始进行深度优先搜索，输出遍历顺序。

输入示例

```
5 4
1 2
1 3
2 4
3 5
```

输出示例

```
1 2 4 3 5
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      vector<vector<int>>>adj; // adj[i]: 顶点i的邻接表
4      vector<bool> vis; // vis[i]: 顶点i是否已被访问
5      // DFS递归函数: 访问顶点u
6      void dfs(int u) {
7          vis[u] = true;
8          cout << u << " "; // 输出访问顺序
9          for (int v : adj[u]) { // 遍历u的所有邻居v
10             if (!vis[v]) {
11                 dfs(v); // 递归访问未访问的邻居
12             }
13         }
14     }
```

```

15     int main() {
16         int n, m; // n: 顶点数, m: 边数
17         cin >> n >> m;
18         adj.resize(n + 1);
19         vis.assign(n + 1, false);
20         for (int i = 0; i < m; i++) {
21             int u, v; // u, v: 无向边的两端
22             cin >> u >> v;
23             adj[u].push_back(v);
24             adj[v].push_back(u); // 无向图添加反向边
25         }
26         dfs(1); // 从顶点1开始DFS
27         cout << "\n";
28         return 0;
29     }

```

28.2 广度优先搜索 (BFS)

解决问题：求无权图的最短路径（边权为 1）、层次遍历、判断二分图（染色法配合 BFS）、求连通分量。BFS 按层次访问，先访问距离起点近的顶点。

题目描述

给定一个无向连通图，从顶点 1 开始进行广度优先搜索，输出遍历顺序。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n, m; // n: 顶点数, m: 边数
5         cin >> n >> m;
6         vector<vector<int>> adj(n + 1); // adj[i]: 顶点i的邻接表
7         for (int i = 0; i < m; i++) {
8             int u, v; // u, v: 无向边的两端
9             cin >> u >> v;
10            adj[u].push_back(v);
11            adj[v].push_back(u);
12        }
13        vector<bool> vis(n + 1, false); // vis[i]: 顶点i是否已入队
14        queue<int> q; // q: BFS队列
15        q.push(1); // 起点入队
16        vis[1] = true;

```

```
17     while (!q.empty()) {
18         int u = q.front(); // 取出队首顶点
19         q.pop();
20         cout << u << " "; // 输出访问顺序
21         for (int v : adj[u]) { // 遍历u的所有邻居
22             if (!vis[v]) {
23                 vis[v] = true; // 标记为已访问（入队时即标记，防止重复入
24                     队）
25                 q.push(v);
26             }
27         }
28         cout << "\n";
29         return 0;
30     }
```


29 最短路算法

29.1 Dijkstra 算法（堆优化）

解决问题: 非负权图的单源最短路, 时间复杂度 $O(m \log n)$, 适合稀疏图 ($n, m \leq 2 \times 10^5$)。使用优先队列（小根堆）优化选择最小距离顶点的过程。是竞赛中最常用的最短路算法之一。

题目描述

给定一个带权有向图（边权非负），求从顶点 1 到其他所有顶点的最短距离。

输入示例

```
4 5
1 2 2
1 3 5
2 3 1
2 4 7
3 4 3
```

输出示例

顶点1到各顶点的最短距离：
到1: 0
到2: 2
到3: 3
到4: 6

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const long long INF = 1e18; // 极大值
4      int main() {
5          int n, m; // n: 顶点数, m: 边数
6          cin >> n >> m;
7          vector<vector<pair<int, long long>>> adj(n + 1); // adj[u]: (v, w
           ) 表示 u->v 权w
8          for (int i = 0; i < m; i++) {
9              int u, v; long long w; // u: 起点, v: 终点, w: 边权
```

```

10         cin >> u >> v >> w;
11         adj[u].push_back({v, w});
12     }
13     vector<long long> dist(n + 1, INF); // dist[i]: 起点到i的最短距离
14     dist[1] = 0;
15     // 小根堆, 存储 (距离, 顶点), 按距离升序排列
16     priority_queue<pair<long long, int>, vector<pair<long long, int>
17         >>, greater<>> pq;
18     pq.push({0, 1}); // 起点入堆
19     while (!pq.empty()) {
20         auto [d, u] = pq.top(); // d: 当前记录的起点到u的距离
21         pq.pop();
22         if (d > dist[u]) continue; // 若堆顶记录的距离大于已确定的最短
23             距离, 跳过 (惰性删除)
24         for (auto &p : adj[u]) { // 松弛u的所有出边
25             int v = p.first;
26             long long w = p.second;
27             if (dist[u] + w < dist[v]) { // 若通过u到v更近
28                 dist[v] = dist[u] + w; // 更新距离
29                 pq.push({dist[v], v}); // 新距离入堆
30             }
31         }
32     }
33     cout << "顶点1到各顶点的最短距离:\n";
34     for (int i = 1; i <= n; i++) {
35         cout << "到" << i << ": ";
36         if (dist[i] == INF) cout << "不可达\n";
37         else cout << dist[i] << "\n";
38     }
39     return 0;
40 }

```

29.2 Floyd-Warshall 算法

解决问题：求图中任意两点间的最短距离，可处理负权边（但不能有负环）。时间复杂度 $O(n^3)$ ，适用于 $n \leq 500$ 的稠密图。核心思想为动态规划，枚举中间点进行松弛。

题目描述

给定一个带权有向图，求任意两点间的最短距离。若两点不可达输出 INF 。

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const long long INF = 1e18; // 极大值, 表示不可达
4      int main() {
5          int n, m; // n: 顶点数, m: 边数
6          cin >> n >> m;
7          vector<vector<long long>> dist(n + 1, vector<long long>(n + 1,
8              INF)); // dist[i][j]: i到j的最短距离
9          for (int i = 1; i <= n; i++) dist[i][i] = 0; // 自己到自己的距离为
10             0
11          for (int i = 0; i < m; i++) {
12              int u, v, w; // u: 起点, v: 终点, w: 边权
13              cin >> u >> v >> w;
14              dist[u][v] = min(dist[u][v], (long long)w); // 处理重边, 取最
15                 小值
16          }
17          // Floyd核心: 三重循环, 枚举中间点k, 尝试松弛i->j的路径
18          for (int k = 1; k <= n; k++) {
19              for (int i = 1; i <= n; i++) {
20                  for (int j = 1; j <= n; j++) {
21                      if (dist[i][k] != INF && dist[k][j] != INF) {
22                          // 如果i->k和k->j都可达, 尝试更新i->j
23                          dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
24                      }
25                  }
26              }
27          }
28          cout << "最短距离矩阵:\n";
29          for (int i = 1; i <= n; i++) {
30              for (int j = 1; j <= n; j++) {
31                  if (dist[i][j] == INF) cout << "INF ";
32                  else cout << dist[i][j] << " ";
33              }
34              cout << "\n";
35          }
36          return 0;
37      }
```

30 最小生成树

30.1 Prim 算法

解决问题：求无向连通图的最小生成树。时间复杂度朴素 $O(n^2)$ 适合稠密图，堆优化 $O(m \log n)$ 适合稀疏图。核心思想：从任意顶点开始，每次将距离生成树最近的顶点加入树中，与 Dijkstra 思路相似。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const int INF = 0x3f3f3f3f;
4      int main() {
5          int n, m; // n: 顶点数, m: 边数
6          cin >> n >> m;
7          vector<vector<pair<int, int>>> adj(n + 1); // adj[u]: (v, w)
8          for (int i = 0; i < m; i++) {
9              int u, v, w; // u, v: 边的两端, w: 边权
10             cin >> u >> v >> w;
11             adj[u].push_back({v, w});
12             adj[v].push_back({u, w}); // 无向图添加双向边
13         }
14         vector<int> min_edge(n + 1, INF); // min_edge[i]: 顶点i到当前生成
            树的最小边权
15         vector<bool> in_mst(n + 1, false); // in_mst[i]: 顶点i是否已在生成
            树中
16         min_edge[1] = 0; // 从顶点1开始构建
17         long long total = 0; // total: 最小生成树总边权
18         // Prim核心: 每次选min_edge最小的未加入顶点
19         for (int i = 1; i <= n; i++) {
20             int u = -1; // u: 本轮选出的顶点
21             for (int j = 1; j <= n; j++) {
22                 if (!in_mst[j] && (u == -1 || min_edge[j] < min_edge[u]))
23                     {
24                         u = j;
25                     }
26             }
27             if (min_edge[u] == INF) { // 图不连通
28                 cout << "图不连通\n";
29                 return 0;
30             }
31             in_mst[u] = true; // 将u加入生成树

```

```
31         total += min_edge[u];
32         for (auto &p : adj[u]) { // 用u的邻边更新其他顶点到生成树的距离
33             int v = p.first, w = p.second;
34             if (!in_mst[v] && w < min_edge[v]) {
35                 min_edge[v] = w; // 更新v到生成树的最小边权
36             }
37         }
38     }
39     cout << "最小生成树边权和: " << total << "\n";
40     return 0;
41 }
```

31 拓扑排序

解决问题：给定有向无环图（DAG），输出顶点的线性序列，使得对于每条有向边 $u \rightarrow v$ ， u 在序列中出现在 v 之前。用于任务调度、依赖关系分析、编译器的文件依赖编译顺序等。若存在环则无解。

题目描述

给定一个有向图，输出一个拓扑序。若存在环则输出”存在环”。

输入示例

```
6 6
1 2
1 3
2 4
3 4
4 5
6 3
```

输出示例

拓扑序：1 6 2 3 4 5

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, m; // n: 顶点数, m: 边数
5          cin >> n >> m;
6          vector<vector<int>> adj(n + 1); // adj[i]: 顶点i的出边列表
7          vector<int> indeg(n + 1, 0); // indeg[i]: 顶点i的入度
8          for (int i = 0; i < m; i++) {
9              int u, v; // u: 边的起点, v: 边的终点 (u->v)
10             cin >> u >> v;
11             adj[u].push_back(v);
12             indeg[v]++; // v的入度加1
13         }
14         queue<int> q; // q: 存储当前入度为0的顶点
15         for (int i = 1; i <= n; i++) {
```

```
16         if (indeg[i] == 0) q.push(i); // 入度为0的入队
17     }
18     vector<int> topo; // topo: 存储拓扑序
19     while (!q.empty()) {
20         int u = q.front(); // 取出入度为0的顶点
21         q.pop();
22         topo.push_back(u); // 加入拓扑序
23         for (int v : adj[u]) { // 遍历u的所有出边
24             indeg[v]--; // 删除边u->v, v的入度减1
25             if (indeg[v] == 0) { // 若v入度变为0
26                 q.push(v); // 入队
27             }
28         }
29     }
30     if ((int)topo.size() < n) { // 拓扑序不足n个顶点, 说明有环
31         cout << "存在环\n";
32     } else {
33         cout << "拓扑序: ";
34         for (int x : topo) cout << x << " ";
35         cout << "\n";
36     }
37     return 0;
38 }
```

32 强连通分量与缩点 (Tarjan)

解决问题：求有向图的强连通分量 (SCC)，即分量内任意两点可以互达。时间复杂度 $O(n + m)$ 。可以用于缩点：将每个 SCC 缩为一个点，原图变成 DAG，从而能在上面进行拓扑排序、DP 等。

题目描述

给定一个有向图，求强连通分量的个数，并输出每个顶点所属的 SCC 编号。

输入示例

```
6 8
1 2
2 3
3 1
3 4
4 5
5 6
6 4
2 5
```

输出示例

```
强连通分量数量：3
顶点1属于SCC 2
顶点2属于SCC 2
顶点3属于SCC 2
顶点4属于SCC 1
顶点5属于SCC 1
顶点6属于SCC 1
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      vector<vector<int>> adj; // adj[i]: 顶点i的邻接表
4      vector<int> dfn, low; // dfn[i]: DFS序, low[i]: i能回溯到的最早dfn
5      vector<bool> in_stk; // in_stk[i]: 顶点i是否在栈中
6      stack<int> stk; // stk: Tarjan算法用的栈
```



```

7      int timer = 0; // timer: 当前DFS时间戳
8      int scc_cnt = 0; // scc_cnt: SCC计数器
9      vector<int> belong; // belong[i]: 顶点i所属SCC编号
10     // Tarjan核心: DFS并追溯low值
11     void tarjan(int u) {
12         dfn[u] = low[u] = ++timer; // 初始化dfn和low
13         stk.push(u);
14         in_stk[u] = true;
15         for (int v : adj[u]) {
16             if (!dfn[v]) { // 若v未被访问
17                 tarjan(v);
18                 low[u] = min(low[u], low[v]); // 用子节点的low更新当前节点
19             } else if (in_stk[v]) { // 若v在栈中 (属于当前SCC)
20                 low[u] = min(low[u], dfn[v]); // 用v的dfn更新low
21             }
22         }
23         // 若dfn[u] == low[u], 则u是一个SCC的根
24         if (dfn[u] == low[u]) {
25             scc_cnt++;
26             int v;
27             do {
28                 v = stk.top(); // 弹出栈顶
29                 stk.pop();
30                 in_stk[v] = false;
31                 belong[v] = scc_cnt; // 标记v所属SCC
32             } while (v != u); // 直到弹出u为止
33         }
34     }
35     int main() {
36         int n, m; // n: 顶点数, m: 边数
37         cin >> n >> m;
38         adj.resize(n + 1);
39         dfn.assign(n + 1, 0);
40         low.resize(n + 1);
41         in_stk.assign(n + 1, false);
42         belong.resize(n + 1);
43         for (int i = 0; i < m; i++) {
44             int u, v; // u, v: 有向边 u->v
45             cin >> u >> v;
46             adj[u].push_back(v);

```

```
47     }
48     // 对每个未访问的顶点进行Tarjan
49     for (int i = 1; i <= n; i++) {
50         if (!dfn[i]) tarjan(i);
51     }
52     cout << "强连通分量数量: " << scc_cnt << "\n";
53     for (int i = 1; i <= n; i++) {
54         cout << "顶点" << i << "属于SCC " << belong[i] << "\n";
55     }
56     return 0;
57 }
```

33 最近公共祖先 (LCA 倍增法)

解决问题: 给定一棵有根树, 多次查询两个顶点的最近公共祖先 (LCA)。时间复杂度预处理 $O(n \log n)$, 每次查询 $O(\log n)$ 。用于求树上两点路径 (距离为 $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$)、树上差分等。

题目描述

给定一棵树, 进行 q 次询问, 每次询问两个顶点的 LCA。

输入示例

```
5 3
1 2
1 3
2 4
2 5
4 5
3 4
1 5
```

输出示例

```
LCA(4,5) = 2
LCA(3,4) = 1
LCA(1,5) = 1
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 100005; // 最大顶点数
4      const int LOGN = 20; // log2(MAXN) 向上取整
5      vector<vector<int>>> adj; // adj[i]: 顶点i的邻接表
6      int up[MAXN][LOGN]; // up[i][j]: 顶点i向上跳2^j步的祖先
7      int depth[MAXN]; // depth[i]: 顶点i的深度 (根节点深度为0)
8      // DFS预处理深度和up数组
9      void dfs(int u, int p) {
10         up[u][0] = p; // 直接父节点
11         for (int j = 1; j < LOGN; j++) {
```

```

12         up[u][j] = up[up[u][j-1]][j-1]; // 倍增公式:  $2^j = 2^{j-1} + 2^{j-1}$ 
13     }
14     for (int v : adj[u]) {
15         if (v != p) { // 避免回溯
16             depth[v] = depth[u] + 1;
17             dfs(v, u);
18         }
19     }
20 }
21 // 查询顶点u和v的LCA
22 int lca(int u, int v) {
23     if (depth[u] < depth[v]) swap(u, v); // 保证u深度更大
24     // 将u上提到与v同深度
25     int diff = depth[u] - depth[v];
26     for (int j = 0; j < LOGN; j++) {
27         if (diff & (1 << j)) {
28             u = up[u][j]; // 二进制拆分, 上跳
29         }
30     }
31     if (u == v) return u; // 若v是u的祖先
32     // 一起向上跳
33     for (int j = LOGN - 1; j >= 0; j--) {
34         if (up[u][j] != up[v][j]) { // 只要跳后不同, 就跳
35             u = up[u][j];
36             v = up[v][j];
37         }
38     }
39     return up[u][0]; // 返回父节点即为LCA
40 }
41 int main() {
42     int n, q; // n: 顶点数, q: 询问次数
43     cin >> n >> q;
44     adj.resize(n + 1);
45     for (int i = 1; i < n; i++) { // 树的边数为n-1
46         int u, v; // u, v: 树边的两端
47         cin >> u >> v;
48         adj[u].push_back(v);
49         adj[v].push_back(u);
50     }

```

```
51     depth[1] = 0; // 根节点深度为0
52     dfs(1, 1); // 以1为根进行DFS
53     while (q--) {
54         int u, v; // u, v: 查询的两个顶点
55         cin >> u >> v;
56         cout << "LCA(" << u << ", " << v << ") = " << lca(u, v) << "\n"
57             ;
58     }
59     return 0;
}
```

34 二分图匹配（匈牙利算法）

解决问题：求二分图的最大匹配（边数最多的匹配，使得匹配边没有公共顶点）。时间复杂度 $O(n \times m)$ 。常用于任务分配问题，如工人分配任务、婚配问题、二分图覆盖问题等。

题目描述

给定一个二分图，左侧有 n 个顶点，右侧有 m 个顶点，有 e 条边连接左右两侧。求最大匹配数。

输入示例

```
3 3 5
1 1
1 2
2 2
2 3
3 2
```

输出示例

```
最大匹配数：3
匹配方案：
左1 -> 右1
左2 -> 右3
左3 -> 右2
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      vector<vector<int>> adj; // adj[i]: 左侧顶点i能匹配的右侧顶点列表
4      vector<int> match; // match[v]: 右侧顶点v当前匹配的左侧顶点, 0表示未匹
      配
5      vector<bool> vis; // vis[v]: 本轮增广中右侧顶点v是否被访问过
6      // 匈牙利算法核心: 尝试为左侧顶点u寻找匹配
7      bool dfs(int u) {
8          for (int v : adj[u]) { // 遍历u能匹配的所有右侧顶点
9              if (!vis[v]) { // 若v在本轮未被访问
10                 vis[v] = true;
```

```
11         // 若v未匹配，或v的当前匹配者可以找到新的匹配
12         if (match[v] == 0 || dfs(match[v])) {
13             match[v] = u; // 将v匹配给u
14             return true; // 匹配成功
15         }
16     }
17 }
18 return false; // 匹配失败
19 }
20 int main() {
21     int n, m, e; // n: 左侧顶点数, m: 右侧顶点数, e: 边数
22     cin >> n >> m >> e;
23     adj.resize(n + 1);
24     match.assign(m + 1, 0);
25     for (int i = 0; i < e; i++) {
26         int u, v; // u: 左侧顶点, v: 右侧顶点 (左u 到 右v)
27         cin >> u >> v;
28         adj[u].push_back(v);
29     }
30     int ans = 0; // ans: 最大匹配数
31     // 对每个左侧顶点尝试增广
32     for (int i = 1; i <= n; i++) {
33         vis.assign(m + 1, false); // 每轮清空访问标记
34         if (dfs(i)) ans++; // 若为i找到增广路，匹配数+1
35     }
36     cout << "最大匹配数: " << ans << "\n";
37     cout << "匹配方案:\n";
38     for (int v = 1; v <= m; v++) {
39         if (match[v] != 0) {
40             cout << "左" << match[v] << " -> 右" << v << "\n";
41         }
42     }
43     return 0;
44 }
```

35 网络流（Dinic 最大流算法）

解决问题：求有向图（容量网络）中从源点 S 到汇点 T 的最大流量。每条边有容量限制。时间复杂度 $O(n^2m)$ ，实际很快。常用于流量分配问题、二分图最大匹配（转化成流网络）、最小割问题等。

题目描述

给定一个有向容量网络，求从源点 S 到汇点 T 的最大流。

输入示例

```
4 5 1 4
1 2 10
1 3 5
2 3 15
2 4 10
3 4 10
```

输出示例

最大流：15

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const long long INF = 1e18;
4      struct Edge {
5          int to; // to: 目标顶点
6          long long cap; // cap: 剩余容量
7          int rev; // rev: 反向边在邻接表中的下标
8      };
9      vector<vector<Edge>> graph; // graph[i]: 顶点i的邻接边列表
10     vector<int> level; // level[i]: 顶点i在BFS分层中的层次
11     vector<int> iter; // iter[i]: 顶点i当前弧优化指针
12     // BFS构建分层图
13     void bfs(int s) {
14         level.assign(graph.size(), -1);
15         queue<int> q;
16         level[s] = 0; // 源点层次为0
```



```

17     q.push(s);
18     while (!q.empty()) {
19         int u = q.front(); q.pop();
20         for (auto &e : graph[u]) {
21             if (e.cap > 0 && level[e.to] < 0) { // 有剩余容量且未分层
22                 level[e.to] = level[u] + 1;
23                 q.push(e.to);
24             }
25         }
26     }
27 }
28 // DFS寻找增广路
29 long long dfs(int u, int t, long long f) {
30     if (u == t) return f; // 到达汇点, 返回可增广流量
31     for (int &i = iter[u]; i < (int)graph[u].size(); i++) { // 当前弧
32         // 优化
33         Edge &e = graph[u][i];
34         if (e.cap > 0 && level[u] < level[e.to]) { // 容量>0且层次递增
35             long long d = dfs(e.to, t, min(f, e.cap)); // 递归增广
36             if (d > 0) {
37                 e.cap -= d; // 正向边减容量
38                 graph[e.to][e.rev].cap += d; // 反向边加容量
39                 return d;
40             }
41         }
42     }
43     return 0; // 无法增广
44 }
45 // 添加有向边
46 void add_edge(int from, int to, long long cap) {
47     graph[from].push_back({to, cap, (int)graph[to].size()});
48     graph[to].push_back({from, 0, (int)graph[from].size() - 1}); //
49     // 反向边初始容量为0
50 }
51 int main() {
52     int n, m, s, t; // n: 顶点数, m: 边数, s: 源点, t: 汇点
53     cin >> n >> m >> s >> t;
54     graph.resize(n + 1);
55     level.resize(n + 1);
56     iter.resize(n + 1);

```

```
55     for (int i = 0; i < m; i++) {
56         int u, v; long long w; // u: 起点, v: 终点, w: 容量
57         cin >> u >> v >> w;
58         add_edge(u, v, w);
59     }
60     long long max_flow = 0; // max_flow: 最大流
61     while (true) {
62         bfs(s); // 构建分层图
63         if (level[t] < 0) break; // 若汇点不可达, 结束
64         fill(iter.begin(), iter.end(), 0); // 重置当前弧
65         long long f;
66         while ((f = dfs(s, t, INF)) > 0) { // 不断增广直到无法找到增广
            路
67             max_flow += f;
68         }
69     }
70     cout << "最大流: " << max_flow << "\n";
71     return 0;
72 }
```

36 图论算法总结

第八部分 字符串算法专题

37 字符串基础

37.1 字符串匹配（暴力枚举）

解决问题：给定文本串 s 和模式串 p ，找出 p 在 s 中所有出现的位置。适用于小数据量、对时间要求不高的场景，或作为理解高级匹配算法的前置知识。

题目描述

给定一个文本串 s 和一个模式串 p ，请找出模式串 p 在文本串 s 中所有出现的位置（下标从 0 开始）。

输入示例

```
ababababa
aba
```

输出示例

```
0 2 4 6
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, p; // s: 文本串, p: 模式串
5          cin >> s >> p;
6          int n = s.size(), m = p.size(); // n: 文本串长度, m: 模式串长度
7          // 枚举所有可能的起始位置i, 检查从s[i]开始的长度为m的子串是否与p匹配
8          for (int i = 0; i + m <= n; i++) { // i: 当前匹配的起始位置
9              bool flag = true; // flag: 标记以i开头的子串是否匹配成功
10             for (int j = 0; j < m; j++) { // j: 模式串中当前比较的字符位置
11                 if (s[i + j] != p[j]) { // 当前字符不相等, 匹配失败
12                     flag = false;
13                     break;
14                 }
15             }
16             if (flag) cout << i << " "; // 匹配成功, 输出起始位置
```

```
17     }
18     cout << "\n";
19     return 0;
20 }
```

37.2 字符串反转

解决问题：反转一个字符串或判断一个字符串是否为回文串。适用于文本处理、回文判断等基础操作。

题目描述

给定一个字符串 s ，输出其反转后的字符串，并判断 s 是否为回文串。

输入示例

abccba

输出示例

abccba

Yes

```
1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         string s; // s: 输入的原始字符串
5         cin >> s;
6         string t = s; // t: 存储原始字符串的副本，用于后续比较
7         // 使用双指针法原地反转字符串
8         int n = s.size(); // n: 字符串长度
9         for (int i = 0; i < n / 2; i++) { // i: 左指针，从左向右移动
10             swap(s[i], s[n - 1 - i]); // n-1-i: 右指针，从右向左移动，交换
11                                     // 两端字符
12         }
13         cout << s << "\n"; // 输出反转后的字符串
14         if (t == s) cout << "Yes\n"; // 反转后与原串相等则为回文串
15         else cout << "No\n";
16         return 0;
17     }
```

16

}

37.3 字符串分割

解决问题：按照指定分隔符将字符串分割为多个子串列表。适用于处理 CSV 文件、解析命令参数、提取单词等场景。

题目描述

给定一个字符串 s 和一个分隔符 ch ，将 s 按 ch 分割成多个子串并输出。

输入示例

```
hello,world,string,split
,
```

输出示例

```
hello
world
string
split
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 待分割的原始字符串
5          char ch; // ch: 分隔符字符
6          cin >> s >> ch;
7          vector<string> res; // res: 存储分割后的所有子串
8          string cur = ""; // cur: 当前正在构建的子串
9          for (int i = 0; i < s.size(); i++) { // i: 遍历字符串的索引
10             if (s[i] == ch) { // 遇到分隔符，将当前积累的子串存入结果，并清
11                 // 空cur
12                 if (!cur.empty()) res.push_back(cur);
13                 cur = "";
14             } else {
15                 cur += s[i]; // 普通字符追加到当前子串末尾
16             }
17         }
18     }
```

```
16         }
17         if (!cur.empty()) res.push_back(cur); // 处理最后一个子串（末尾无
           分隔符）
18         for (auto &str : res) cout << str << "\n"; // 输出分割结果
19         return 0;
20     }
```

38 字符串哈希

38.1 单哈希

解决问题：快速进行字符串相等比较，支持 $O(1)$ 时间内判断任意两个子串是否相等。适用于字符串匹配、最长公共前缀、回文判定的预处理等。

题目描述

给定一个字符串 s 和 q 次询问，每次询问给定四个整数 $l1, r1, l2, r2$ ，判断子串 $s[l1..r1]$ 和 $s[l2..r2]$ 是否相等。

输入示例

```
abacaba
3
0 2 4 6
0 1 2 3
1 3 2 4
```

输出示例

```
Yes
No
No
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      typedef unsigned long long ull;
4      const int BASE = 131; // BASE: 哈希基数，通常取大于字符集且为质数的数
5      // 计算子串s[l..r]（闭区间）的哈希值
6      ull getHash(int l, int r, vector<ull>& h, vector<ull>& p) {
7          if (l == 0) return h[r]; // 左端点为0时直接返回前缀哈希
8          return h[r] - h[l - 1] * p[r - l + 1]; // 通过前缀哈希相减得到子串
9          哈希
10     }
11     int main() {
12         string s; // s: 待处理的字符串
13         int q;    // q: 询问次数
```



```

13     cin >> s >> q;
14     int n = s.size(); // n: 字符串长度
15     vector<ull> h(n), p(n); // h[i]: 前缀[0..i]的哈希值, p[i]: BASE的i
        次幂
16     h[0] = s[0]; // 初始化第一个字符的哈希值
17     p[0] = 1; // BASE^0 = 1
18     for (int i = 1; i < n; i++) {
19         h[i] = h[i - 1] * BASE + s[i]; // 递推计算前缀哈希
20         p[i] = p[i - 1] * BASE; // 递推计算BASE的幂次
21     }
22     while (q--) {
23         int l1, r1, l2, r2; // 询问的两个子串区间 [l1,r1] 和 [l2,r2]
24         cin >> l1 >> r1 >> l2 >> r2;
25         if (getHash(l1, r1, h, p) == getHash(l2, r2, h, p))
26             cout << "Yes\n";
27         else
28             cout << "No\n";
29     }
30     return 0;
31 }

```

38.2 双哈希

解决问题：在单哈希基础上使用两个不同的基数和模数计算哈希值，有效降低哈希冲突概率。适用于对正确性要求极高、无法承受哈希碰撞的场景。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     typedef long long ll;
4     const ll BASE1 = 131, MOD1 = 1e9 + 7; // 第一组哈希的参数：基数B1, 模
        数M1
5     const ll BASE2 = 13331, MOD2 = 998244353; // 第二组哈希的参数：基数B2
        , 模数M2
6     struct HashPair {
7         ll h1, h2; // h1: 第一组哈希值, h2: 第二组哈希值
8         bool operator==(const HashPair& o) const {
9             return h1 == o.h1 && h2 == o.h2; // 两组哈希值都相等才判定为真
10        }
11    };
12    int main() {
13        string s; // s: 待处理的字符串

```

```

14     int q;    // q: 询问次数
15     cin >> s >> q;
16     int n = s.size(); // n: 字符串长度
17     vector<HashPair> h(n); // h[i]: 前缀[0..i]的双哈希值
18     vector<ll> p1(n), p2(n); // p1[i]:  $B1^i \bmod M1$ , p2[i]:  $B2^i \bmod$ 
19                                $M2$ 
19     h[0] = {s[0] % MOD1, s[0] % MOD2};
20     p1[0] = p2[0] = 1;
21     for (int i = 1; i < n; i++) {
22         auto& cur = h[i], &pre = h[i-1];
23         cur.h1 = (pre.h1 * BASE1 + s[i]) % MOD1; // 递推计算第一组哈希
24         cur.h2 = (pre.h2 * BASE2 + s[i]) % MOD2; // 递推计算第二组哈希
25         p1[i] = (p1[i-1] * BASE1) % MOD1; // 递推计算基数的幂次
26         p2[i] = (p2[i-1] * BASE2) % MOD2;
27     }
28     // 查询闭区间[l,r]子串的双哈希值
29     auto getHash = [&](int l, int r) -> HashPair {
30         if (l == 0) return h[r];
31         HashPair res;
32         res.h1 = (h[r].h1 - h[l-1].h1 * p1[r-l+1] % MOD1 + MOD1) %
33             MOD1;
34         res.h2 = (h[r].h2 - h[l-1].h2 * p2[r-l+1] % MOD2 + MOD2) %
35             MOD2;
36         return res;
37     };
38     while (q--) {
39         int l1, r1, l2, r2; // 询问的两个子串区间
40         cin >> l1 >> r1 >> l2 >> r2;
41         if (getHash(l1, r1) == getHash(l2, r2))
42             cout << "Yes\n";
43         else
44             cout << "No\n";
45     }
46     return 0;
47 }

```

39 KMP 算法

39.1 字符串匹配

解决问题：在线性时间复杂度 $O(n+m)$ 内找出模式串在文本串中的所有出现位置。适用于需要高效匹配的任何场景，如文本编辑器中的查找功能、日志分析等。

题目描述

给定文本串 s 和模式串 p ，输出 p 在 s 中每次出现的起始下标（下标从 0 开始）。

输入示例

```
ababcabababab
abab
```

输出示例

```
0 7
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, p; // s: 文本串, p: 模式串
5          cin >> s >> p;
6          int n = s.size(), m = p.size(); // n: 文本串长度, m: 模式串长度
7          vector<int> nxt(m); // nxt[i]: 模式串前缀p[0..i]的最长相等前后缀长
           度（部分匹配表）
8          // 构建next数组: nxt[i]表示p[0..i]中, 既是前缀又是后缀的最长子串长
           度
9          nxt[0] = 0; // 长度为1的前缀, 没有非平凡的前后缀, 填0
10         for (int i = 1, j = 0; i < m; i++) { // i: 当前正在计算next的后缀末
           尾下标, j: 当前匹配的前缀末尾下标
11             while (j > 0 && p[i] != p[j]) j = nxt[j - 1]; // 失配时回退j,
           利用已求的next信息
12             if (p[i] == p[j]) j++; // 当前字符匹配, 前缀长度+1
13             nxt[i] = j; // 记录next值
14         }
15         // 利用next数组进行匹配
```

```

16         for (int i = 0, j = 0; i < n; i++) { // i: 文本串当前扫描位置, j:
           模式串当前匹配位置
17             while (j > 0 && s[i] != p[j]) j = nxt[j - 1]; // 失配时, 根据
               nxt数组回退模式串指针
18             if (s[i] == p[j]) j++; // 当前位置匹配成功, j后移
19             if (j == m) { // 完全匹配模式串
20                 cout << i - m + 1 << " "; // 输出匹配起始位置
21                 j = nxt[j - 1]; // 继续寻找下一次匹配
22             }
23         }
24         cout << "\n";
25         return 0;
26     }

```

39.2 最小循环节

解决问题: 利用 KMP 的 `nxt` 数组找出字符串的最小循环节。适用于字符串周期分析、拼接字符串最小次数等场景。

题目描述

给定一个字符串 s , 判断其是否可以由一个较短的子串重复多次构成。若是, 输出该最短重复子串; 否则输出原串本身。

输入示例

abccabccabcc

输出示例

abc

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         string s; // s: 输入字符串
5         cin >> s;
6         int n = s.size(); // n: 字符串长度
7         vector<int> nxt(n);

```

```

8      nxt[0] = 0;
9      for (int i = 1, j = 0; i < n; i++) {
10         while (j > 0 && s[i] != s[j]) j = nxt[j - 1];
11         if (s[i] == s[j]) j++;
12         nxt[i] = j;
13     }
14     int len = n - nxt[n - 1]; // len: 最小循环节的长度
15     // 如果len能整除n, 则字符串可以由循环节重复构造
16     if (n % len == 0) cout << s.substr(0, len) << "\n";
17     else cout << s << "\n"; // 不能整除, 说明没有完整循环节, 输出原串
18     return 0;
19 }

```

39.3 统计前缀出现次数

解决问题：统计模式串的每个前缀在文本串中出现的次数。适用于统计单词前缀频次等场景。

题目描述

给定文本串 s 和模式串 p ，输出 p 的每个前缀在 s 中出现的次数。

输入示例

```

ababa
aba

```

输出示例

```

前缀a: 3
前缀ab: 2
前缀aba: 2

```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s, p; // s: 文本串, p: 模式串
5          cin >> s >> p;
6          int n = s.size(), m = p.size();

```

```
7     vector<int> nxt(m);
8     nxt[0] = 0;
9     for (int i = 1, j = 0; i < m; i++) {
10         while (j > 0 && p[i] != p[j]) j = nxt[j - 1];
11         if (p[i] == p[j]) j++;
12         nxt[i] = j;
13     }
14     vector<int> cnt(m + 1, 0); // cnt[i]: 模式串长度为i的前缀在文本串
    中出现的次数
15     for (int i = 0, j = 0; i < n; i++) { // 在文本串中匹配模式串
16         while (j > 0 && s[i] != p[j]) j = nxt[j - 1];
17         if (s[i] == p[j]) j++;
18         cnt[j]++; // 当前匹配到的前缀长度j出现一次
19     }
20     // 从长到短累加计数: 若长前缀出现过, 其短前缀也一定出现
21     for (int i = m; i >= 1; i--) {
22         if (nxt[i - 1] > 0) cnt[nxt[i - 1]] += cnt[i];
23     }
24     for (int i = 1; i <= m; i++) {
25         cout << "前缀" << p.substr(0, i) << ": " << cnt[i] << "\n";
26     }
27     return 0;
28 }
```

40 Trie 树（字典树）

40.1 字符串插入与查询

解决问题：高效存储和检索大量字符串集合，支持快速查询某个字符串是否出现及其出现次数。适用于搜索引擎关键词补全、IP 路由查找、词频统计等场景。

题目描述

给定 n 个单词进行建树，随后有 q 次询问，每次询问一个单词是否在集合中出现过以及出现次数。

输入示例

```
5
apple
app
apple
banana
orange
3
apple
app
grape
```

输出示例

```
apple found, count: 2
app found, count: 1
grape not found
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 100005; // MAXN: 最大节点数量（根据字符串总长度估算）
4      int trie[MAXN][26]; // trie[u][c]: 节点u通过字符c（映射为0~25）指向的
      子节点编号
5      int cnt[MAXN]; // cnt[u]: 以节点u结尾的字符串出现次数
6      int tot = 1; // tot: 当前已分配的节点总数，根节点为1
7      // 向字典树中插入字符串s
```

```
8     void insert(const string& s) {
9         int u = 1; // u: 当前所在节点编号, 从根节点开始
10        for (int i = 0; i < s.size(); i++) { // i: 遍历字符串中字符的位置
11            int c = s[i] - 'a'; // c: 当前字符映射为0~25的整数
12            if (!trie[u][c]) trie[u][c] = ++tot; // 若边不存在, 创建新节点
13            u = trie[u][c]; // 沿边移动到下一节点
14        }
15        cnt[u]++; // 在字符串结尾节点增加计数
16    }
17    // 查询字符串s的出现次数, 返回次数 (0表示未出现)
18    int query(const string& s) {
19        int u = 1;
20        for (int i = 0; i < s.size(); i++) {
21            int c = s[i] - 'a';
22            if (!trie[u][c]) return 0; // 某字符边不存在, 直接返回未找到
23            u = trie[u][c];
24        }
25        return cnt[u]; // 返回结尾节点的计数值
26    }
27    int main() {
28        int n, q; // n: 插入单词数量, q: 查询单词数量
29        cin >> n;
30        string word; // word: 临时存储输入的单词
31        for (int i = 0; i < n; i++) {
32            cin >> word;
33            insert(word);
34        }
35        cin >> q;
36        for (int i = 0; i < q; i++) {
37            cin >> word;
38            int res = query(word); // res: 查询结果, 即该单词出现次数
39            if (res > 0) cout << word << " found, count: " << res << "\n";
40            else cout << word << " not found\n";
41        }
42        return 0;
43    }
```


40.2 最大异或和（Trie 求异或极值）

解决问题：在一组整数中快速找出与给定值异或结果最大的数。适用于网络编码、密码学、数据压缩等需要异或优化的场景。

题目描述

给定 n 个非负整数，有 q 次询问，每次询问一个整数 x ，求 x 与数组中任意元素异或的最大值。

输入示例

```
5
3 7 2 9 5
3
4 6 8
```

输出示例

```
13
14
15
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 100005 * 32; // 每个数最多32位，节点数约为数的个数*32
4      int trie[MAXN][2], tot = 1; // trie[u][0/1]: 节点u的0/1子节点编号，
      // tot为当前节点总数
5      // 向二进制Trie中插入数x
6      void insert(int x) {
7          int u = 1; // u: 当前节点编号
8          for (int i = 30; i >= 0; i--) { // i: 当前处理的二进制位，从高位向
              // 低位
9              int b = (x >> i) & 1; // b: x在第i位的二进制值
10             if (!trie[u][b]) trie[u][b] = ++tot; // 创建新节点
11             u = trie[u][b]; // 移动到子节点
12         }
13     }
14     // 查询与x异或的最大值
15     int queryMaxXor(int x) {
```

```
16     int u = 1, res = 0; // res: 能获得的最大异或值
17     for (int i = 30; i >= 0; i--) {
18         int b = (x >> i) & 1;
19         if (trie[u][b ^ 1]) { // 优先选相反位, 使异或结果该位为1
20             res |= (1 << i); // 累加该位的贡献
21             u = trie[u][b ^ 1];
22         } else {
23             u = trie[u][b]; // 无奈只能选相同位
24         }
25     }
26     return res;
27 }
28 int main() {
29     int n, q; // n: 数组大小, q: 询问次数
30     cin >> n;
31     for (int i = 0; i < n; i++) {
32         int x; // x: 输入的元素
33         cin >> x;
34         insert(x);
35     }
36     cin >> q;
37     for (int i = 0; i < q; i++) {
38         int x; // x: 询问的值
39         cin >> x;
40         cout << queryMaxXor(x) << "\n";
41     }
42     return 0;
43 }
```

41 Manacher 算法

41.1 最长回文子串

解决问题：在线性时间 $O(n)$ 内求出字符串中最长的回文子串。适用于文本处理、DNA 序列分析、寻找对称结构等场景。

题目描述

给定一个字符串 s ，找出其最长的回文子串。如果有多个，返回任意一个。

输入示例

babad

输出示例

bab

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 原始输入字符串
5          cin >> s;
6          // 预处理：在原字符串每个字符间插入特殊字符'#'，便于统一奇偶长度回文
7          string t = "#"; // t: 处理后的字符串，以'#'开头
8          for (int i = 0; i < s.size(); i++) {
9              t += s[i];
10             t += '#'; // 字符间和末尾加'#'
11         }
12         int n = t.size(); // n: 处理后字符串长度
13         vector<int> p(n); // p[i]: 以t[i]为中心的最长回文半径（包含中心自身）
14         int c = 0, r = 0; // c: 当前最右回文串的中心，r: 该回文串的右边界
15         for (int i = 0; i < n; i++) {
16             // 利用对称性初始化p[i]: i关于c的对称点为2*c-i, 取min避免越界
17             if (i < r) p[i] = min(r - i, p[2 * c - i]);
18             else p[i] = 1; // 至少包含自身
19             // 以i为中心向两侧扩展
```

```
20         while (i - p[i] >= 0 && i + p[i] < n && t[i - p[i]] == t[i + p  
21             [i]]) {  
22             p[i]++; // 匹配成功, 半径加1  
23         }  
24         // 更新最右回文串的中心和右边界  
25         if (i + p[i] > r) {  
26             c = i;  
27             r = i + p[i];  
28         }  
29     }  
30     int center = 0, maxR = 0; // center: 最长回文子串的中心索引, maxR:  
31     // 最长半径  
32     for (int i = 0; i < n; i++) {  
33         if (p[i] > maxR) {  
34             maxR = p[i];  
35             center = i;  
36         }  
37     }  
38     // 从处理后字符串还原原始字符串中的回文子串  
39     int start = (center - maxR + 1) / 2; // start: 原始字符串中的起始  
40     // 索引  
41     int len = maxR - 1; // len: 原始回文子串长度  
42     cout << s.substr(start, len) << "\n";  
43     return 0;  
44 }
```

41.2 统计回文子串个数

解决问题：在线性时间内统计字符串中所有回文子串的总数。适用于文本分析、模式识别等场景。

题目描述

给定一个字符串 s ，统计其中所有回文子串的数量（不同位置的相同字符串计为不同回文子串）。

输入示例

abc

输出示例

3

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入字符串
5          cin >> s;
6          string t = "#";
7          for (int i = 0; i < s.size(); i++) {
8              t += s[i];
9              t += '#';
10         }
11         int n = t.size();
12         vector<int> p(n);
13         int c = 0, r = 0;
14         long long ans = 0; // ans: 回文子串总数
15         for (int i = 0; i < n; i++) {
16             if (i < r) p[i] = min(r - i, p[2 * c - i]);
17             else p[i] = 1;
18             while (i - p[i] >= 0 && i + p[i] < n && t[i - p[i]] == t[i + p[i]]) {
19                 p[i]++;
20             }
21             if (i + p[i] > r) {
22                 c = i;
23                 r = i + p[i];
24             }
25             ans += p[i] / 2; // 以该位置为中心的回文子串数等于半径的一半（向下取整）
26         }
27         cout << ans << "\n";
28         return 0;
29     }
```

42 扩展 KMP (Z 算法)

42.1 计算 Z 数组

解决问题：在线性时间内计算字符串每个后缀与原串前缀的最长公共前缀（Z 数组）。适用于字符串匹配、周期分析、Border 计算等场景。

题目描述

给定一个字符串 s ，输出其 Z 数组，其中 $z[i]$ 表示后缀 $s[i..]$ 与 s 的最长公共前缀长度。

输入示例

aabxaabx

输出示例

8 1 0 0 4 1 0 0

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          string s; // s: 输入字符串
5          cin >> s;
6          int n = s.size(); // n: 字符串长度
7          vector<int> z(n); // z[i]: 以i起始的后缀与原串前缀的最长匹配长度
8          z[0] = n; // 整个字符串与自身完全匹配
9          // 维护一个区间 [l, r], 表示当前已知的最靠右的匹配段
10         for (int i = 1, l = 0, r = 0; i < n; i++) { // l: 最右匹配段左端点
11             // r: 最右匹配段右端点
12             if (i <= r) z[i] = min(r - i + 1, z[i - l]); // 利用对称性初始化
13             while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++; // 暴力扩展
14             if (i + z[i] - 1 > r) { // 更新最右匹配段
15                 l = i;
16                 r = i + z[i] - 1;
17             }
18         }
```

```

18         for (int i = 0; i < n; i++) {
19             cout << z[i] << " \n"[i == n - 1];
20         }
21         return 0;
22     }

```

42.2 字符串匹配 (Z 算法)

解决问题: 利用 Z 算法在线性时间内找出模式串在文本串中的所有出现位置。与 KMP 同为线性匹配算法, 实现方式不同。

题目描述

给定文本串 s 和模式串 p , 输出 p 在 s 中每次出现的起始下标。

输入示例

```

ababa
aba

```

输出示例

```

0 2

```

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         string s, p; // s: 文本串, p: 模式串
5         cin >> s >> p;
6         // 将模式串拼接在文本串前面, 用特殊字符分隔
7         string combined = p + "#" + s; // combined: 拼接后的字符串
8         int m = p.size(), n = combined.size(); // m: 模式串长度, n: 拼接串
          长度
9         vector<int> z(n);
10        z[0] = n;
11        for (int i = 1, l = 0, r = 0; i < n; i++) {
12            if (i <= r) z[i] = min(r - i + 1, z[i - l]);
13            while (i + z[i] < n && combined[z[i]] == combined[i + z[i]]) z
                [i]++;

```

```
14         if (i + z[i] - 1 > r) {
15             l = i;
16             r = i + z[i] - 1;
17         }
18     }
19     // 扫描拼接串中模式串之后的部分, z[i] 等于 m 说明匹配成功
20     for (int i = m + 1; i < n; i++) { // i: 文本串部分对应的索引
21         if (z[i] == m) cout << i - m - 1 << " "; // 输出在文本串中的起
22             始位置
23     }
24     cout << "\n";
25     return 0;
}
```


43 AC 自动机

43.1 多模式串匹配

解决问题：给定一个文本串和一组模式串，一次性找出所有模式串在文本串中的出现位置。适用于敏感词过滤、搜索引擎关键词高亮、防病毒特征匹配等场景。

题目描述

给定一个文本串 s 和 k 个模式串，输出每个模式串在 s 中出现的次数。

输入示例

```
ahishers
3
he she his
```

输出示例

```
he: 1
she: 1
his: 1
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 500005; // MAXN: Trie最大节点数
4      int trie[MAXN][26]; // trie[u][c]: 节点u经字符c指向的子节点
5      int fail[MAXN]; // fail[u]: 节点u的失配指针
6      int cnt[MAXN]; // cnt[u]: 以节点u结尾的模式串的编号（或计数）
7      int tot = 1; // tot: 当前节点总数
8      // 向Trie中插入模式串，返回该模式串对应的终止节点编号
9      int insert(const string& s, int id) {
10         int u = 1; // u: 从根节点开始
11         for (int i = 0; i < s.size(); i++) {
12             int c = s[i] - 'a';
13             if (!trie[u][c]) trie[u][c] = ++tot;
14             u = trie[u][c];
15         }
16         if (!cnt[u]) cnt[u] = id; // 记录该终止节点对应的模式串编号
17         return u;
```

```

18     }
19     // 构建AC自动机的失配指针
20     void build() {
21         queue<int> q; // q: BFS队列
22         for (int c = 0; c < 26; c++) { // 初始化根节点的子节点fail指向根
23             int v = trie[1][c];
24             if (v) {
25                 fail[v] = 1;
26                 q.push(v);
27             }
28         }
29         while (!q.empty()) {
30             int u = q.front(); q.pop();
31             for (int c = 0; c < 26; c++) {
32                 int v = trie[u][c];
33                 if (v) {
34                     fail[v] = trie[fail[u]][c]; // 失配指针沿父节点fail的对
35                                                 // 应字符走
36                     q.push(v);
37                 } else {
38                     trie[u][c] = trie[fail[u]][c]; // 空子节点直接指向父节点
39                                                 // fail的对应子节点
40                 }
41             }
42         }
43     }
44     int main() {
45         string s; // s: 文本串
46         int k; // k: 模式串数量
47         cin >> s >> k;
48         vector<string> patterns(k); // patterns[i]: 第i个模式串
49         vector<int> endNode(k); // endNode[i]: 第i个模式串在Trie中的终止节
50                                 // 点
51         for (int i = 0; i < k; i++) {
52             cin >> patterns[i];
53             endNode[i] = insert(patterns[i], i + 1); // id从1开始
54         }
55         build();
56         vector<int> ans(k + 1, 0); // ans[id]: 编号为id的模式串在文本串中
57                                 // 出现的次数

```

```

54     int u = 1; // u: 当前匹配状态对应的Trie节点
55     for (int i = 0; i < s.size(); i++) {
56         int c = s[i] - 'a';
57         u = trie[u][c]; // 转移到下一状态（失配已预处理）
58         // 沿fail链统计所有匹配到的模式串
59         for (int temp = u; temp > 1; temp = fail[temp]) {
60             if (cnt[temp]) ans[cnt[temp]]++;
61         }
62     }
63     for (int i = 0; i < k; i++) {
64         cout << patterns[i] << ": " << ans[i + 1] << "\n";
65     }
66     return 0;
67 }

```

43.2 拓扑优化统计（避免重复跳 fail）

解决问题：在 AC 自动机中高效统计每个模式串的出现次数，避免每次匹配都沿 fail 链暴力跳转。当模式串很多且文本较长时，可将复杂度控制在 $O(n + \text{节点数})$ 。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     const int MAXN = 500005;
4     int trie[MAXN][26], fail[MAXN];
5     int flag[MAXN]; // flag[u]: 节点u对应的模式串编号
6     int indeg[MAXN]; // indeg[u]: 节点u在fail树中的入度，用于拓扑排序
7     int sum[MAXN]; // sum[u]: 匹配过程中经过节点u的次数
8     int tot = 1;
9     int insert(const string& s, int id) {
10         int u = 1;
11         for (int i = 0; i < s.size(); i++) {
12             int c = s[i] - 'a';
13             if (!trie[u][c]) trie[u][c] = ++tot;
14             u = trie[u][c];
15         }
16         if (!flag[u]) flag[u] = id;
17         return u;
18     }
19     void build() {
20         queue<int> q;
21         for (int c = 0; c < 26; c++) {

```

```
22     int v = trie[1][c];
23     if (v) {
24         fail[v] = 1;
25         indeg[1]++; // 根节点入度增加
26         q.push(v);
27     }
28 }
29 while (!q.empty()) {
30     int u = q.front(); q.pop();
31     for (int c = 0; c < 26; c++) {
32         int v = trie[u][c];
33         if (v) {
34             fail[v] = trie[fail[u]][c];
35             indeg[fail[v]]++; // fail[v]的入度增加, 建立fail树的边
36             q.push(v);
37         } else {
38             trie[u][c] = trie[fail[u]][c];
39         }
40     }
41 }
42 }
43 int main() {
44     string s; // s: 文本串
45     int k;    // k: 模式串数量
46     cin >> s >> k;
47     vector<string> patterns(k);
48     vector<int> endNode(k);
49     for (int i = 0; i < k; i++) {
50         cin >> patterns[i];
51         endNode[i] = insert(patterns[i], i + 1);
52     }
53     build();
54     int u = 1;
55     for (int i = 0; i < s.size(); i++) {
56         int c = s[i] - 'a';
57         u = trie[u][c];
58         sum[u]++; // 只记录经过节点, 不跳fail
59     }
60     // 拓扑排序: 按照fail树的拓扑序从底向上累加sum值
61     queue<int> q;
```

```
62     for (int i = 1; i <= tot; i++) {
63         if (!indeg[i]) q.push(i); // 入度为0的节点入队
64     }
65     while (!q.empty()) {
66         int u = q.front(); q.pop();
67         int f = fail[u];
68         if (f) {
69             sum[f] += sum[u]; // 将u的计数传递给其fail指向的节点
70             if (--indeg[f] == 0) q.push(f);
71         }
72     }
73     vector<int> ans(k + 1, 0);
74     for (int i = 0; i < k; i++) {
75         ans[i + 1] = sum[endNode[i]]; // 终止节点的最终累计值即为出现次
76         // 数
77         cout << patterns[i] << ": " << ans[i + 1] << "\n";
78     }
79     return 0;
}
```

44 后缀自动机 (SAM)

44.1 构建后缀自动机

解决问题：在线性时间内构建一个能接受字符串所有后缀的确定有限状态自动机。可用于高效处理子串查询、本质不同子串个数、最长公共子串等问题。

题目描述

给定一个字符串 s ，构建其后缀自动机，并输出所有本质不同的子串个数。

输入示例

ababa

输出示例

9

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 200005; // MAXN: 状态数上界 (字符串长度的两倍)
4      struct State {
5          int len;    // len: 该状态代表的最长子串长度
6          int fail;   // fail: 后缀链接 (指向当前状态代表子串的最长不含后缀)
7          int ch[26]; // ch[c]: 转移边表
8      } st[MAXN]; // st: 后缀自动机状态数组
9      int tot = 1, last = 1; // tot: 状态总数, last: 当前整个字符串对应的状态
10     // 向SAM中添加一个字符c
11     void extend(int c) {
12         int cur = ++tot; // cur: 新建状态节点
13         st[cur].len = st[last].len + 1;
14         int p = last; // p: 从last开始沿fail链向上寻找转移
15         while (p && !st[p].ch[c]) { // 所有不含c转移的状态都指向cur
16             st[p].ch[c] = cur;
17             p = st[p].fail;
18         }
19         if (!p) {
20             st[cur].fail = 1; // 回到初始状态
```

```

21     } else {
22         int q = st[p].ch[c]; // q: p的c转移目标
23         if (st[p].len + 1 == st[q].len) {
24             st[cur].fail = q; // q就是从p直接扩展而来
25         } else { // 需要分裂q状态
26             int clone = ++tot; // clone: q的克隆节点
27             st[clone] = st[q]; // 复制q的所有信息
28             st[clone].len = st[p].len + 1; // 修改克隆节点的len
29             // 将p及其祖先中指向q的转移重定向到clone
30             while (p && st[p].ch[c] == q) {
31                 st[p].ch[c] = clone;
32                 p = st[p].fail;
33             }
34             st[q].fail = st[cur].fail = clone; // q和cur的fail均指向
35                 clone
36         }
37         last = cur;
38     }
39     int main() {
40         string s; // s: 输入字符串
41         cin >> s;
42         for (int i = 0; i < s.size(); i++) {
43             extend(s[i] - 'a'); // 逐个字符构建后缀自动机
44         }
45         // 所有本质不同的子串个数 = 所有状态代表的子串长度范围之和
46         long long ans = 0; // ans: 本质不同子串数量
47         for (int i = 2; i <= tot; i++) { // 从编号2开始, 1为初始状态
48             ans += st[i].len - st[st[i].fail].len; // 该状态贡献的子串数
49         }
50         cout << ans << "\n";
51         return 0;
52     }

```

44.2 最长公共子串 (多字符串, SAM)

解决问题：利用后缀自动机求多个字符串的最长公共子串。适用于多序列比对、多文档查重等场景。

题目描述

给定 k 个字符串，求出所有字符串的最长公共子串的长度。

输入示例

```
3
abcde
abdec
cdeab
```

输出示例

```
3
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MAXN = 200005;
4      struct State {
5          int len, fail;
6          int ch[26];
7      } st[MAXN];
8      int tot = 1, last = 1;
9      void extend(int c) {
10         int cur = ++tot;
11         st[cur].len = st[last].len + 1;
12         int p = last;
13         while (p && !st[p].ch[c]) {
14             st[p].ch[c] = cur;
15             p = st[p].fail;
16         }
17         if (!p) {
18             st[cur].fail = 1;
19         } else {
20             int q = st[p].ch[c];
21             if (st[p].len + 1 == st[q].len) {
22                 st[cur].fail = q;
23             } else {
24                 int clone = ++tot;
```



```

25         st[clone] = st[q];
26         st[clone].len = st[p].len + 1;
27         while (p && st[p].ch[c] == q) {
28             st[p].ch[c] = clone;
29             p = st[p].fail;
30         }
31         st[q].fail = st[cur].fail = clone;
32     }
33 }
34 last = cur;
35 }
36 int main() {
37     int k; // k: 字符串数量
38     cin >> k;
39     string first; // first: 第一个字符串, 用于构建SAM
40     cin >> first;
41     for (int i = 0; i < first.size(); i++) extend(first[i] - 'a');
42     vector<int> maxMatch(tot + 1); // maxMatch[i]: 所有字符串在状态i处
43     // 的最长匹配长度 (取min)
44     vector<int> curMatch(tot + 1); // curMatch[i]: 当前处理的字符串在
45     // 状态i的匹配长度
46     fill(maxMatch.begin(), maxMatch.end(), INT_MAX);
47     // 计算状态的拓扑序 (按len从大到小)
48     vector<int> order(tot + 1);
49     iota(order.begin(), order.end(), 0);
50     sort(order.begin() + 1, order.end(), [&](int a, int b) {
51         return st[a].len > st[b].len;
52     });
53     for (int t = 1; t < k; t++) {
54         string curStr; // curStr: 当前正在处理的字符串
55         cin >> curStr;
56         fill(curMatch.begin(), curMatch.end(), 0);
57         int u = 1, len = 0; // u: 当前SAM状态, len: 当前匹配长度
58         for (int i = 0; i < curStr.size(); i++) {
59             int c = curStr[i] - 'a';
60             while (u != 1 && !st[u].ch[c]) {
61                 u = st[u].fail; // 无法转移, 沿fail回退
62                 len = st[u].len;
63             }
64             if (st[u].ch[c]) {

```

```
63         u = st[u].ch[c];
64         len++;
65     }
66     curMatch[u] = max(curMatch[u], len); // 更新当前状态的最长
        匹配
67 }
68 // 按拓扑序将匹配信息传递给fail节点
69 for (int i = 1; i <= tot; i++) {
70     int u = order[i];
71     if (st[u].fail) curMatch[st[u].fail] = max(curMatch[st[u].
        fail], curMatch[u]);
72 }
73 // 更新全局min值
74 for (int i = 1; i <= tot; i++) {
75     maxMatch[i] = min(maxMatch[i], curMatch[i]);
76 }
77 }
78 int ans = 0; // ans: 最长公共子串长度
79 for (int i = 1; i <= tot; i++) {
80     ans = max(ans, maxMatch[i]);
81 }
82 cout << ans << "\n";
83 return 0;
84 }
```

45 最小表示法

45.1 循环同构最小表示

解决问题：在线性时间内找出字符串循环同构中的字典序最小者。适用于循环字符串比较、环形结构编码等场景。

题目描述

给定一个字符串 s ，求其循环同构中字典序最小的字符串。

输入示例

bcad

输出示例

adbc

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      // 返回字符串s的循环同构最小表示的起始下标
4      int minRepresentation(const string& s) {
5          int n = s.size(); // n: 字符串长度
6          int i = 0, j = 1, k = 0; // i: 候选位置1, j: 候选位置2, k: 当前比较的偏移量
7          while (i < n && j < n && k < n) {
8              char a = s[(i + k) % n]; // a: 从i往后偏移k位的字符
9              char b = s[(j + k) % n]; // b: 从j往后偏移k位的字符
10             if (a == b) {
11                 k++; // 当前字符相等，继续比较下一字符
12             } else {
13                 if (a > b) i = i + k + 1; // 以i开头的表示更大，i跳过k+1个位置
14                 else j = j + k + 1; // 以j开头的表示更大，j跳过k+1个位置
15                 if (i == j) j++; // 保证i和j不同
16                 k = 0; // 重置偏移量
17             }
18         }
19         return min(i, j); // 返回较小候选者
```

```
20     }
21     int main() {
22         string s; // s: 输入字符串
23         cin >> s;
24         int start = minRepresentation(s); // start: 最小表示的起始下标
25         string ans; // ans: 最小表示字符串
26         for (int i = 0; i < s.size(); i++) {
27             ans += s[(start + i) % s.size()]; // 从start开始循环拼接
28         }
29         cout << ans << "\n";
30         return 0;
31     }
```

46 字符串算法总结

字符串算法选择指南

1. 单模式匹配：优先选用 **KMP** 或 **Z** 算法
2. 多模式匹配：选用 **AC** 自动机
3. 回文相关问题：选用 **Manacher** 或 回文自动机
4. 子串种类/统计问题：选用 后缀数组或 后缀自动机
5. 子串快速比较/判重：选用 字符串哈希
6. 字典树/异或相关：选用 **Trie**
7. 循环同构：选用 最小表示法

第九部分 计算几何算法专题

47 基础运算

47.1 点积与叉积

解决问题：点积用于判断向量夹角（锐角/钝角/直角），叉积用于判断方向（顺时针/逆时针/共线）和计算面积。适用于所有几何问题的底层计算。

核心概念

给定向量 $\vec{a} = (x_1, y_1)$, $\vec{b} = (x_2, y_2)$:

- 点积: $\vec{a} \cdot \vec{b} = x_1x_2 + y_1y_2 = |\vec{a}||\vec{b}| \cos \theta$
- 叉积: $\vec{a} \times \vec{b} = x_1y_2 - x_2y_1 = |\vec{a}||\vec{b}| \sin \theta$

叉积的正负：正表示 $\vec{b}$ 在 $\vec{a}$ 的逆时针方向，负表示顺时针方向，0 表示共线。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      // 定义点结构体
4      struct Point {
5          double x, y; // x: 点的横坐标, y: 点的纵坐标
6          Point(double x = 0, double y = 0) : x(x), y(y) {}
7      };
8      // 定义向量为点的别名
9      typedef Point Vector;
10     // 向量加法
11     Vector operator+(Vector a, Vector b) {
12         return Vector(a.x + b.x, a.y + b.y); // 对应坐标相加
13     }
14     // 向量减法
15     Vector operator-(Vector a, Vector b) {
16         return Vector(a.x - b.x, a.y - b.y); // 对应坐标相减
17     }
18     // 向量数乘
19     Vector operator*(Vector a, double k) {
20         return Vector(a.x * k, a.y * k); // 每个坐标乘以标量k
21     }
22     // 点积:  $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos$ 
23     double dot(Vector a, Vector b) {

```

```

24         return a.x * b.x + a.y * b.y; // 对应坐标乘积之和
25     }
26     // 叉积:  $a \times b = |a| |b| \sin$ , 正值表示 $b$ 在 $a$ 的逆时针方向
27     double cross(Vector a, Vector b) {
28         return a.x * b.y - a.y * b.x; // 行列式值
29     }
30     // 向量长度
31     double length(Vector a) {
32         return sqrt(dot(a, a)); // 点积开方
33     }
34     // 两向量夹角 (弧度)
35     double angle(Vector a, Vector b) {
36         return acos(dot(a, b) / length(a) / length(b)); // 点积除以模长积
37     }
38     // 向量旋转 (逆时针旋转弧度rad)
39     Vector rotate(Vector a, double rad) {
40         return Vector(a.x * cos(rad) - a.y * sin(rad), a.x * sin(rad) + a
41             .y * cos(rad));
42     }
43     int main() {
44         Point p1(1, 0), p2(0, 1); // p1: 点1坐标(1,0), p2: 点2坐标(0,1)
45         Vector v1 = p1, v2 = p2; // v1: 向量1, v2: 向量2
46         cout << "点积: " << dot(v1, v2) << "\n"; // 输出0, 表示垂直
47         cout << "叉积: " << cross(v1, v2) << "\n"; // 输出1, 表示v2在v1逆
48             时针方向
49         return 0;
50     }

```

47.2 点到直线距离

解决问题: 给定一点和一条直线 (由两点确定), 求点到直线的距离。适用于碰撞检测、路径规划等场景。

题目描述

给定点 P 和直线 AB , 求点 P 到直线 AB 的距离。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标

```

```

5      Point(double x = 0, double y = 0) : x(x), y(y) {}
6  };
7  typedef Point Vector;
8  double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
9  double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
10 double length(Vector a) { return sqrt(dot(a, a)); }
11 // 点到直线距离：利用叉积的几何意义（平行四边形面积/底边长）
12 double pointToLine(Point p, Point a, Point b) {
13     Vector ap = p - a;    // ap: 向量从a指向p
14     Vector ab = b - a;    // ab: 向量从a指向b
15     return fabs(cross(ap, ab)) / length(ab); // 叉积绝对值 = 平行四边
        形面积，除以底边长得高
16 }
17 int main() {
18     Point p(0, 0); // p: 目标点坐标(0,0)
19     Point a(1, 1); // a: 直线上一点(1,1)
20     Point b(2, 2); // b: 直线上另一点(2,2)
21     double dist = pointToLine(p, a, b); // dist: 点到直线距离
22     cout << fixed << setprecision(2) << dist << "\n"; // 输出0.00（点
        在直线上）
23     return 0;
24 }

```

47.3 点到线段距离

解决问题：给定一点和一条线段，求点到线段的最短距离。适用于最近点查找、碰撞检测等场景。

题目描述

给定点 P 和线段 AB ，求点 P 到线段 AB 的最短距离。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      typedef Point Vector;
8      double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
9      double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }

```



```
10     double length(Vector a) { return sqrt(dot(a, a)); }
11     // 点到线段距离：需要判断投影点是否在线段上
12     double pointToSegment(Point p, Point a, Point b) {
13         Vector ap = p - a; // ap: 向量从a指向p
14         Vector ab = b - a; // ab: 向量从a指向b
15         Vector bp = p - b; // bp: 向量从b指向p
16         double t = dot(ap, ab) / dot(ab, ab); // t: p在直线AB上的投影参数
           // (0~1之间表示在线段上)
17         if (t < 0) return length(ap); // 投影在A点左侧，最近点为A
18         if (t > 1) return length(bp); // 投影在B点右侧，最近点为B
19         Point projection = a + ab * t; // projection: 投影点坐标
20         return length(p - projection); // 投影点在线段上，返回点到投影点
           的距离
21     }
22     int main() {
23         Point p(1, 1); // p: 目标点坐标(1,1)
24         Point a(0, 0); // a: 线段端点A(0,0)
25         Point b(2, 0); // b: 线段端点B(2,0)
26         double dist = pointToSegment(p, a, b); // dist: 点到线段距离
27         cout << fixed << setprecision(2) << dist << "\n"; // 输出1.00
28         return 0;
29     }
```

48 线段相交

48.1 判断线段相交（规范相交）

解决问题：判断两条线段是否相交（交点在线段内部，不包括端点）。适用于地图导航、电路布线等场景。

题目描述

给定两条线段 AB 和 CD ，判断它们是否规范相交（交点在内部，非端点）。

核心原理

利用叉积判断跨立：点 A 和 B 在线段 CD 两侧，且点 C 和 D 在线段 AB 两侧。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      typedef Point Vector;
8      double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9      // 判断点是否在线段上（包括端点）
10     bool onSegment(Point p, Point a, Point b) {
11         double minx = min(a.x, b.x), maxx = max(a.x, b.x); // minx: 线段x
12         // 坐标最小值, maxx: 线段x坐标最大值
13         double miny = min(a.y, b.y), maxy = max(a.y, b.y); // miny: 线段y
14         // 坐标最小值, maxy: 线段y坐标最大值
15         return cross(p - a, p - b) == 0 && p.x >= minx && p.x <= maxx &&
16             p.y >= miny && p.y <= maxy;
17     }
18     // 判断两线段是否相交（规范相交：交点在线段内部，不包括端点）
19     bool segmentsIntersect(Point a, Point b, Point c, Point d) {
20         double d1 = cross(b - a, c - a); // d1: 向量AB与AC的叉积
21         double d2 = cross(b - a, d - a); // d2: 向量AB与AD的叉积
22         double d3 = cross(d - c, a - c); // d3: 向量CD与CA的叉积
23         double d4 = cross(d - c, b - c); // d4: 向量CD与CB的叉积
24         // 规范相交：两线段互相跨立（叉积异号）
25         if (d1 * d2 < 0 && d3 * d4 < 0) return true;
26         // 非规范相交（端点相交）返回false，本函数只判断规范相交
27         return false;

```

```

25     }
26     int main() {
27         Point a(0, 0), b(2, 2); // a,b: 第一条线段端点
28         Point c(0, 2), d(2, 0); // c,d: 第二条线段端点
29         if (segmentsIntersect(a, b, c, d)) {
30             cout << "相交\n"; // 输出交集
31         } else {
32             cout << "不相交\n";
33         }
34         return 0;
35     }

```

48.2 判断线段相交（包括端点）

解决问题：判断两条线段是否相交（包括交点在端点的情况）。适用于多边形切割、图形学等场景。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     typedef Point Vector;
8     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9     double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
10    // 判断点是否在线段上（包括端点）
11    bool onSegment(Point p, Point a, Point b) {
12        double minx = min(a.x, b.x), maxx = max(a.x, b.x); // minx: 线段x
13        // 最小值, maxx: 线段x最大值
14        double miny = min(a.y, b.y), maxy = max(a.y, b.y); // miny: 线段y
15        // 最小值, maxy: 线段y最大值
16        return cross(p - a, p - b) == 0 && p.x >= minx && p.x <= maxx &&
17        p.y >= miny && p.y <= maxy;
18    }
19    // 判断两线段是否相交（包括端点在线上）
20    bool segmentsIntersectIncludeEnds(Point a, Point b, Point c, Point d
    ) {
21        double d1 = cross(b - a, c - a); // d1: AB与AC叉积
22        double d2 = cross(b - a, d - a); // d2: AB与AD叉积
23        double d3 = cross(d - c, a - c); // d3: CD与CA叉积

```

```

21     double d4 = cross(d - c, b - c); // d4: CD与CB叉积
22     // 规范相交: 互相跨立
23     if (d1 * d2 < 0 && d3 * d4 < 0) return true;
24     // 端点在线段上
25     if (onSegment(c, a, b)) return true;
26     if (onSegment(d, a, b)) return true;
27     if (onSegment(a, c, d)) return true;
28     if (onSegment(b, c, d)) return true;
29     return false;
30 }
31 int main() {
32     Point a(0, 0), b(2, 0); // a,b: 第一条线段端点
33     Point c(1, 0), d(3, 0); // c,d: 第二条线段端点
34     if (segmentsIntersectIncludeEnds(a, b, c, d)) {
35         cout << "相交\n"; // 输出相交 (共享端点)
36     } else {
37         cout << "不相交\n";
38     }
39     return 0;
40 }

```

48.3 求线段交点坐标

解决问题：给定两条相交线段，求它们的交点坐标。适用于几何计算、CAD 系统等场景。

核心原理

利用定比分点公式或参数方程求解。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     typedef Point Vector;
8     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9     // 求两直线交点 (使用参数方程, 要求不平行)
10    Point lineIntersection(Point a, Point b, Point c, Point d) {
11        Vector ab = b - a; // ab: 直线AB的方向向量

```

```
12     Vector cd = d - c; // cd: 直线CD的方向向量
13     double t = cross(c - a, cd) / cross(ab, cd); // t: 交点在线段AB上
           的参数
14     return a + ab * t; // 返回交点坐标
15 }
16 int main() {
17     Point a(0, 0), b(2, 2); // a,b: 第一条线段端点
18     Point c(0, 2), d(2, 0); // c,d: 第二条线段端点
19     Point inter = lineIntersection(a, b, c, d); // inter: 交点坐标
20     cout << fixed << setprecision(2) << inter.x << " " << inter.y <<
           "\n"; // 输出1.00 1.00
21     return 0;
22 }
```

49 凸包

49.1 Andrew 算法求凸包

解决问题：给定平面上一组点，求包含所有点的最小凸多边形（凸包）。适用于形状分析、碰撞检测、图像处理等场景。

题目描述

给定 n 个平面点，求它们的凸包（按逆时针顺序输出凸包顶点）。

核心原理

Andrew 算法：按 x 坐标排序后，分别构造上凸壳和下凸壳，使用单调栈维护。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6          bool operator<(const Point& p) const {
7              return x != p.x ? x < p.x : y < p.y; // 按x升序, x相同按y升序
8          }
9      };
10     typedef Point Vector;
11     double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
12     Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y -
13         b.y); }
14     // Andrew算法求凸包, 返回凸包顶点 (逆时针顺序)
15     vector<Point> convexHull(vector<Point>& pts) {
16         int n = pts.size(); // n: 点的总数
17         if (n <= 1) return pts;
18         sort(pts.begin(), pts.end()); // 按x坐标排序
19         vector<Point> hull(2 * n); // hull: 存储凸包顶点的数组 (临时)
20         int k = 0; // k: 当前凸包顶点数量
21         // 构造下凸壳 (从左到右)
22         for (int i = 0; i < n; i++) { // i: 当前处理的点索引
23             while (k >= 2 && cross(hull[k-1] - hull[k-2], pts[i] - hull[k-2]) <= 0) {
24                 k--; // 弹出不满足逆时针的点
25             }
26             hull[k++] = pts[i];
27         }
28     }

```

```

26     }
27     // 构造上凸壳（从右到左）
28     for (int i = n - 2, t = k + 1; i >= 0; i--) { // i: 当前处理的点索引
29         while (k >= t && cross(hull[k-1] - hull[k-2], pts[i] - hull[k-2]) <= 0) {
30             k--; // 弹出不满足逆时针的点
31         }
32         hull[k++] = pts[i];
33     }
34     hull.resize(k - 1); // 移除重复的起点
35     return hull;
36 }
37 int main() {
38     int n; // n: 点的数量
39     cin >> n;
40     vector<Point> pts(n); // pts: 存储所有点的数组
41     for (int i = 0; i < n; i++) {
42         cin >> pts[i].x >> pts[i].y;
43     }
44     vector<Point> hull = convexHull(pts); // hull: 凸包顶点列表
45     cout << "凸包顶点数: " << hull.size() << "\n";
46     for (auto& p : hull) {
47         cout << p.x << " " << p.y << "\n";
48     }
49     return 0;
50 }

```

49.2 判断点是否在凸多边形内

解决问题：给定一个凸多边形和一个点，判断点是否在多边形内部（包括边界）。适用于碰撞检测、地理信息系统等场景。

核心原理

利用叉积：点在所有边的左侧（逆时针多边形）则表示在内部。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标

```

```

5      Point(double x = 0, double y = 0) : x(x), y(y) {}
6  };
7  typedef Point Vector;
8  double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9  // 判断点是否在凸多边形内（多边形顶点逆时针顺序，包括边界）
10 bool pointInConvexPolygon(Point p, vector<Point>& poly) {
11     int n = poly.size(); // n: 多边形顶点数
12     for (int i = 0; i < n; i++) { // i: 当前边的起点索引
13         Vector edge = poly[(i+1)%n] - poly[i]; // edge: 当前边向量
14         Vector vp = p - poly[i]; // vp: 从当前顶点指向点p的向量
15         if (cross(edge, vp) < 0) return false; // 点在边的右侧，不在内部
16     }
17     return true; // 所有边都满足点在左侧或边上
18 }
19 int main() {
20     vector<Point> poly = {{0,0}, {10,0}, {10,10}, {0,10}}; // poly:
21     // 正方形凸包顶点
22     Point p(5, 5); // p: 待测试的点
23     if (pointInConvexPolygon(p, poly)) {
24         cout << "点在多边形内\n";
25     } else {
26         cout << "点在多边形外\n";
27     }
28     return 0;
}

```

49.3 判断点是否在任意多边形内（射线法）

解决问题：给定任意简单多边形（凸或凹）和一个点，判断点是否在多边形内部。适用于地图选点、区域判定等场景。

核心原理

从点发射一条射线，计算与多边形边的交点个数，奇数为内，偶数为外。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标

```



```

5     Point(double x = 0, double y = 0) : x(x), y(y) {}
6 };
7     double cross(Point a, Point b, Point c) {
8         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
9     }
10    // 判断点是否在线段上
11    bool onSegment(Point p, Point a, Point b) {
12        double minx = min(a.x, b.x), maxx = max(a.x, b.x);
13        double miny = min(a.y, b.y), maxy = max(a.y, b.y);
14        return cross(a, b, p) == 0 && p.x >= minx && p.x <= maxx && p.y
            >= miny && p.y <= maxy;
15    }
16    // 射线法判断点是否在多边形内
17    bool pointInPolygon(Point p, vector<Point>& poly) {
18        int n = poly.size(); // n: 多边形顶点数
19        int cnt = 0;          // cnt: 射线与多边形边的交点计数
20        for (int i = 0; i < n; i++) { // i: 当前边的起点索引
21            Point a = poly[i];        // a: 当前边起点
22            Point b = poly[(i+1)%n]; // b: 当前边终点
23            if (onSegment(p, a, b)) return true; // 点在边界上
24            // 判断射线（向右水平）是否与边相交
25            if ((a.y > p.y) != (b.y > p.y)) { // 边的两端点在射线的两侧（垂
                直方向）
26                double xIntersect = a.x + (p.y - a.y) * (b.x - a.x) / (b.y
                    - a.y); // 交点x坐标
27                if (xIntersect > p.x) cnt++; // 交点在射线右侧（正方向）
28            }
29        }
30        return cnt % 2 == 1; // 奇数在内，偶数在外
31    }
32    int main() {
33        vector<Point> poly = {{0,0}, {10,0}, {10,10}, {0,10}}; // poly:
            多边形顶点
34        Point p(5, 5); // p: 待测试的点
35        if (pointInPolygon(p, poly)) {
36            cout << "点在多边形内\n";
37        } else {
38            cout << "点在多边形外\n";
39        }
40        return 0;

```

41

}

50 多边形面积

50.1 计算多边形面积（鞋带公式）

解决问题：给定多边形顶点（按顺序），计算多边形的面积。适用于土地面积测量、图形学等场景。

核心原理

鞋带公式（Shoelace formula）： $S = \frac{1}{2} |\sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i)|$ 。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      // 计算多边形面积（顶点按顺序给出，顺时针或逆时针均可）
8      double polygonArea(vector<Point>& poly) {
9          int n = poly.size(); // n: 多边形顶点数
10         double area = 0; // area: 面积（两倍值，最后除以2）
11         for (int i = 0; i < n; i++) { // i: 当前顶点索引
12             int j = (i + 1) % n; // j: 下一个顶点索引
13             area += poly[i].x * poly[j].y - poly[j].x * poly[i].y; // 叉积
14                                     累加
15         }
16         return fabs(area) / 2.0; // 取绝对值后除以2得面积
17     }
18     int main() {
19         vector<Point> poly = {{0,0}, {3,0}, {3,4}, {0,4}}; // poly: 矩形
20                                     顶点
21         double area = polygonArea(poly); // area: 多边形面积
22         cout << fixed << setprecision(2) << area << "\n"; // 输出12.00
23         return 0;
24     }

```

51 最近点对

51.1 分治法求最近点对距离

解决问题：给定平面上一组点，找出距离最近的两个点。适用于聚类分析、碰撞检测等场景。

题目描述

给定 n 个平面点，找出欧几里得距离最近的两个点，输出最短距离。

核心原理

分治法：按 x 坐标排序后分治，合并时只检查分界线两侧 d 范围内的点。

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6          bool operator<(const Point& p) const {
7              return x != p.x ? x < p.x : y < p.y;
8          }
9      };
10     double dist(Point a, Point b) {
11         double dx = a.x - b.x, dy = a.y - b.y; // dx: x坐标差, dy: y坐标差
12         return sqrt(dx * dx + dy * dy); // 欧几里得距离
13     }
14     // 分治法求最近点对距离
15     double closestPair(vector<Point>& pts, int l, int r) {
16         if (r - l <= 1) return 1e20; // 点数少于2, 返回无穷大
17         int mid = (l + r) / 2; // mid: 中间分割点索引
18         double d = min(closestPair(pts, l, mid), closestPair(pts, mid, r)
19             ); // d: 左右两侧的最小距离
20         // 收集分界线两侧d范围内的点
21         vector<Point> strip; // strip: 分界线附近的点集
22         double midX = pts[mid].x; // midX: 中间点的x坐标
23         for (int i = l; i <= r; i++) {
24             if (fabs(pts[i].x - midX) < d) { // x坐标差小于d才可能更近
25                 strip.push_back(pts[i]);
26             }
27         }
28     }
```

```
27 // 按y坐标排序
28 sort(strip.begin(), strip.end(), [](Point a, Point b) { return a.
    y < b.y; });
29 // 检查带内点对
30 for (int i = 0; i < strip.size(); i++) {
31     for (int j = i + 1; j < strip.size() && strip[j].y - strip[i].
        y < d; j++) {
32         d = min(d, dist(strip[i], strip[j])); // 更新最小距离
33     }
34 }
35 return d;
36 }
37 int main() {
38     int n; // n: 点的数量
39     cin >> n;
40     vector<Point> pts(n); // pts: 存储所有点的数组
41     for (int i = 0; i < n; i++) {
42         cin >> pts[i].x >> pts[i].y;
43     }
44     sort(pts.begin(), pts.end()); // 按x坐标排序
45     double ans = closestPair(pts, 0, n - 1); // ans: 最近点对距离
46     cout << fixed << setprecision(4) << ans << "\n";
47     return 0;
48 }
```

52 旋转卡壳

52.1 凸包直径（最远点对）

解决问题：给定凸多边形，找出距离最远的两个点（凸包直径）。适用于最小包围盒计算、形状分析等场景。

核心原理

旋转卡壳：利用对踵点对，旋转平行线扫描所有顶点。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      typedef Point Vector;
8      double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
9      double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
10     double dist2(Point a, Point b) {
11         double dx = a.x - b.x, dy = a.y - b.y; // dx: x坐标差, dy: y坐标差
12         return dx * dx + dy * dy; // 平方距离避免开方
13     }
14     Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y -
15         b.y); }
16     // 旋转卡壳求凸包直径（最远点对距离）
17     double rotatingCalipers(vector<Point>& hull) {
18         int n = hull.size(); // n: 凸包顶点数
19         if (n == 2) return dist2(hull[0], hull[1]); // 只有两个点
20         int k = 1; // k: 对踵点索引
21         double ans = 0; // ans: 最大距离平方
22         for (int i = 0; i < n; i++) { // i: 当前边起点索引
23             int j = (i + 1) % n; // j: 当前边终点索引
24             // 旋转找到距离当前边最远的点
25             while (cross(hull[j] - hull[i], hull[(k+1)%n] - hull[i]) >
26                 cross(hull[j] - hull[i], hull[k] - hull[i])) {
27                 k = (k + 1) % n; // 更新对踵点
28             }
29             ans = max(ans, dist2(hull[i], hull[k])); // 更新距离
30             ans = max(ans, dist2(hull[j], hull[k])); // 更新距离
31         }
32     }

```

```
30     return sqrt(ans); // 返回实际距离
31 }
32 int main() {
33     vector<Point> hull = {{0,0}, {5,0}, {5,5}, {0,5}}; // hull: 正方形凸包
34     double diameter = rotatingCalipers(hull); // diameter: 凸包直径
35     cout << fixed << setprecision(2) << diameter << "\n"; // 输出7.07
36     // (对角线长度)
37     return 0;
}
```

53 圆的相关计算

53.1 两圆交点

解决问题：给定两个圆，求它们的交点坐标。适用于碰撞检测、几何作图等场景。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      struct Point {
4          double x, y; // x: 横坐标, y: 纵坐标
5          Point(double x = 0, double y = 0) : x(x), y(y) {}
6      };
7      struct Circle {
8          Point c; // c: 圆心坐标
9          double r; // r: 半径
10     };
11     double dist(Point a, Point b) {
12         double dx = a.x - b.x, dy = a.y - b.y;
13         return sqrt(dx * dx + dy * dy);
14     }
15     // 求两圆交点
16     vector<Point> circleIntersection(Circle c1, Circle c2) {
17         double d = dist(c1.c, c2.c); // d: 圆心距
18         vector<Point> res;           // res: 存储交点的数组
19         if (d > c1.r + c2.r + 1e-9 || d < fabs(c1.r - c2.r) - 1e-9)
20             return res; // 相离或内含
21         if (fabs(d) < 1e-9) return res; // 同心圆
22         double a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 * d); // a:
23         // 交点到c1.c的距离在连线上的投影
24         double h = sqrt(fabs(c1.r * c1.r - a * a)); // h: 交点到连
25         // 线的垂直距离
26         Point dir = {(c2.c.x - c1.c.x) / d, (c2.c.y - c1.c.y) / d}; //
27         // dir: 从c1.c指向c2.c的单位向量
28         Point mid = {c1.c.x + dir.x * a, c1.c.y + dir.y * a}; // mid: 连
29         // 线与弦的交点
30         Point perp = {-dir.y, dir.x}; // perp: 垂直
31         // 于dir的单位向量
32         if (fabs(h) < 1e-9) { // 一个交点 (相切)
33             res.push_back(mid);
34         } else { // 两个交点
35             res.push_back({mid.x + perp.x * h, mid.y + perp.y * h});
36         }
37     }

```



```

30         res.push_back({mid.x - perp.x * h, mid.y - perp.y * h});
31     }
32     return res;
33 }
34 int main() {
35     Circle c1 = {{0,0}, 5}; // c1: 第一个圆, 圆心(0,0)半径5
36     Circle c2 = {{8,0}, 5}; // c2: 第二个圆, 圆心(8,0)半径5
37     vector<Point> inter = circleIntersection(c1, c2); // inter: 交点
38                                     坐标列表
39     for (auto& p : inter) {
40         cout << fixed << setprecision(2) << p.x << " " << p.y << "\n";
41     }
42     return 0;
}

```

53.2 圆与直线交点

解决问题：给定圆和直线，求它们的交点。适用于射线检测、碰撞判定等场景。

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     struct Point {
4         double x, y; // x: 横坐标, y: 纵坐标
5         Point(double x = 0, double y = 0) : x(x), y(y) {}
6     };
7     typedef Point Vector;
8     struct Circle {
9         Point c; // c: 圆心
10        double r; // r: 半径
11    };
12    double dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y; }
13    double cross(Vector a, Vector b) { return a.x * b.y - a.y * b.x; }
14    double length(Vector a) { return sqrt(dot(a, a)); }
15    Vector operator+(Point a, Vector b) { return Vector(a.x + b.x, a.y +
16        b.y); }
17    Vector operator-(Point a, Point b) { return Vector(a.x - b.x, a.y -
18        b.y); }
19    Vector operator*(Vector a, double t) { return Vector(a.x * t, a.y *
20        t); }
21    // 求直线与圆的交点（直线由两点确定）

```

```
19     vector<Point> lineCircleIntersection(Point a, Point b, Circle cir) {
20         Vector ab = b - a;          // ab: 直线方向向量
21         Vector ac = cir.c - a;      // ac: 从a指向圆心的向量
22         double t = dot(ac, ab) / dot(ab, ab); // t: 投影参数
23         Point foot = a + ab * t;     // foot: 圆心在直线上的垂足
24         double d = length(cir.c - foot); // d: 圆心到直线的距离
25         vector<Point> res;           // res: 存储交点
26         if (d > cir.r + 1e-9) return res; // 相离
27         if (fabs(d - cir.r) < 1e-9) { // 相切
28             res.push_back(foot);
29             return res;
30         }
31         double h = sqrt(cir.r * cir.r - d * d); // h: 弦长的一半
32         Vector dir = ab * (1.0 / length(ab)); // dir: 直线单位方向向量
33         res.push_back(foot + dir * h);
34         res.push_back(foot - dir * h);
35         return res;
36     }
37     int main() {
38         Circle cir = {{0,0}, 5}; // cir: 圆心(0,0)半径5的圆
39         Point a(-10, 5);         // a: 直线上一点
40         Point b(10, 5);          // b: 直线上另一点
41         vector<Point> inter = lineCircleIntersection(a, b, cir); // inter
                                // : 交点
42         for (auto& p : inter) {
43             cout << fixed << setprecision(2) << p.x << " " << p.y << "\n";
44         }
45         return 0;
46     }
```

54 最小圆覆盖

54.1 随机增量法求最小圆覆盖

解决问题：给定平面上一组点，求覆盖所有点的最小圆。适用于聚类分析、碰撞检测等场景。

核心原理

随机增量法：随机打乱点顺序，维护当前最小圆，如果新点在圆外则更新圆。

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const double eps = 1e-9;
4      struct Point {
5          double x, y; // x: 横坐标, y: 纵坐标
6          Point(double x = 0, double y = 0) : x(x), y(y) {}
7      };
8      struct Circle {
9          Point c; // c: 圆心
10         double r; // r: 半径
11         Circle(Point c = Point(), double r = 0) : c(c), r(r) {}
12     };
13     double dist(Point a, Point b) {
14         double dx = a.x - b.x, dy = a.y - b.y;
15         return sqrt(dx * dx + dy * dy);
16     }
17     // 三点定圆（外接圆）
18     Circle circleFromThreePoints(Point a, Point b, Point c) {
19         double D = 2 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.
20             y - b.y));
21         double ux = ((a.x*a.x + a.y*a.y) * (b.y - c.y) + (b.x*b.x + b.y*b
22             .y) * (c.y - a.y) + (c.x*c.x + c.y*c.y) * (a.y - b.y)) / D;
23         double uy = ((a.x*a.x + a.y*a.y) * (c.x - b.x) + (b.x*b.x + b.y*b
24             .y) * (a.x - c.x) + (c.x*c.x + c.y*c.y) * (b.x - a.x)) / D;
25         Point center(ux, uy); // center: 圆心坐标
26         return Circle(center, dist(center, a));
27     }
28     // 两点定圆（以线段为直径）
29     Circle circleFromTwoPoints(Point a, Point b) {
30         Point center((a.x + b.x) / 2, (a.y + b.y) / 2); // center: 圆心
31         // (中点)

```

```

28         return Circle(center, dist(a, b) / 2);
29     }
30     // 随机增量法求最小圆覆盖
31     Circle minEnclosingCircle(vector<Point>& pts) {
32         int n = pts.size(); // n: 点的数量
33         random_shuffle(pts.begin(), pts.end()); // 随机打乱顺序
34         Circle cir(pts[0], 0); // cir: 当前最小圆, 初始为第一个点
35         for (int i = 1; i < n; i++) { // i: 当前处理的点索引
36             if (dist(pts[i], cir.c) <= cir.r + eps) continue; // 点在圆
37                 // 内, 无需更新
38             cir = Circle(pts[i], 0); // 新圆以当前点为圆心, 半径为0
39             for (int j = 0; j < i; j++) { // j: 第二个点索引
40                 if (dist(pts[j], cir.c) <= cir.r + eps) continue;
41                 cir = circleFromTwoPoints(pts[i], pts[j]); // 以i,j为直径作
42                     // 圆
43                 for (int k = 0; k < j; k++) { // k: 第三个点索引
44                     if (dist(pts[k], cir.c) <= cir.r + eps) continue;
45                     cir = circleFromThreePoints(pts[i], pts[j], pts[k]); //
46                         // 三点定圆
47                 }
48             }
49         }
50         return cir;
51     }
52     int main() {
53         int n; // n: 点的数量
54         cin >> n;
55         vector<Point> pts(n); // pts: 存储所有点的数组
56         for (int i = 0; i < n; i++) {
57             cin >> pts[i].x >> pts[i].y;
58         }
59         Circle cir = minEnclosingCircle(pts); // cir: 最小覆盖圆
60         cout << fixed << setprecision(4);
61         cout << "圆心: (" << cir.c.x << ", " << cir.c.y << ")\n";
62         cout << "半径: " << cir.r << "\n";
63         return 0;
64     }

```

55 计算几何总结

计算几何解题步骤

1. 定义点、向量结构体，重载基本运算符
2. 实现点积、叉积等基础函数
3. 根据问题选择合适的算法（凸包、相交、面积等）
4. 注意精度处理（eps 容差）
5. 边界条件判断（共线、重合、退化为点等）

精度处理技巧

- 使用 `const double eps = 1e-9` 作为误差阈值
- 判断相等: `fabs(a - b) < eps`
- 判断大于: `a > b + eps`
- 判断小于: `a < b - eps`
- 输出时使用 `fixed << setprecision(8)` 控制精度

所有代码均已测试，可直接复制运行。

持续更新中——保持练习，保持优秀！

算法类型	核心思想	时间复杂度	典型应用场景
DFS/BFS	递归/队列遍历	$O(n + m)$	连通性、路径存在性、二分图判定
Dijkstra	贪心 + 松弛	$O(m \log n)$	非负权单源最短路、网络延迟计算
Bellman-Ford/SPFA	全边松弛 + 负环检测	$O(nm)$ / 平均 $O(km)$	含负权最短路、差分约束
Floyd	动态规划中间点	$O(n^3)$	全源最短路、传递闭包
Kruskal	边排序 + 并查集	$O(m \log m)$	稀疏图最小生成树、网络建设
Prim	最小边扩展	$O(n^2)$ 或 $O(m \log n)$	稠密图最小生成树
拓扑排序	BFS+ 入度	$O(n + m)$	任务调度、依赖关系、DAG 判环
Tarjan(SCC)	DFS+ 追溯 low	$O(n + m)$	有向图强连通分量、缩点成 DAG
匈牙利算法	DFS 增广路	$O(nm)$	二分图最大匹配、任务分配
Dinic	BFS 分层 + DFS 增广	$O(n^2m)$	最大流、最小割、二分图匹配
费用流	最短路增广	$O(F \cdot nm)$	最小费用最大流、运输规划
倍增 LCA	二进制上跳	$O((n + q) \log n)$	树上查询、路径距离、树上差分
Hierholzer	DFS 删边	$O(n + m)$	欧拉回路/路径、一笔画
Tarjan 桥/割点	DFS 追溯 low	$O(n + m)$	网络脆弱点/边分析

类型	核心思想	时间复杂度	典型应用
KMP	部分匹配表 (next)	$O(n + m)$	单模式匹配、循环节
Trie	多叉树存储前缀	$O(L)$ 插入/查询	单词集合、异或极值
Manacher	回文半径对称扩展	$O(n)$	最长回文子串
后缀数组	后缀排序 + height	$O(n \log n)$	不同子串、LCP
Z 算法	Z 数组计算	$O(n)$	字符串匹配
AC 自动机	Trie+fail 指针	$O(n + kL)$	多模式匹配
回文自动机	回文后缀 fail 链	$O(n)$	本质不同回文
后缀自动机	自动机状态 + 后缀链接	$O(n)$	子串统计、LCS
最小表示法	双指针比较	$O(n)$	循环同构最小串
字符串哈希	哈希值快速比较	$O(1)$ 查询	子串相等判定

类型	核心思想	时间复杂度	典型应用
点积/叉积	向量运算基础	$O(1)$	角度判断、面积计算、方向判断
线段相交	跨立实验	$O(1)$	碰撞检测、地图导航
凸包	Andrew/ Graham 扫描	$O(n \log n)$	形状分析、图像处理
点定位	射线法/叉积法	$O(n)$	地理信息系统、区域判定
多边形面积	鞋带公式	$O(n)$	土地测量、图形学
最近点对	分治法	$O(n \log n)$	聚类分析、碰撞检测
旋转卡壳	对踵点扫描	$O(n)$	凸包直径、最小宽度
半平面交	极角排序 + 双端队列	$O(n \log n)$	线性规划、可行域求解
圆相关	几何关系推导	$O(1)$	碰撞检测、路径规划
最小圆覆盖	随机增量法	$O(n)$ 期望	聚类、包围盒计算